BCSE205L

Computer Architecture and Organization

Module 3 – Instruction Sets and Control Unit

Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

Dr. Erapaneni Gayatri

Assistant Professor

School of Computer Science and Engineering

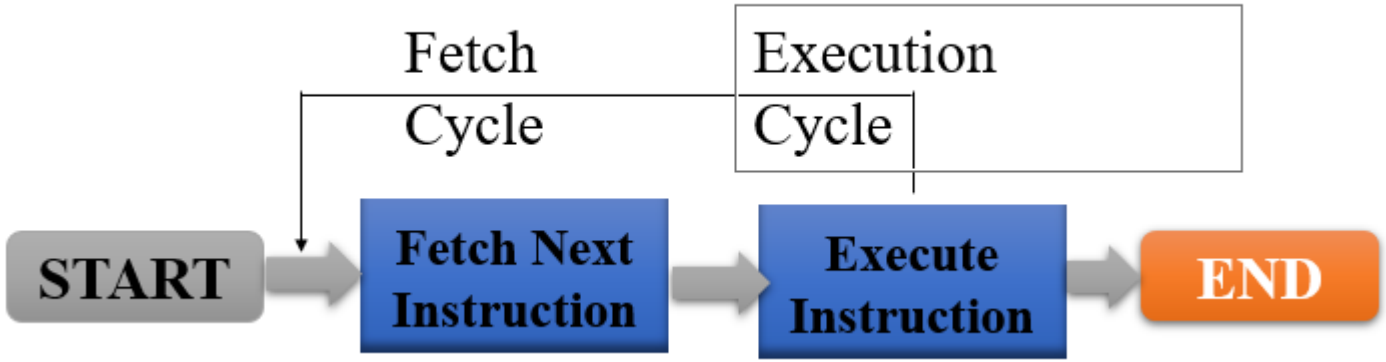
Vellore Institute of Technology, Vellore

Computer Instructions: Instruction sets

- **Instruction**
  - Is a statement by which the operation of CPU is determined.
  - These instructions referred as “Machine instructions or computer Instructions”
- **Elements of Machine instruction**
  - Operation code
  - Source operand reference
  - Result operand Reference
  - Next instruction reference

Computer Instructions: Instruction sets

- **Program**
  - is sequence of instructions which operates on data to perform certain tasks.
- **Instruction**
  - Is a statement by which the operation of CPU is determined
- **Instruction Cycle**



Instruction Set Architecture

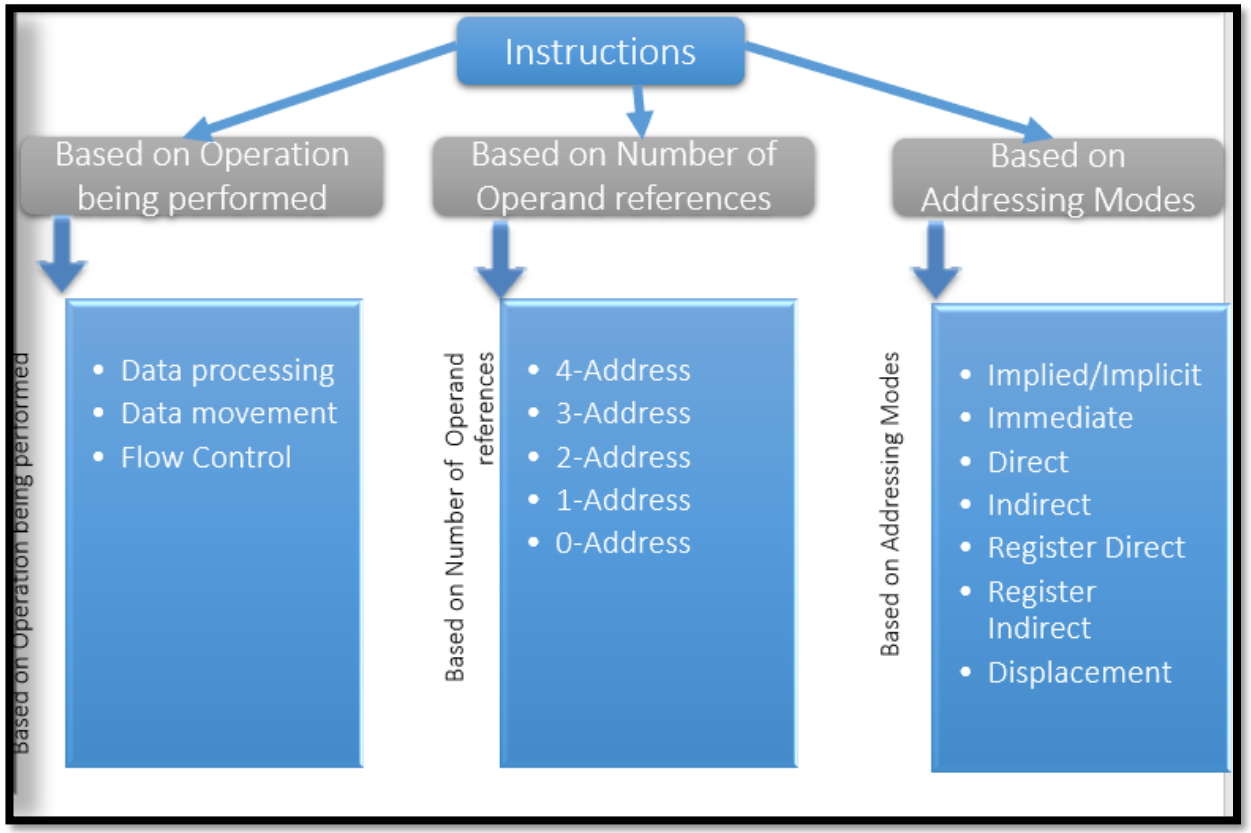
- Instruction Set Architecture (ISA) - defines how the CPU is controlled by the software
- ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done
- It defines the supported data types, the registers, how the hardware manages main memory, key features (such as virtual memory), which instructions a microprocessor can execute, and the input/output model of multiple ISA implementations

Instruction formats

- Each instruction is represented by sequence of bits
- The instruction is divided into two fields
  - Opcode field
  - Operand field
- This operand field further divided into one to four fields.
- This layout of the instruction is known as the “Instruction Format”

Instruction Set category

- Instruction Set is categorized into types based on
  - Operation performed
  - Number of operand addresses
  - Addressing modes.



Instruction Set category

- **Based on Operation being performed**
- Data movement / Transfer → Move data from a memory location or register to another memory location or register without changing its form.
- Different data transfers:
  - Memory ↔ processor registers
  - Processor registers ↔ input or output
  - Processor register ↔ processor register
- **Load**: transfer from memory to a processor register, usually an AC (*memory read*)
- **Store**: transfer from a processor register into memory (*memory write*)
- **Move**: transfer from one register to another register
- **Exchange**: swap information between two registers or a register and a memory word
- **Input/Output**: transfer data among processor registers and input/output device (I/O instructions)
- **Push**: transfer data between processor registers and a memory stack
- **Pop** : transfer data from stack to processor registers.

Instruction Set category

- **Based on Operation being performed**
- Data processing /Manipulation→ Arithmetic and logic (ALU) instructions - Changes the form of one or more operands to produce a result stored in another location

    I. Arithmetic,
    II. Logical and bit manipulation,
    III. Shift Instruction

Arithmetic instructions:

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with Carry	ADDC
Subtract with Borrow	SUBB
Negate(2's Complement)	NEG

Instruction Set category

- **Based on Operation being performed**

Logical and Bit Manipulation

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Instruction Set category

- **Based on Operation being performed**

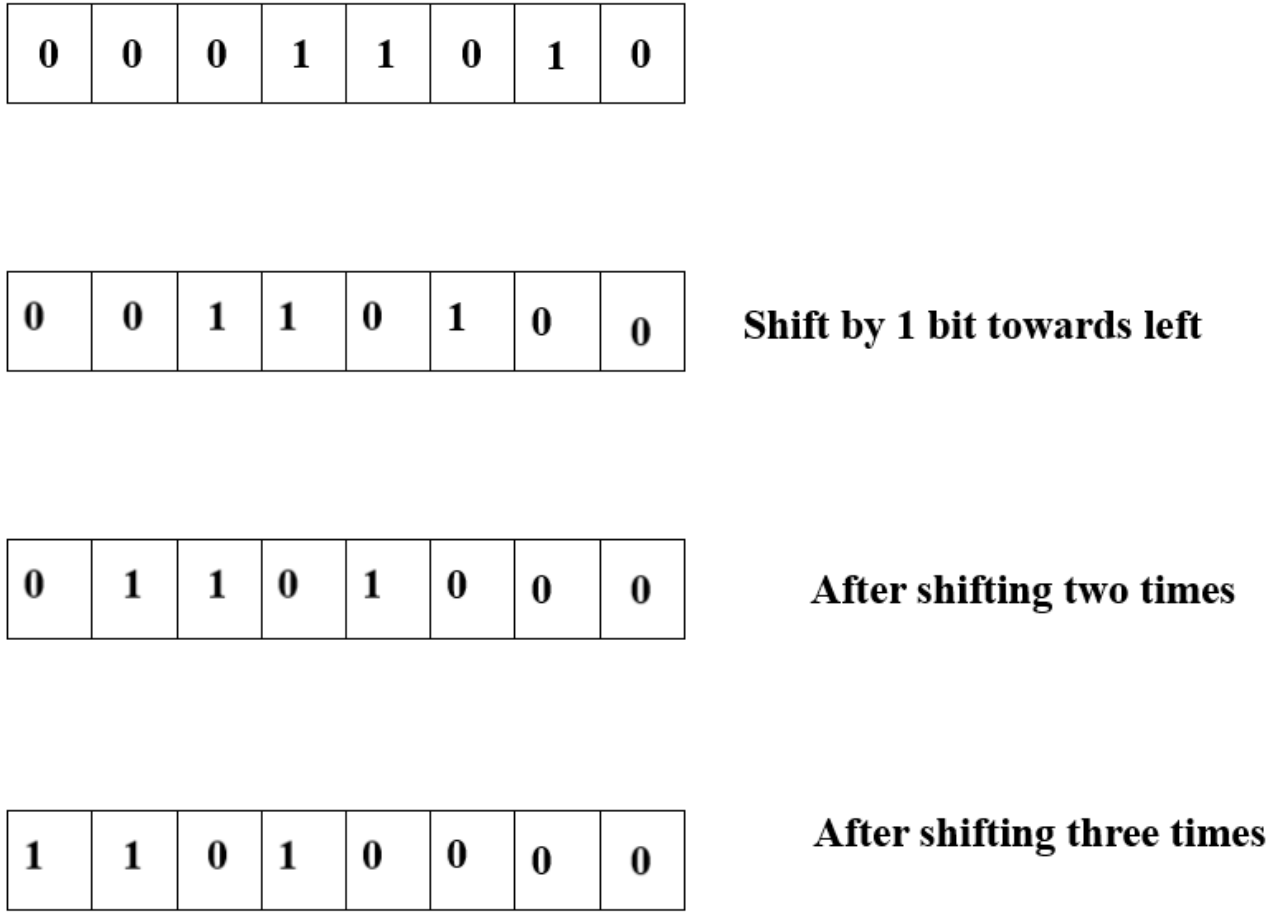
Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right thru carry	RORC
Rotate left thru carry	ROLC

Instruction Set category

- **Based on Operation being performed**

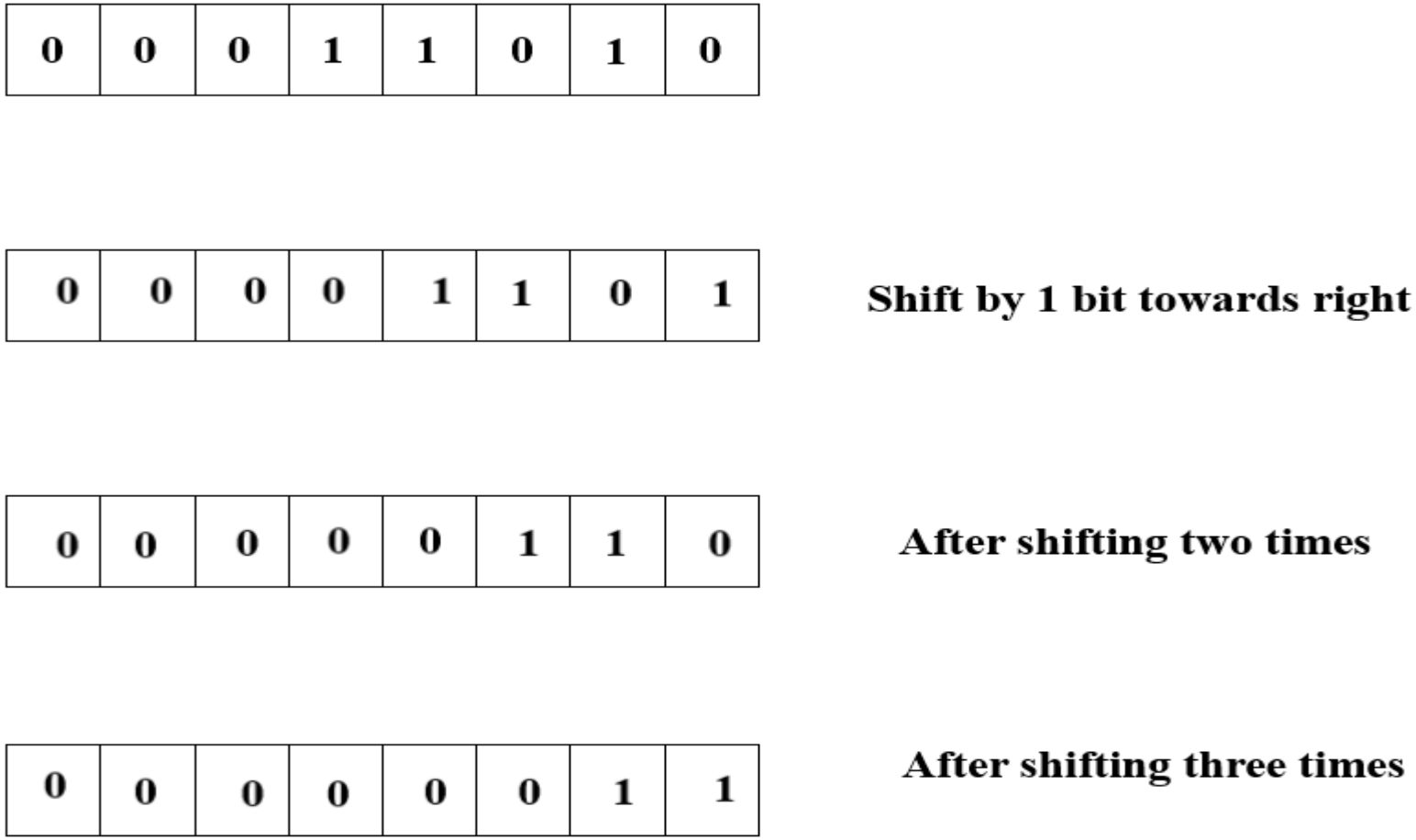
Logical shift left



Instruction Set category

- **Based on Operation being performed**

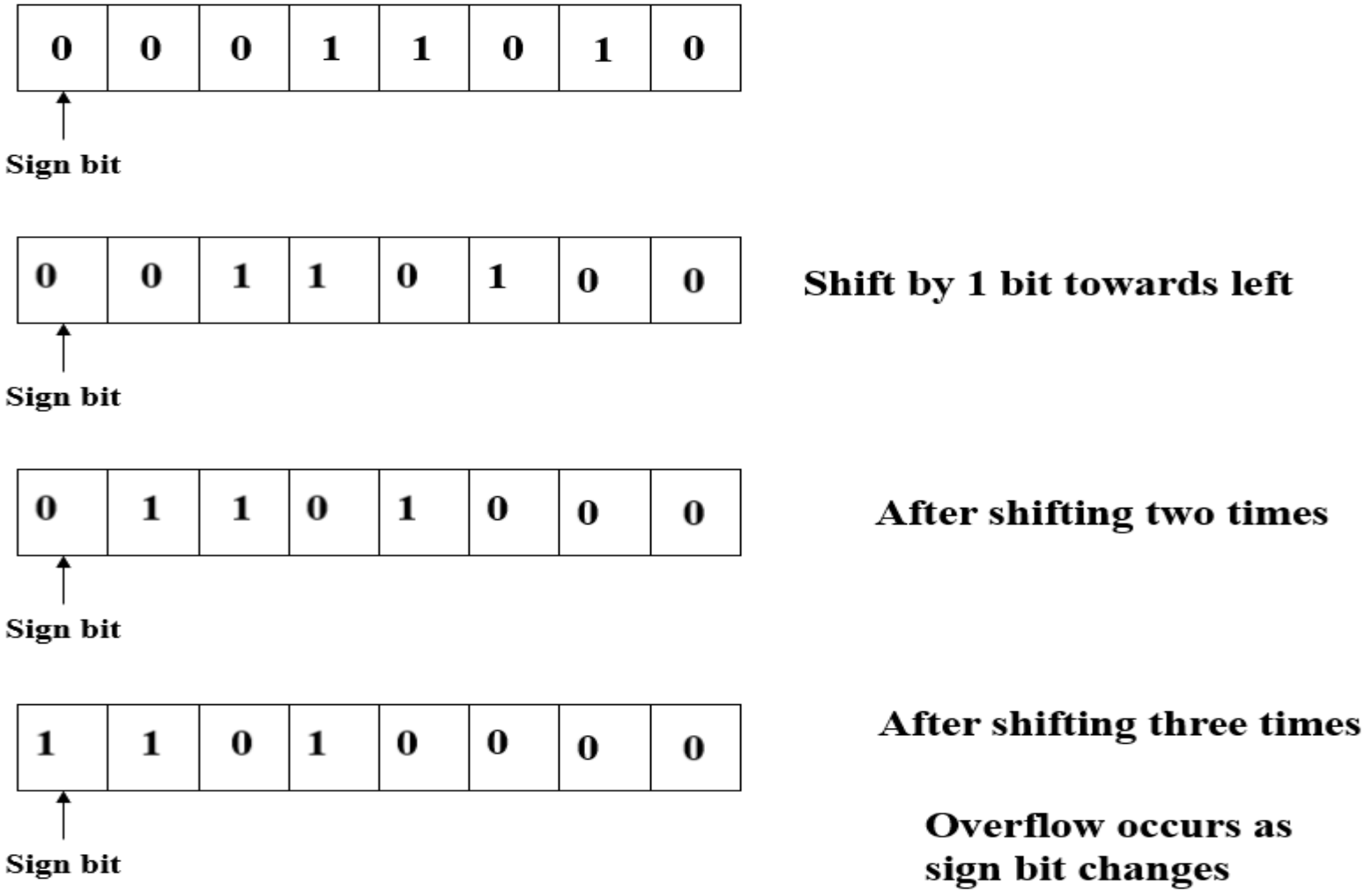
Logical shift Right



Instruction Set category

- **Based on Operation being performed**

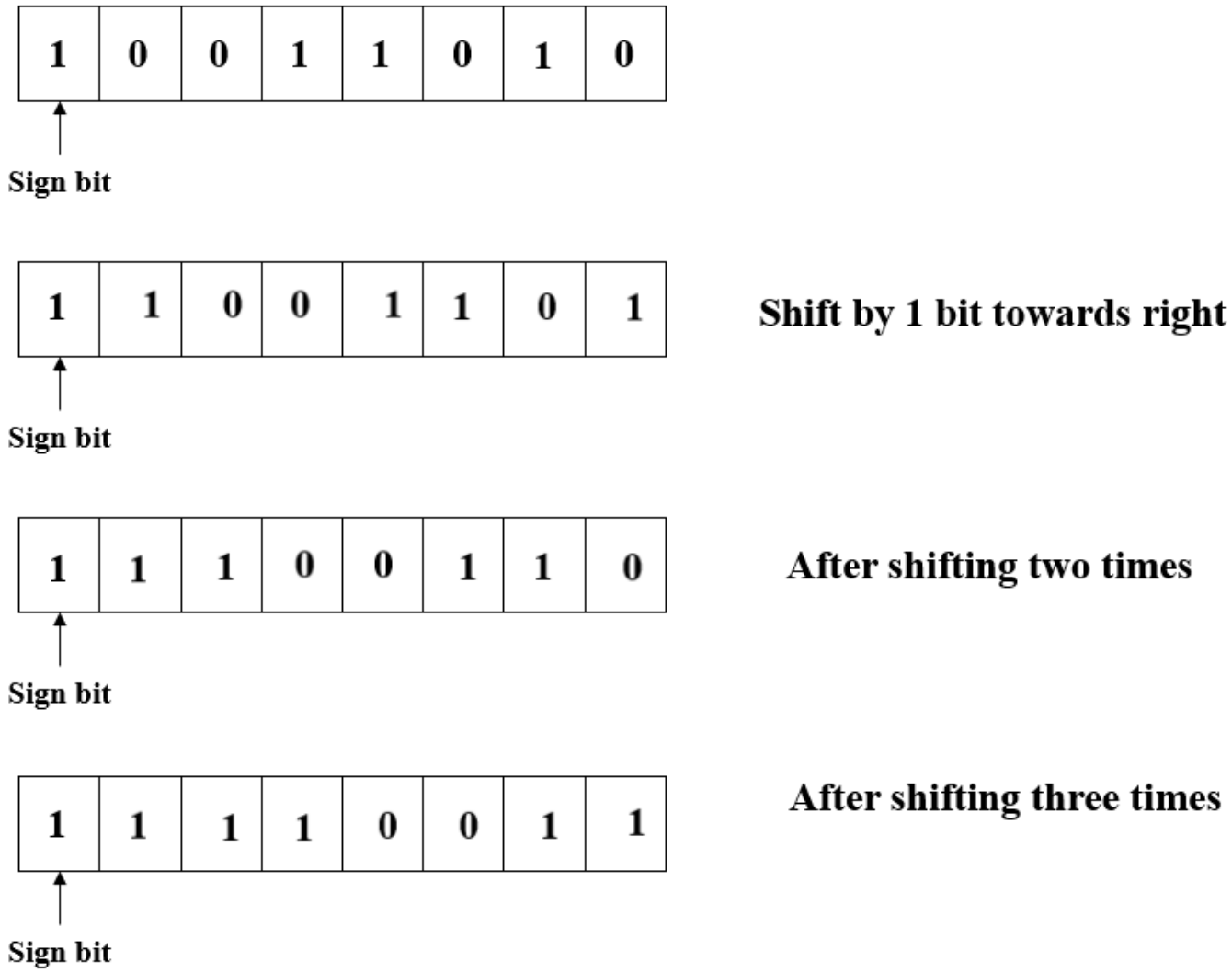
Arithmetic shift left



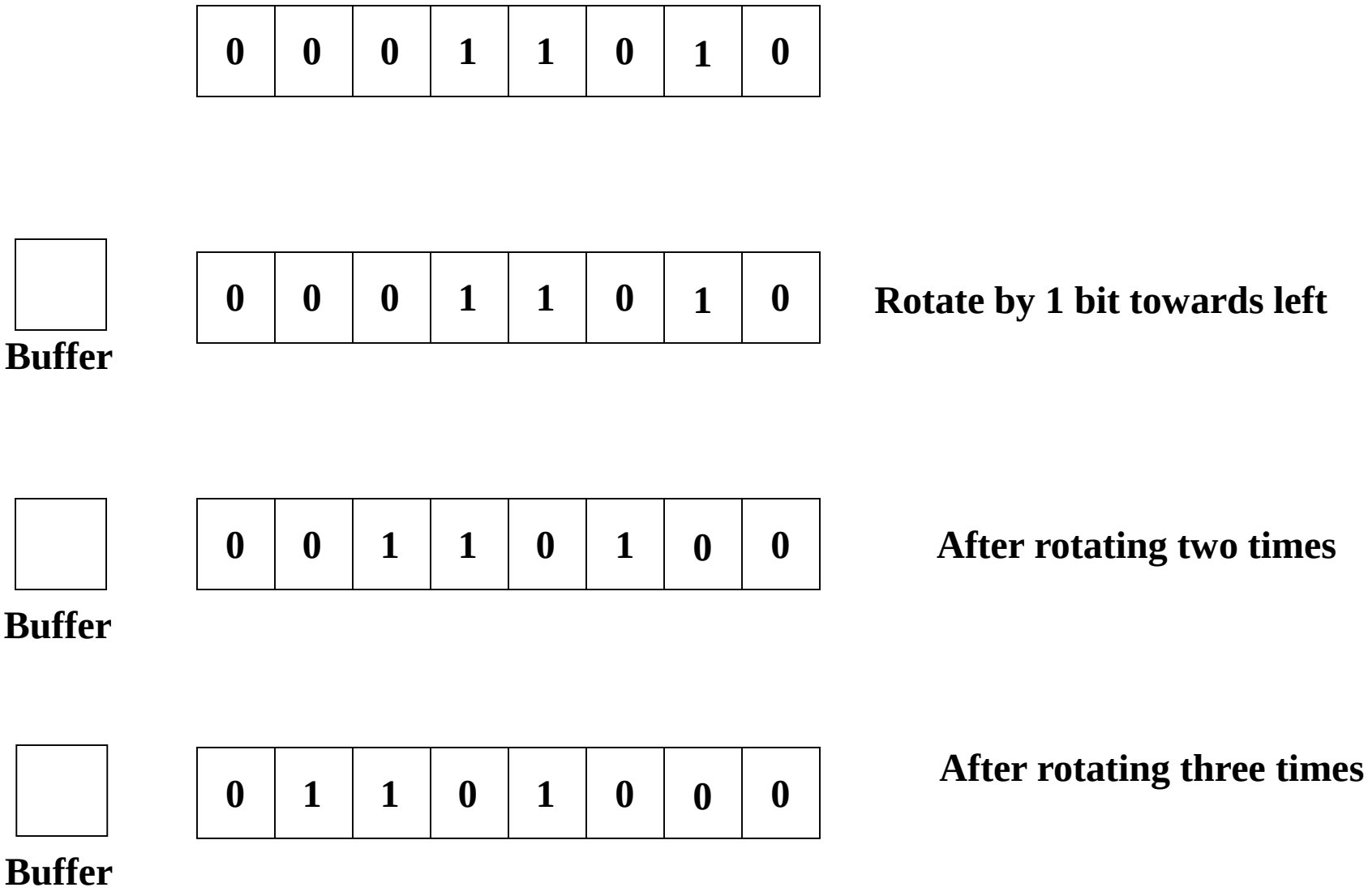
Instruction Set category

- **Based on Operation being performed**

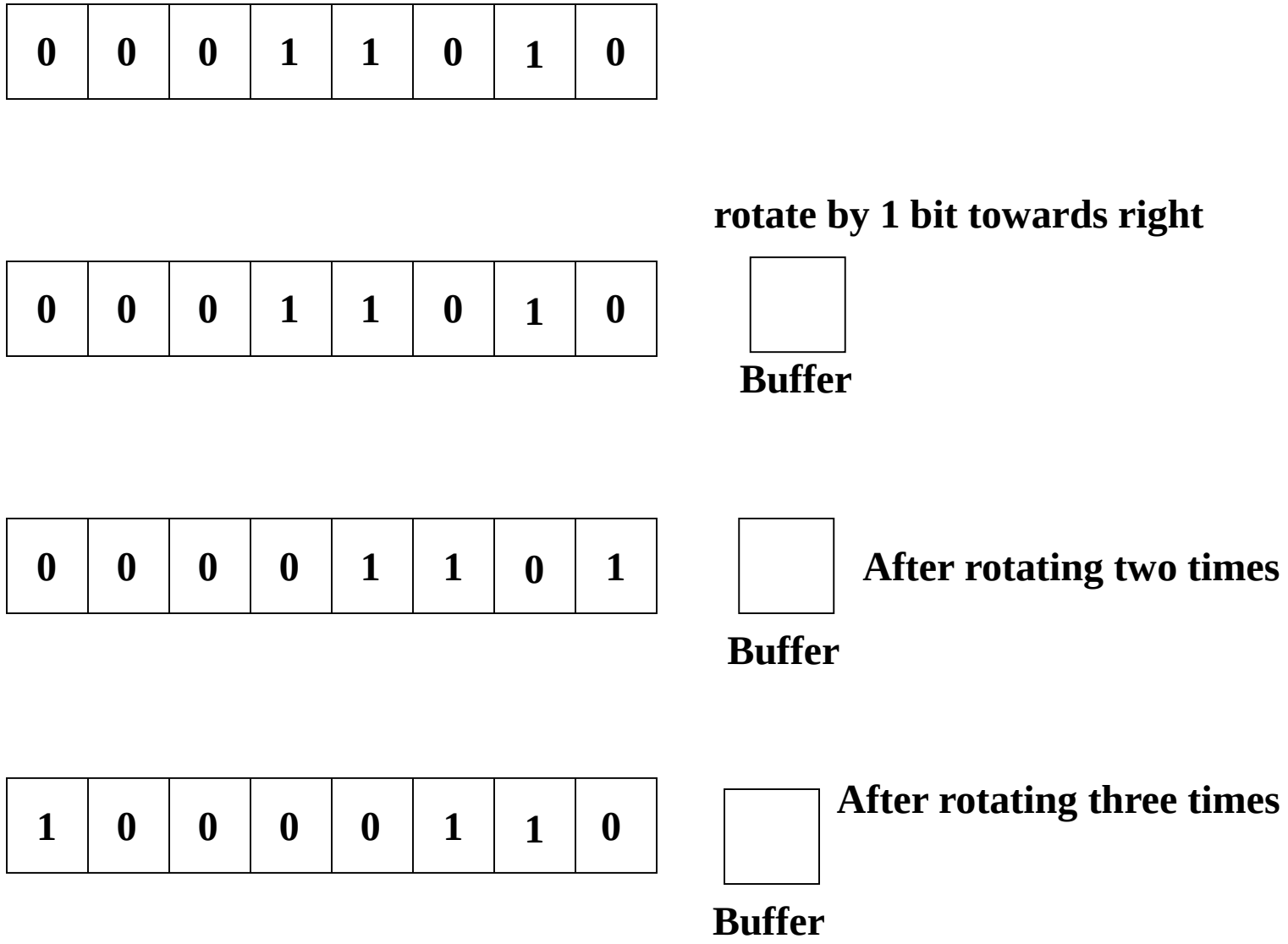
Arithmetic shift Right



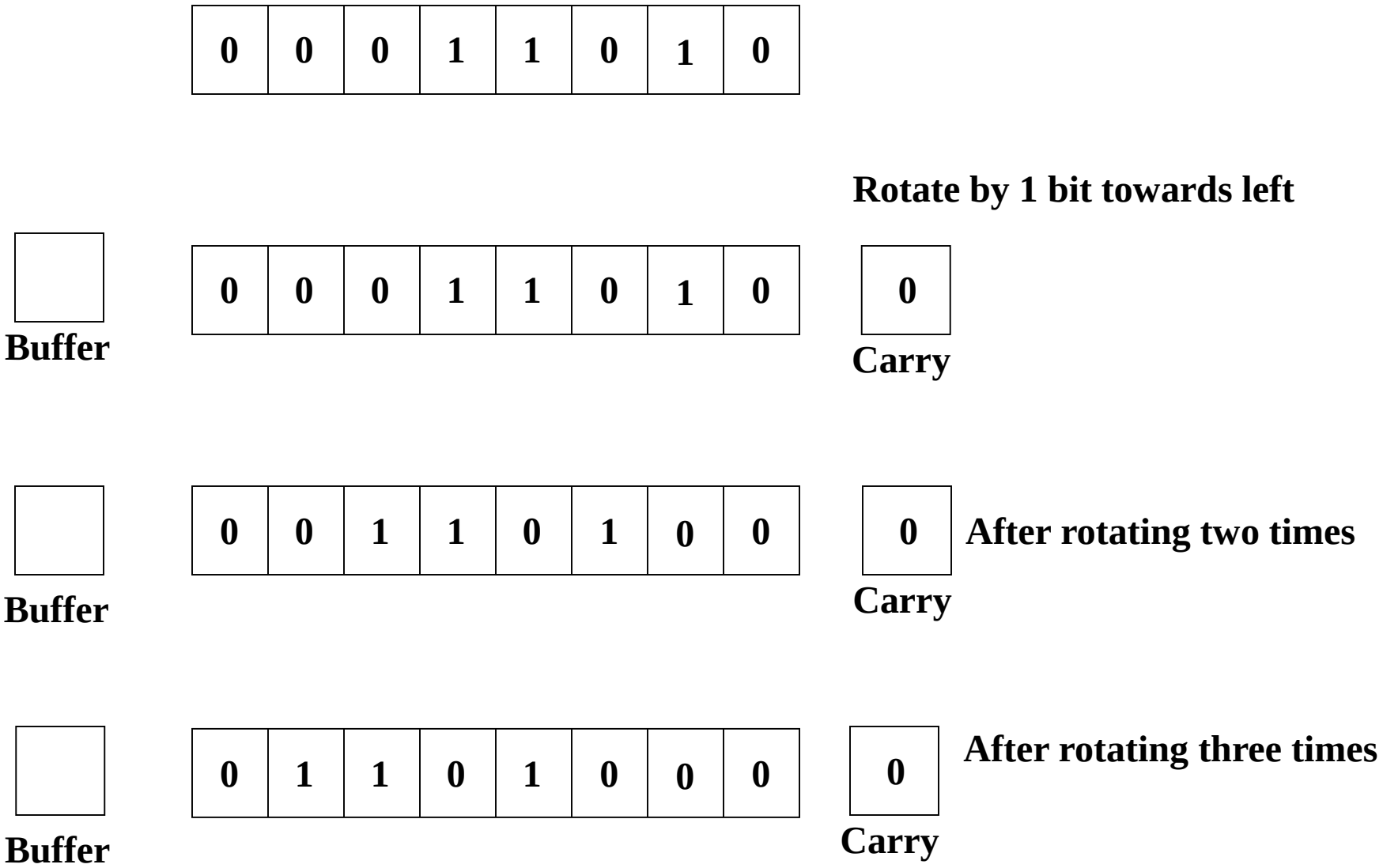
- **Rotate left**


0 0 0 1 1 0 1 0
Buffer
0 0 0 1 1 0 1 0
Rotate by 1 bit towards left
Buffer
0 0 1 1 0 1 0 0
After rotating two times
Buffer
0 1 1 0 1 0 0 0
After rotating three times

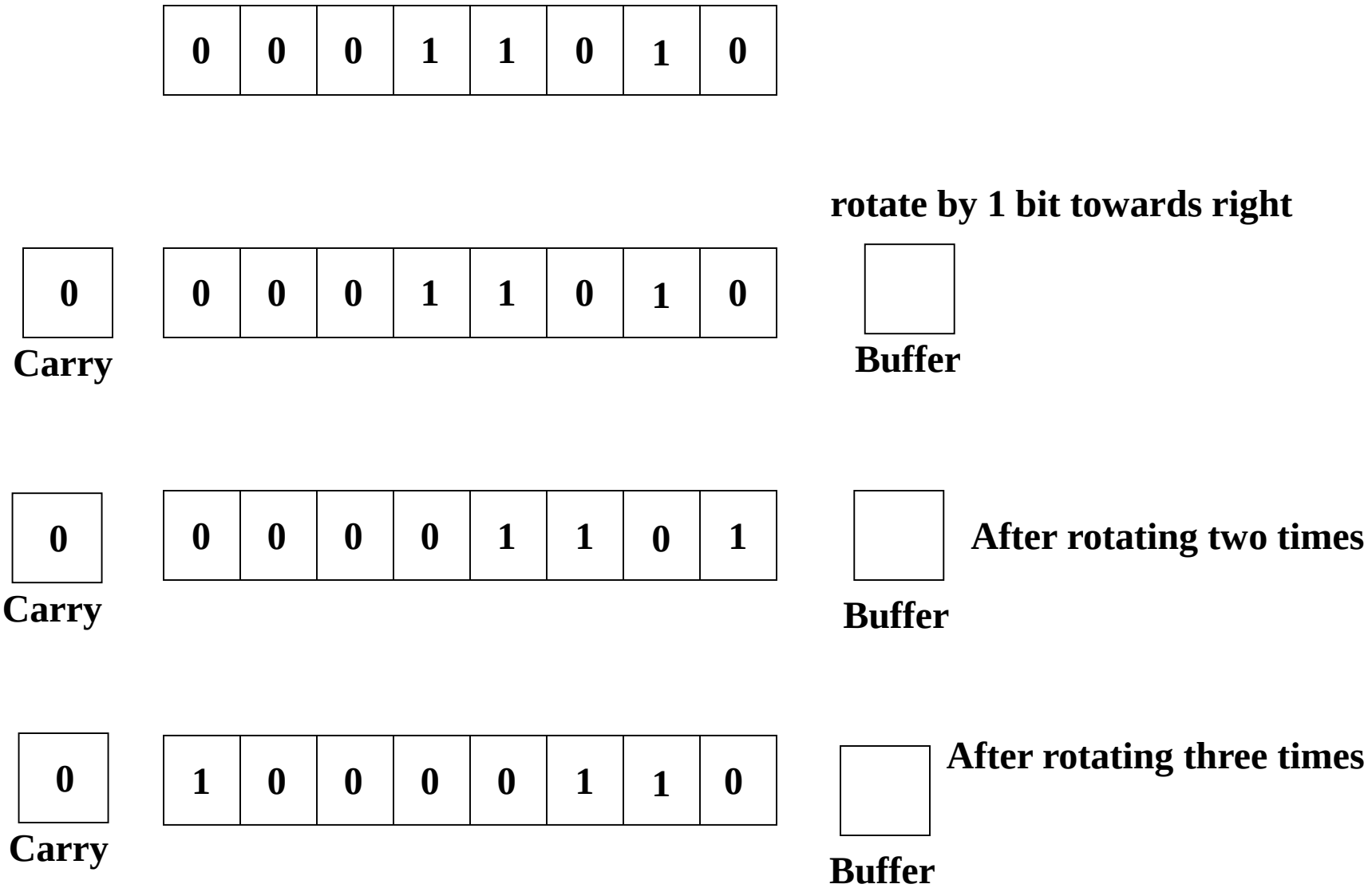
- **Rotate right**



- **Rotate left through carry**



- **Rotate right through carry**



Instruction Set category

- **Based on Operation being performed**
- Flow Control / Program control → Any instruction that alters the normal flow of control from executing the next instruction in sequence
  - Conditional
    - JNZ, JZ
  - Un Conditional
    - Jump

Operation Name	Description
Jump Unconditional	Unconditional transfer
Jump	Test specified condition
Jump to subroutine	Jump to specified address
Return	Replace the content of PC
Execute	Execute instructions
Skip	Increment PC to skip next Instruction
Skip condti	Test conditon for skip
Halt	Stop program execution
Wait (hold)	Stop program execution and resume when condition satisfied
No operation	No operation performed but program execution continued

Instruction Set category

- **Based on Number of operand addresses**
- Instruction Set categorized into four categories based on number of operand address in the instruction.
  - 4-Address Instruction
  - 3-Address Instruction
  - 2-Address Instruction
  - 1-Address Instruction
  - 0-Address Instruction

Example: add M1,M2,M3, nexti

$M(1) \leftarrow M(2)+M(3)$

Instruction Set category

$$X = (A + B) * (C + D)$$

Three Address Instruction

Example:

- $X = (A + B) * (C + D)$
- ADD R1, A, B $R1 \leftarrow M[A] + M[B]$
  ADD R2, C, D $R2 \leftarrow M[C] + M[D]$
  MUL X, R1, R2 $M[X] \leftarrow R1 * R2$

Two Address Instruction

- $X = (A + B) * (C + D)$
- MOV R1, A **R1←A**
  ADD R1, B **R1←R1+B**
  MOV R2, C **R2←C**
  ADD R2, D **R2←R2+D**
  MUL R1, R2 **R1←R1*R2**
  MOV X, R1 **M[X]←R1**

Instruction Set category

One Address Instruction

- X = (A + B) * (C + D)

LOAD A	AC <- M[A]
ADD B	AC <- AC+ M[B]
STORE T	M[T]<- AC
LOAD C	AC <- M [C]
ADD D	AC<- AC+ M[D]
MUL T	AC <- AC * M[T]
STORE X	M[X]<- AC

Zero Address Instruction

- X = (A + B) * (C + D)

PUSH A	TOS<- A
PUSH B	TOS <-B
ADD	TOS<- A+B
PUSH C	TOS<- C
PUSH D	TOS<-D
ADD	TOS<-C+D
MUL	TOS<-(C+D)* (A+B)
POP X	M[X]<- TOS

Note:

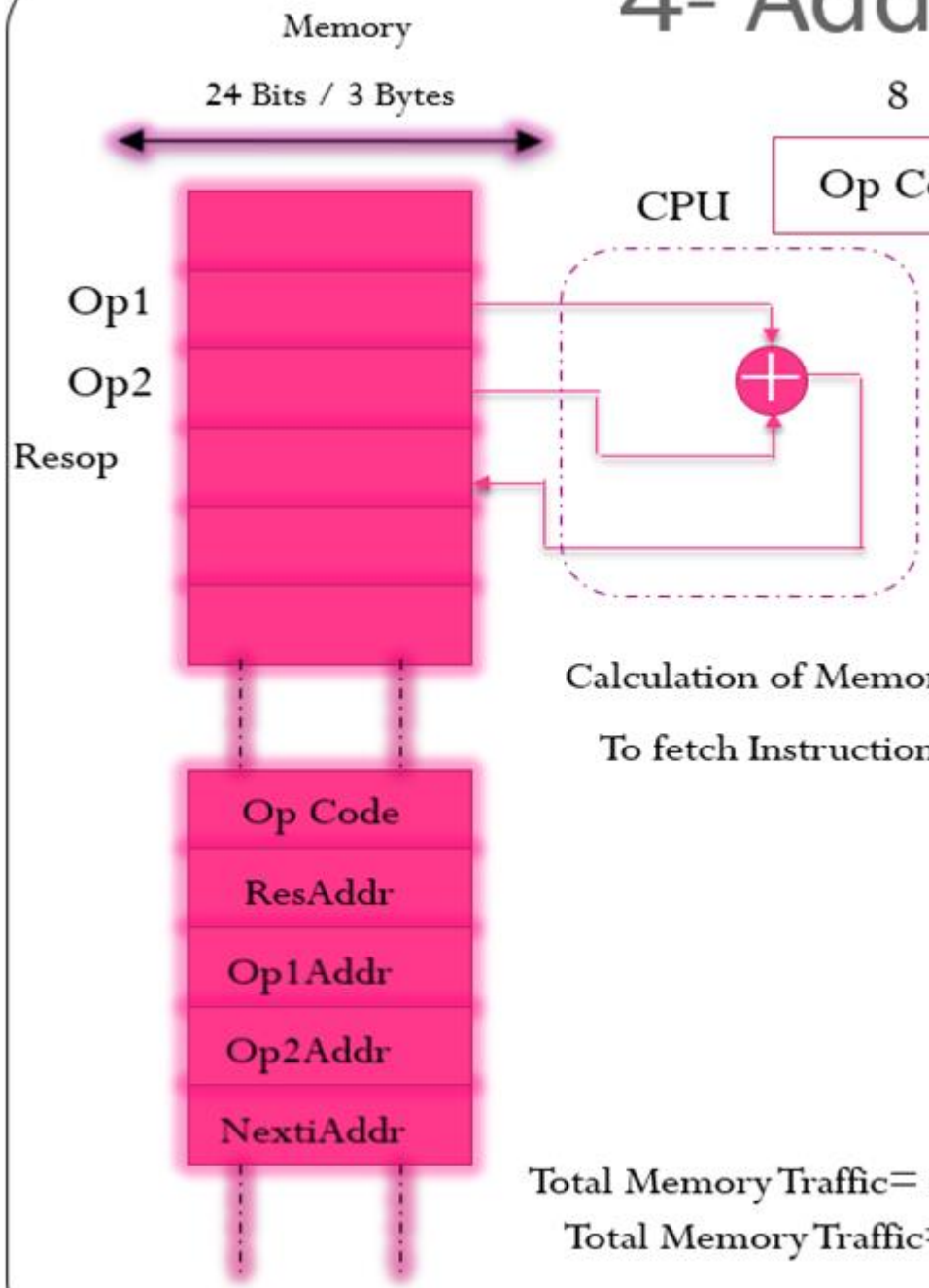
1. One Address and Zero address Instruction (Don't use registers)

2. Zero address →Operation (add) does not require explicit operand addresses because it operates implicitly using a stack.

Calculation of Memory traffic

- **Assumptions**
  - 24-bit memory address (3 bytes)
  - 128 instructions (7 bits rounded to 1 byte)

4- Address Instruction



8	24	24	24	24
Op Code	ResAddr	Op1Addr	Op2Addr	NextiAddr

Example: add M1,M2,M3, nexti

M(1)←M(2)+M(3)

Memory Required to store an Instruction: 5 x 3 bytes = 15 Bytes

Memory Required to Encode an Instruction:

1Byte+ 4 x 3 bytes = 13 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1
ResAddr=1
NextiAddr=1
Total=5

To Fetch Operands and Execute the instruction

Op1=1
Op2=1
Res=1
Total=3

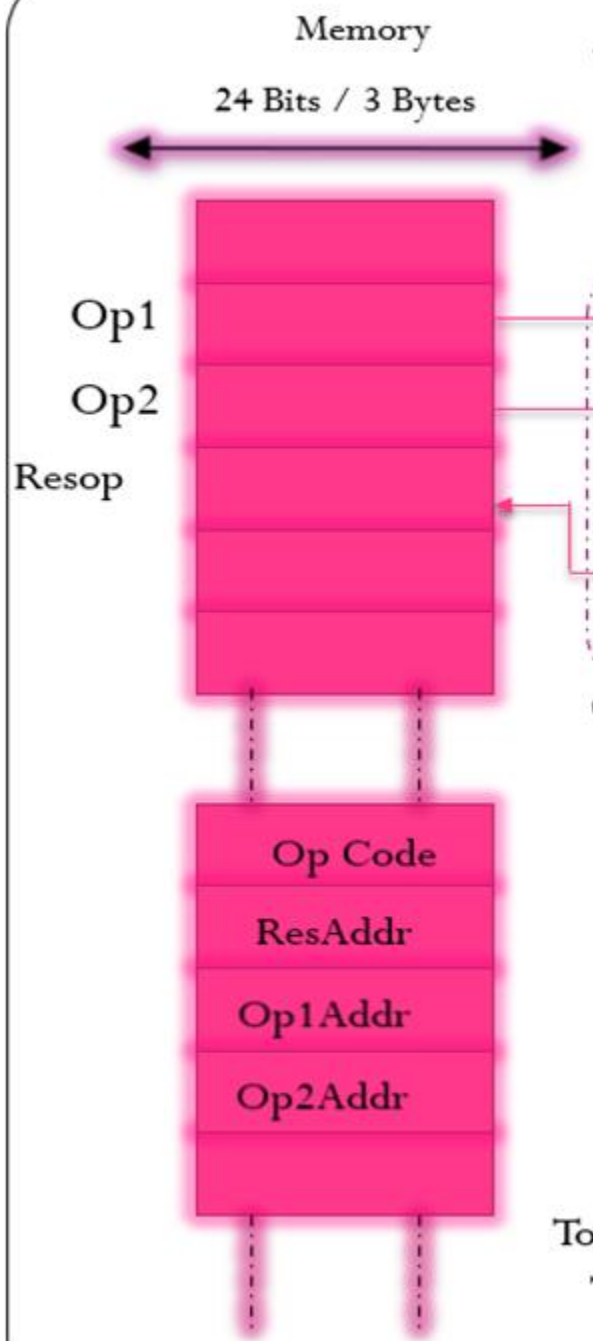
Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

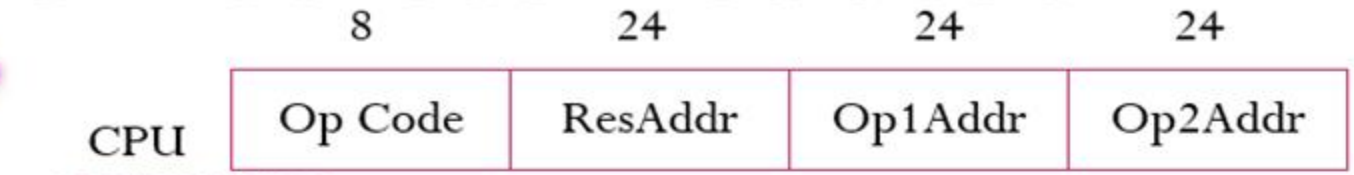
Total Memory Traffic= 5 + 3 =8

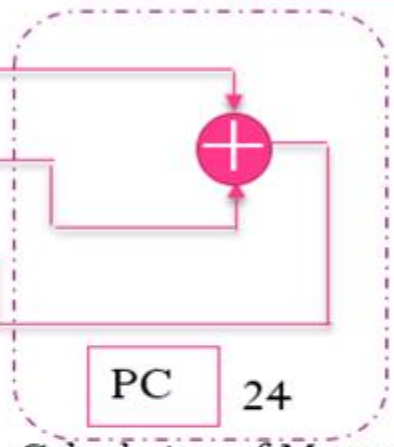
4 – address Instruction Design

- Because of the large instruction word size and number of memory accesses ,the 4- address machine and instruction format is not seen in the machine design.
- Although the 4-address structure is used internally in some implementations of computer control units. This kind of controller implementations is known as *microcoded Control*.

3- Address Instruction







Example: add M1,M2,M3

$M(1) \leftarrow M(2)+M(3)$

Memory Required to store an Instruction: 4 x 3 bytes = 12 Bytes

Memory Required to Encode an Instruction:

1Byte+ 3 x 3 bytes = 10 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1
ResAddr=1
Total=4

To Fetch the Operands and Execute the Instruction

Op1=1
Op2=1
Res=1
Total=3

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 4+ 3 =7

3 – address Instruction Design

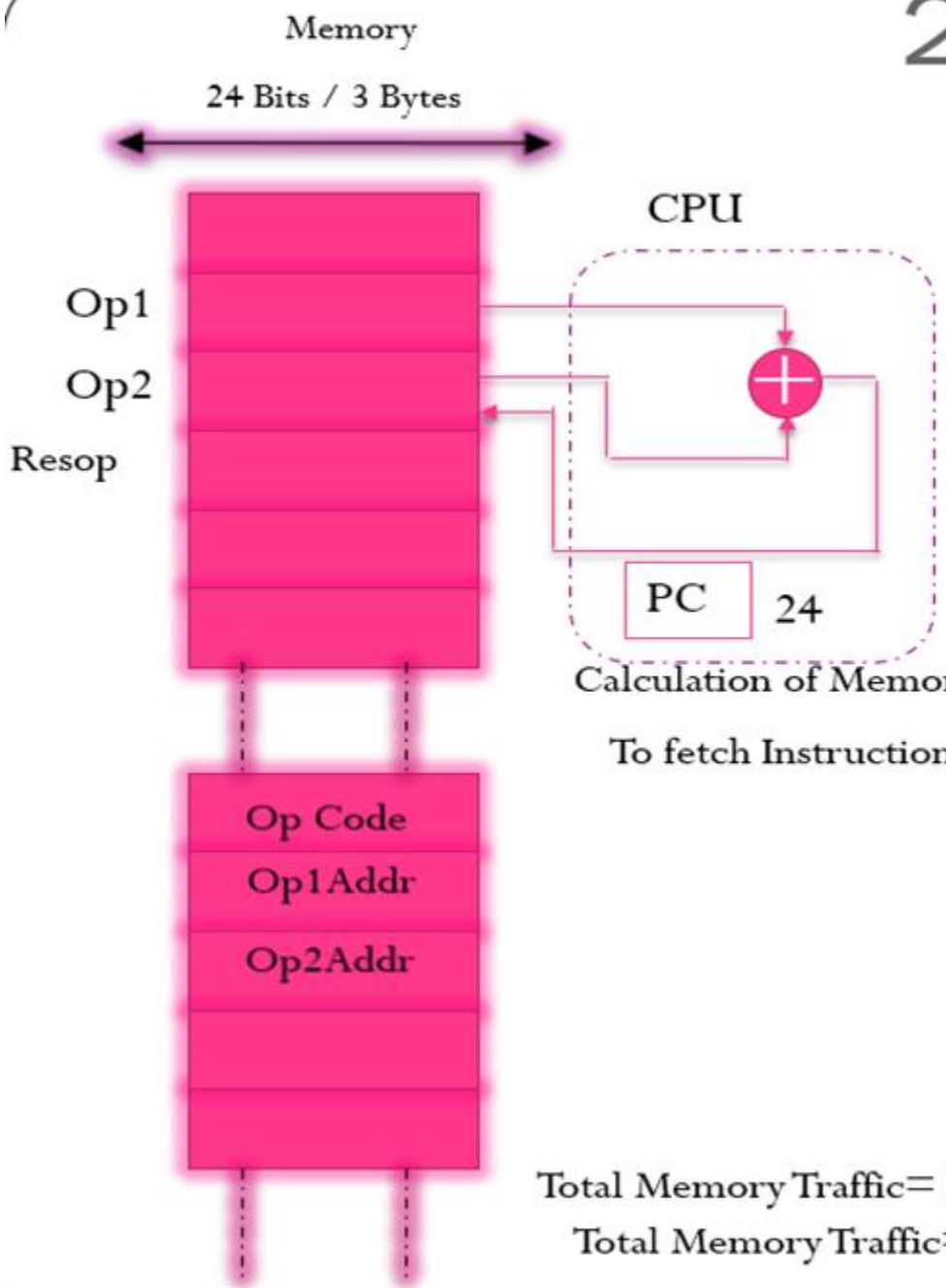
3-Address instruction:

- Address of next instruction kept in processor state register—the PC (Except for explicit Branches/Jumps)
- Rest of addresses in instruction
- This Instruction will require 3X3+1= 10 bytes to encode a 3-address ALU instruction.

The number of *memory access* are required for a 3-address instruction:

- Four words will be transferred to the CPU when the instruction itself is fetched.= 4
- Then the two words representing the operands themselves need to be fetched into the CPU = 2
- And after the addition has been performed, the result needs to be written back to memory = 1 **Total =07**

2- Address Instruction



8	24	24
Op Code	Op1Addr	Op2Addr

Example: add M2,M3

$M(2) \leftarrow M(2)+M(3)$

Memory Required to store an Instruction:

3 x 3 bytes = 09 Bytes

Memory Required to Encode an Instruction:

1Byte+ 2 x 3 bytes = 7 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1

Total=3

To Fetch the Operands and Execute the Instructions

Op1=1
Op2=1
Res=1

Total=3

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 3+ 3 =6

2 – address Instruction Design

2-address Instruction :

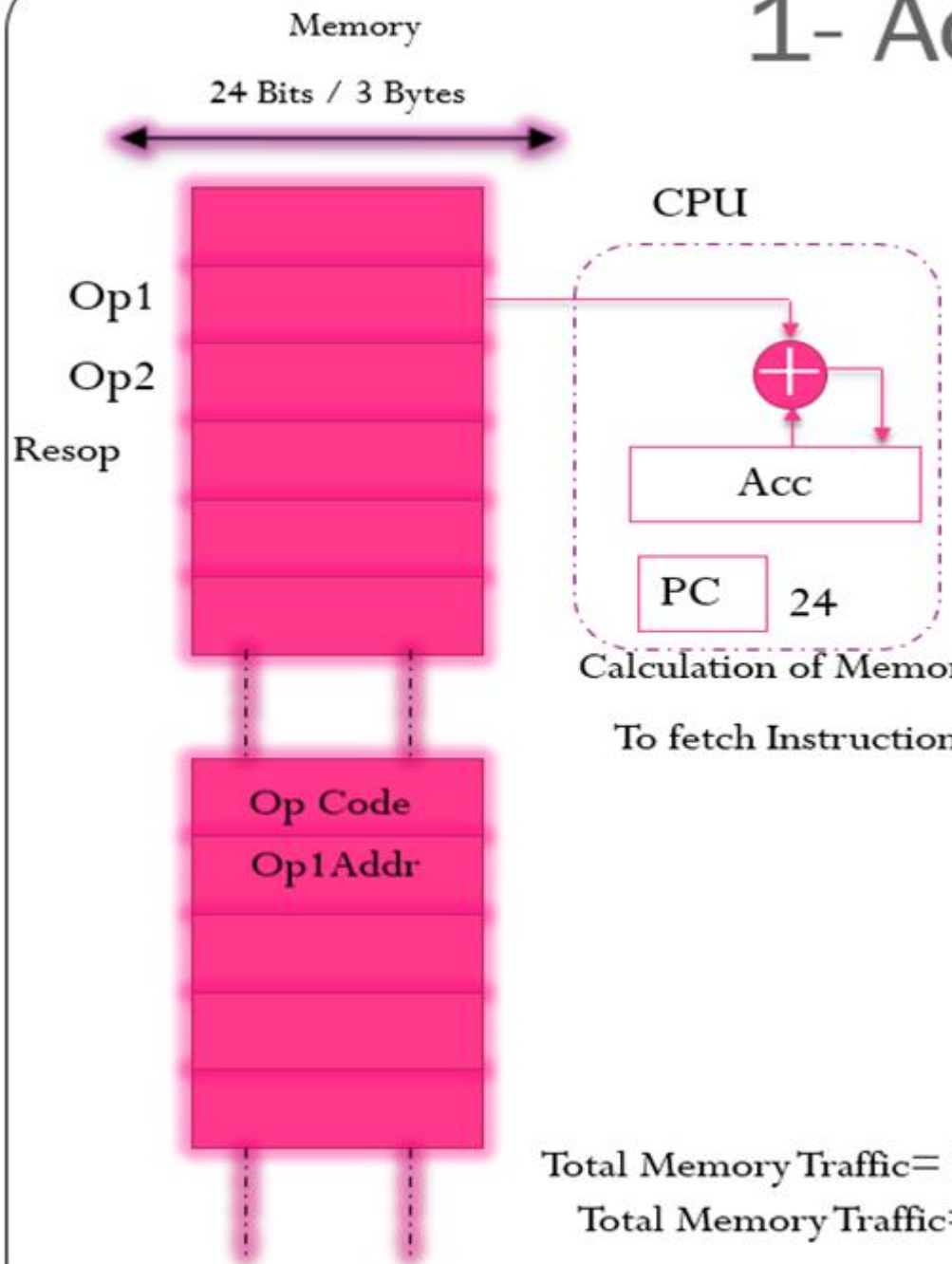
- Result overwrites Operand 2
- Needs only 2 addresses in instruction but less choice in placing data
- This Instruction will require 2X3+1= 7 bytes to encode a 2-address ALU instruction.

The number of *memory access* are required for a 2-address instruction:

- Three words will be transferred to the CPU when the instruction itself

  is fetched. = 3
- Then the two words representing the operands themselves need to be fetched into he CPU and after the addition has been performed, Result overwrites Operand =3
- **Total= 06**
- **add Op1Addr Op2Addr**

1- Address Instruction



8	24
Op Code	Op1Addr

Example: add M2

Memory Required to store an Instruction: 2 x 3 bytes = 06 Bytes

Memory Required to Encode an Instruction:

1Byte+ 1 x 3 bytes = 4Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1

Op1Addr=1

Total=2

To Fetch Operands adn Execute Instruction

Op1=1

Total=1

Result stored in Accumulator – Hence No Memory access required

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

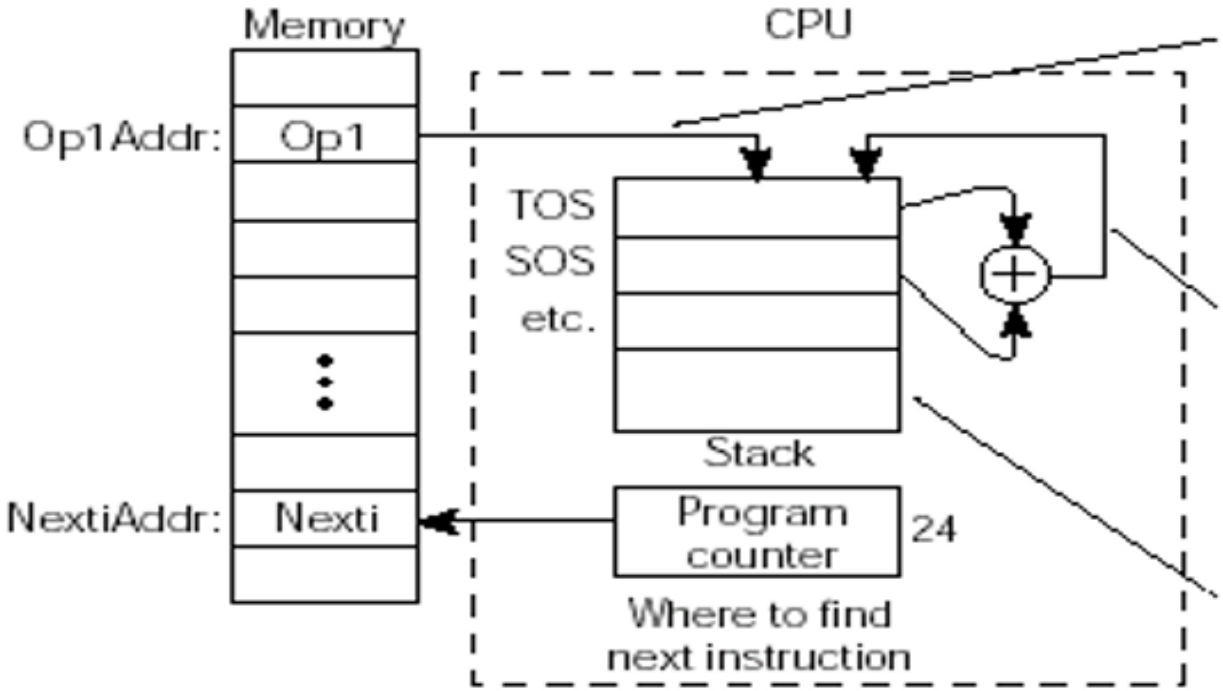
Total Memory Traffic= 2+ 1 =3

0-Address Instruction

Fetch Instruction – 2 cycles
Fetch Operand – 1 cycle
(Uses stack to store when PUSH & hence no store in memory

- Uses a push down stack in CPU
- Arithmetic uses stack for both operands and the result

Computer must have a 1-address instruction to push and pop operands to and from the stack



Push Op1(TOS ← Op1)

Push	Op1Addr

Add (TOS ← TOS + SOS)

add

Where to find operands and where to put the result (on the Stack)

Fetch Instruction – 1 cycles, Fetch Operand & Execute – 0 cycle
(Uses stack to perform ADD & store the result in stack when ADD & No store in memory

Comparisons

Instruction Type	Memory To Store in Bytes	Memory To Encode in Bytes	M/As to fetch an Instruction	M/As to Execute an Instruction	Memory Traffic
4-address	5 x 3 = 15	1+(4 x 3) = 13	5	3	5+3=8
3-Address	4 x 3 = 12	1+(3 x 3) = 10	4	3	4+3=7
2-Address	3 x 3 = 09	1+(2 x 3) = 07	3	3	3+3=6
1-Address	2 x 3 = 06	1+(1 x 3) = 04	2	1	2+1=3
0-Address	1 x 3 = 03	1+(0 x 3) = 01	1 (ADD..) (or) 2 –(PUSH..POP)	0 (ADD..) (or) 1 –(PUSH..POP)	1+0=1

Problems – Example1

- Evaluate **a = (b+c)*d – e** in 3-, 2-, 1-, 0- address machines and compute the memory traffic. Assume 24 bit memory address and one byte opcode.

3-address	2-address	1-address	0-address

add a,b,c
mul a,a,d
sub a,a,e
	load a,b
	Add a,c
	Mul a,d
	Sub a,e
		Load b
		Add c
		Mul d
		Sub e
		Store a
			Push b
			Push c
			Add
			Push d
			Mul
			Push e
			Sub
			Pop a

Memory traffic for 3-address Machine: 7 * 3 = 21(Maximum)

Memory traffic for 2-address Machine: 6 * 4 = 24(Maximum)

Memory traffic for 1-address Machine: 3 * 5 = 15(Maximum)

Memory traffic for 0-address Machine: 3 * 5 + 3 = 18 (Maximum)

Three-address

add a, b, c	a←b+c
mpy a, a, d	a←a*d
sub a, a, e	a←a-e

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
4*3=12	1+(3*3)=10	4	3	4+3=7
4*3=12	1+(3*3)=10	4	3	4+3=7
4*3=12	1+(3*3)=10	4	3	4+3=7
36	**30**	**12**	**9**	**21**

Two-address

load a, b	a←b
add a, c	a←a+c
mpy a, d	a←a*d
sub a, e	a←a-e

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
3*3=9	1+(2*3)=7	3	2	3+2=5
3*3=9	1+(2*3)=7	3	3	3+3=6
3*3=9	1+(2*3)=7	3	3	3+3=6
3*3=9	1+(2*3)=7	3	3	3+3=6
36	**28**	**12**	**11**	**23**

One-address

load b	Acc←b
add c	Acc←Acc+c
mpy d	Acc←Acc*d
sub e	Acc←Acc-e
store a	a←Acc

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
30	**20**	**10**	**5**	**15**

Zero-address

push b
push c
add
push d
mpy
push e
sub
pop a

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
2*3=6	**1+(1*3)=4**	**2**	**1**	**3**
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
39	**23**	**13**	**5**	**18**

Problems – Example 2

Assume 24 bit memory address and one byte opcode.

- Evaluate `a = (b+c)*d-e`
- for 3- 2- 1- and 0-address machines

- What is size of program and amount of memory traffic in bytes?

Instructions

3-Address	2-Address	1-Address	0-Address
`add a, b, c`	`load a, b`	`lda b`	`push b`
`mult a, a, d`	`add a, c`	`add c`	`push c`
`sub a, a, e`	`mult a, d`	`mult d`	`add`
	`sub a, e`	`sub e`	`push d`
		`sta a`	`mult`
			`push e`
			`sub`
			`pop a`

Bytes size

	3-Address	2-Address	1-Address	0-Address
Instruction	30	28	20	23
Memory	27	33	15	15
Total	57	61	35	38

Practice problems

- Evaluate the expression a = b - c * d and compute memory traffic for 4, 3, 2, 1,and 0 address machine. Assuming that addresses are 16 bits, data values are 16 bits, opcodes are 8 bits and 1 byte word length.
- Develop a comparative table for the performance parameters such as memory to store, memory to encode, M/As to fetch , M/As to execute and total memory Traffic for 4-,3-,2-,1-,0- address machine instructions. Consider the following specifications: Memory word size is 1 byte, Memory/register Address size is 2byte, Opcode size is 1 byte.

i. Write an appropriate assembly language programming using 3-Address, 2-Address, 1-Address and 0-address machine instructions for the following expression (with registers & without registers). Assume that all are integer operations.

$$X= (A / B + C * D) / (D * E - F + C / A) + G$$

ii. Compute various performance factors such as memory to store a program, memory to encode a whole program, Memory access to fetch & execute and memory traffic.

Addressing Modes

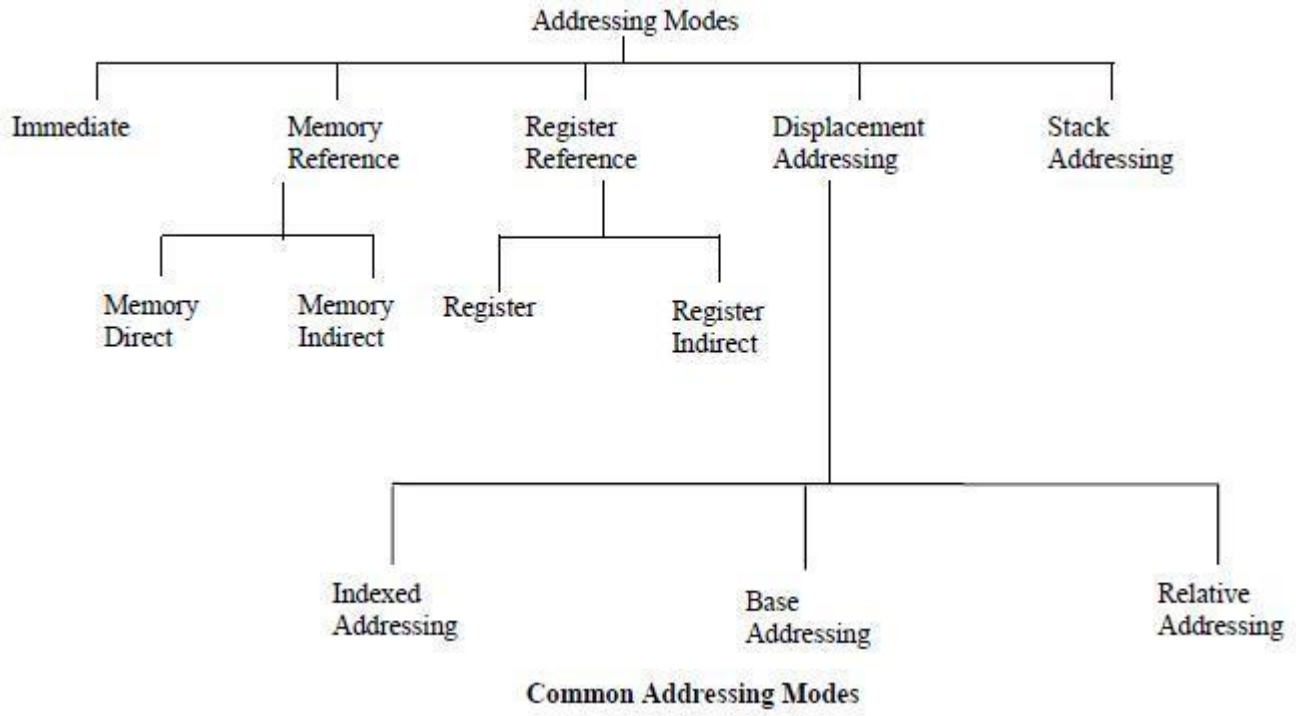
- Instruction Set is categorized **Based on Addressing Modes**
- **Addressing Mode**
  - **The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.**
  - The mode of access of effective address is called addressing mode
  - The Way the operands are specified in the instruction
  - The operation to be performed is indicated by the opcode.
  - Operands can be in registers, memory or embedded in the instruction
- **Effective Address**
  - The address in which the actual operand is available is called as Effective address

Addressing Modes

- **Terminologies**
  - **Displacement:** It is an 8 bit or 16 bit immediate value given in the instruction.
  - **Base**: Contents of base register, BX or BP.
  - **Index**: Content of index register SI or DI.

Classification of Addressing Modes

1. Stack (Implied/Implicit) Addressing mode
2. Immediate Addressing mode
3. (Memory) Direct Addressing mode
4. Register Direct Addressing mode
5. (Memory) Indirect Addressing mo
6. Register Indirect Addressing mode
7. Displacement Addressing modes
    1. Indexed Addressing mode
    2. Base register Addressing mode
    3. Relative Addressing Mode
9. Auto Increment Addressing Mode
10. Auto Decrement Addressing Mode



Common Addressing Modes

1.Stack (Implied/Implicit) Addressing mode

- Definition of the instruction itself specify the operands implicitly.
- Operand is implied / specified implicitly in the instruction itself.
- Operations like PUSH and POP for the computation
- Zero address instructions in a stack organized computer are implied mode instructions.
- Effective Address **(EA) = AC or Stack[SP]**
- Advantage:
  - Instruction specifies a fixed and unvarying address
  - No memory references
- Disadvantage:
  - Limited Computational Capacity

DEC (Decrement A register)

CLC (used to reset Carry flag to 0)

PUSH

POP

2.Immediate Addressing mode

- The simplest form of addressing
- Effective Address **(EA) = Value**
- Data is a part of instruction itself.
- **Example:**
  - **MOVE #100, R1**
    - Here the data 100 is moved to R1.
  - **MVI #01, A**
    - MVI stands for Move Immediate. This basically implies move 01 to A.
- **Advantage:**
  - This mode can be used to define and use constants or set of initial values of variables.
  - No memory references
- **Disadvantage:**
  - Limited Operand size

(a) Immediate addressing:

instruction contains
the operand

3. (Memory) Direct Addressing mode

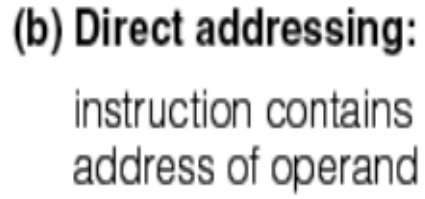
- The address where data is available is part of the instruction
- The address field contains the effective address of the operand
- Effective Address **(EA) = LOC**
- **Example:**
  - **MOVE A, R1**
    - Here the data in memory location A is moved to R1.

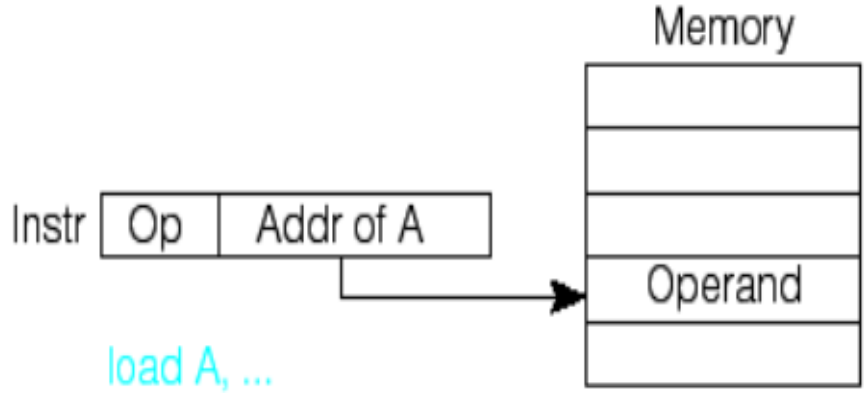
Advantage:

- Large operand Magnitude, Simple

Disadvantage:

- Limited Address Size
- The change in the location of the program is associated with the change in all absolute memory references.





4. Register Direct Addressing mode

- Register addressing is similar to direct addressing
  - The only difference is that the address field refers to a register rather than a main memory address
- Effective Address **(EA) = Ri**
- **Example:**
  - **MOVE R2, R1**
    - Here the data in Register R2 is moved to R1.

Advantage:

- No memory Reference

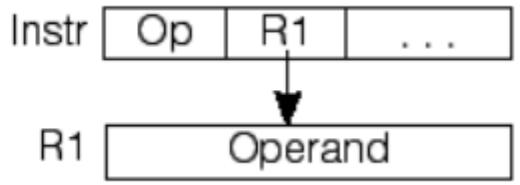
Disadvantage:

- Limited number of registers

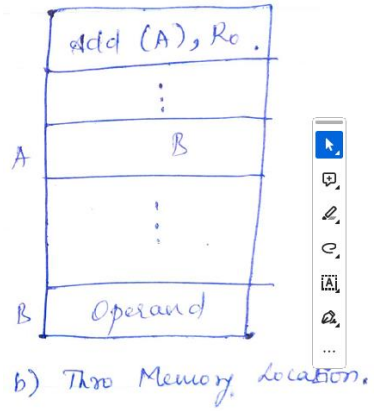
(d) Register direct addressing:

register contains operand

load R1, ...



5. (Memory) Indirect Addressing mode



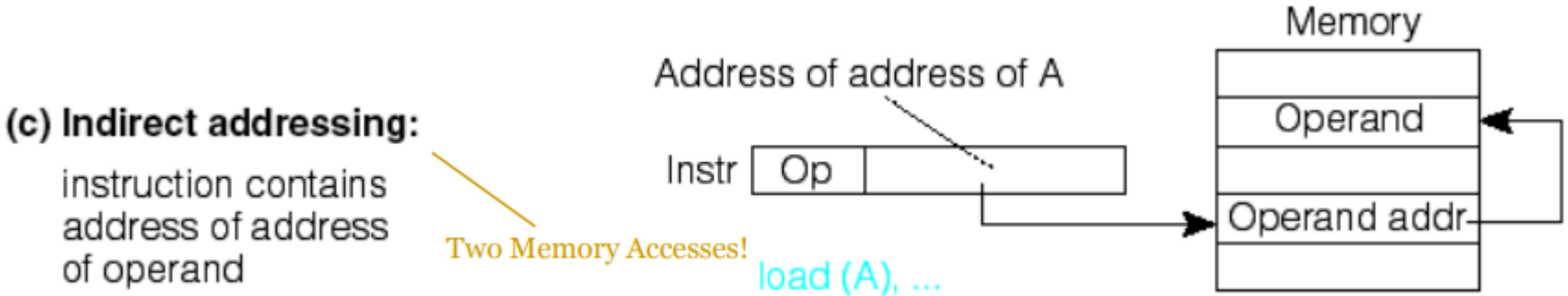
- The address field contains the address of effective address of the operand
  - Contains a full –length address of the operand
- Effective Address **(EA) = (LOC) or [LOC]**
- **Example:**
  - **MOVE (A), R1**
    - Here A – has another memory address (not data)
    - The data in the address in A is moved to R1.

Advantage:

- Large address space

Disadvantage:

- The change in the location of the program is associated with the change in all absolute memory references



6. Register Indirect Addressing mode

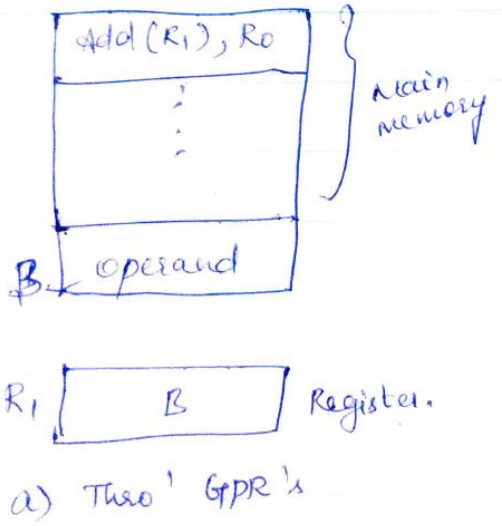
- Register indirect is just analogous to indirect addressing in both cases
  - The only difference is whether the address field refers to memory location or a register.
- Effective Address **(EA) = (Ri) or [Ri]**
- **Example:**
  - **MOVE (R2), R1**
    - Here R2 – has memory address (not data)
    - The data in the address in R2 is moved to R1.

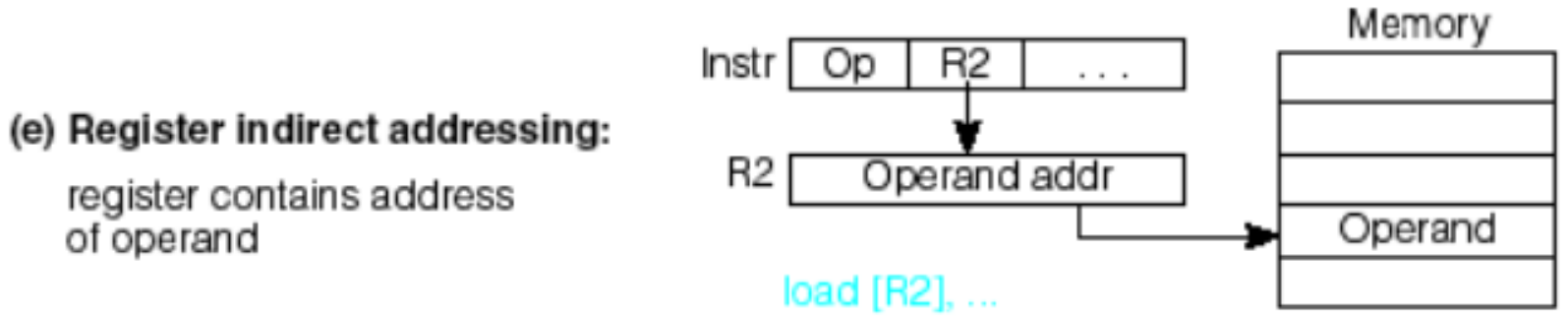
Advantage:

- Large address space

Disadvantage:

- Extra memory space





7. Displacement Addressing modes - Indexed Addressing mode

- The address field reference a main memory address, and the referenced register contain a positive displacement from that address .
- Effective Address **(EA) = (Ri) + X**

Index Register

The base register holds the beginning location of a memory array, while the index register holds the relative position of an element in the array.

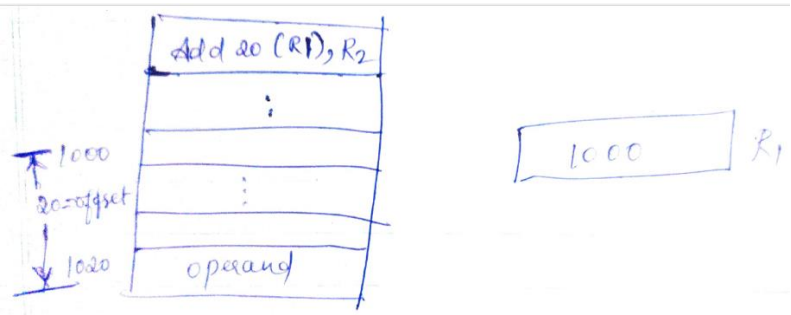
Index value of an array Stored in

Index Register

- **Example:**
  - **MOVE 20 (R2), R1**
    - Here R2 – has memory address (not data).
    - The address in R2 is added with the index value 20 which is the EA.
    - The data in the address in R2+20 is moved to R1.

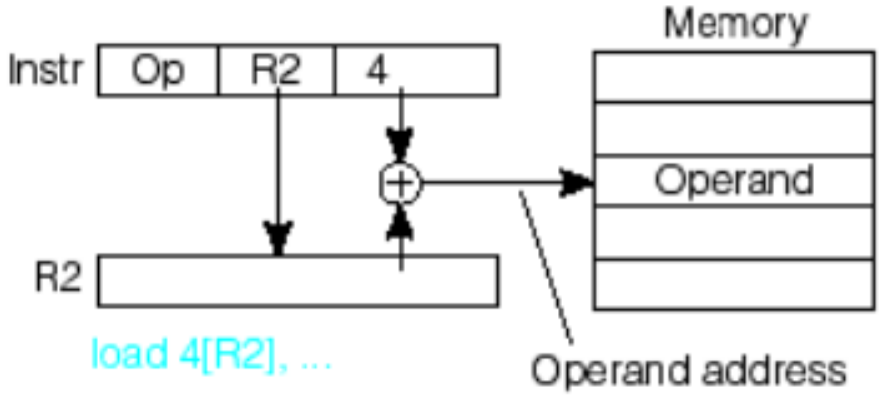
Advantage:

- Special Locality



(f) Displacement (based or indexed) addressing:

address of operand = register + constant



Example: Works for ARRAYS

7. Displacement Addressing modes – Base register Addressing mode

- The address field reference a main memory address, and the referenced register contain a positive displacement from that address .
- Effective Address **(EA) = (Ri + BX)**
- **Example:**
  - **ADD AX, [BX+SI]**
  - **ADD R1, (R2+R3)**
    - **Meaning: R1 ⬅ R1+M[R2+R3]**
    - Here R2 & BX – Base Register
    - R3 & SI – Index register
- EA=sum (content of BR and SI)

ADD R1, (R2+3) --- Base
ADD R1,(R2+R3) --- Base with Index
ADD R1, 20(R2+R3) --- Base with Index and Offset

Base Register

The base register holds the beginning location of a memory array, while the index register holds the relative position of an element in the array.

Advantage:

It use a convenient means of implementing segmentation.

Disadvantage:

Complexity

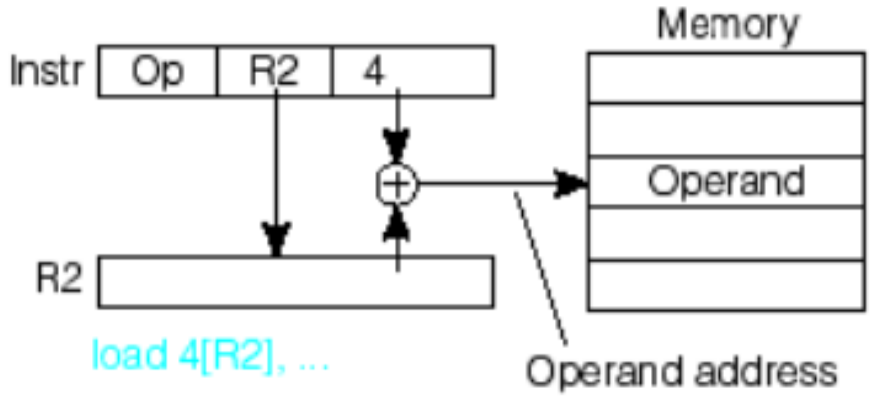
Base address of the array Stored

Base Register

Example: Works for ARRAYS

(f) Displacement (based or indexed) addressing:

address of operand = register + constant



7. Displacement Addressing modes – Relative Addressing Mode

- **PC**- relative addressing - the implicitly referenced register is the program counter (PC)
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = (PC) + X**
- **Example:**
  - **If Branch > 0,**

    JUMP -200

    - Here, based on the condition,

    the **address in PC is incremented with constant value 200.**

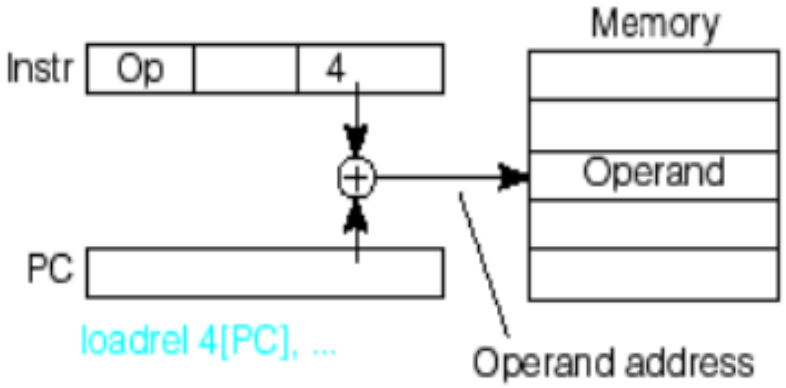
Advantage:

- program-relative addressing is that the code may be position-independent

Disadvantage:

- Complexity

(g) Relative addressing:
address of operand = PC + constant



8. Auto Increment Addressing mode

- Register incremented after accessing memory
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = (Ri); Increment**
- **Example:**
  - **ADD (R2)+, R0**
    - Here R2 – has Operand Address
    - After accessing the operand, the register content is automatically incremented.

Advantage:

- Useful while transferring large chunks of contiguous data.

Disadvantage:

- Complexity

9. Auto Decrement Addressing mode

- Register decremented and then contents accessed memory
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = Decrement; (Ri)**
- **Example:**
  - **ADD -(R2), R0**
    - Here R2 after decrement – has Operand Address
    - The contents of Register is decremented first and then the content gives Operand Address.

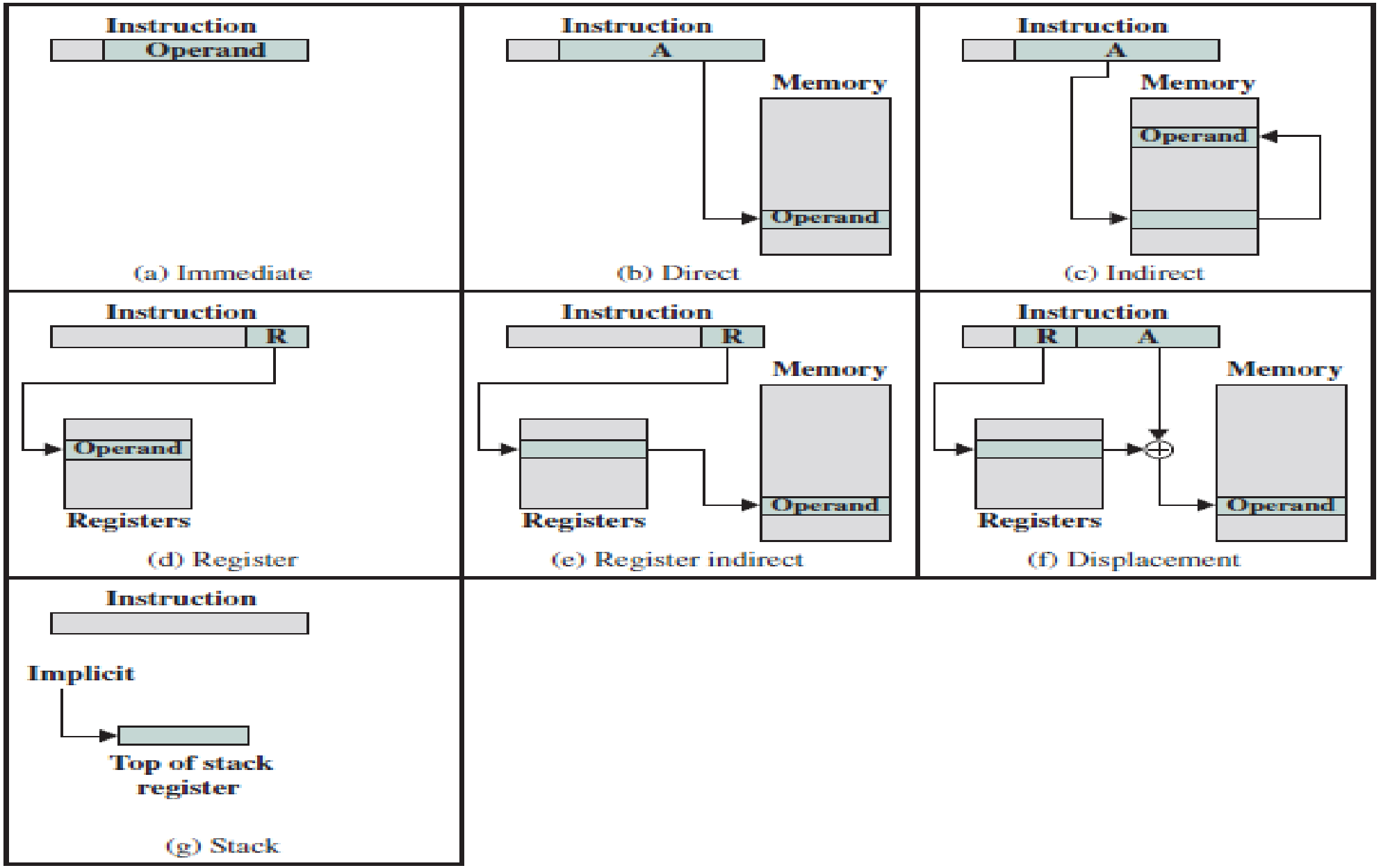
Advantage:

- Useful while transferring large chunks of contiguous data.

Disadvantage:

- Complexity

Addressing modes



Addressing modes

	Mode	Algorithm	Advantage	Disadvantage
1	Implied	Operand=Φ	No Memory references	Limited Computation capability
2	Immediate	Operand=1	No memory Reference	Limited operand magnitude
3	Direct	EA = A	Simple	Limited address space
4	Indirect	EA =(A)	Large Address space	Multiple Memory references
5	Register	EA = R	No memory Reference	Limited address space
6.1	Register Indirect	EA = (R)	Large address space	Extra memory space
6.2	Relative	EA = PC+2's Complement	Use for control instructions	Complexity
6.3	Base Register	EA=BA+ 2's Complement	convenient during segmentation	Complexity
6.4	Indexed	EA= A+ Index value	Accessing array elements	Array lower bounds
7	Stack	EA= Top of Stack	No memory Reference	Limited Applicability

Problems

Find the effective address and the content of AC for the given data.

PC = 200
R1 = 400
XR = 100
AC

Address	Memory	
200	Load to AC	Mode
201	Address = 500	
202	Next instruction	
399	450	
400	700	
500	800	
600	900	
702	325	
800	300	

Addressing Mode	Effective Address		Content of AC
Direct Address	**500**	**AC ← (500)**	**800**
Immediate operand	**201**	**AC ← 500**	**500**
Indirect address	**800**	**AC ← ((500))**	**300**
Relative address	**702**	**AC ← (PC + 500)**	**325**
Indexed address	**600**	**AC ← (XR + 500)**	**900**
Register	**-**	**AC ← R1**	**400**
Register Indirect	**400**	**AC ← (R1)**	**700**
Autoincrement	**400**	**AC ← (R1)+**	**700**
Autodecrement	**399**	**AC ← -(R1)**	**450**

Questions

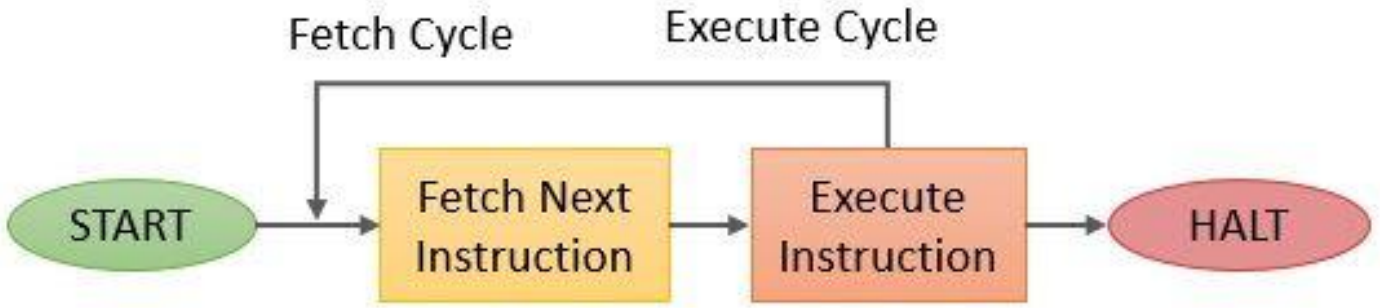
- An instruction is stored at location 300 with its address field at location 301. The address field has the value 400. A processor register R1 contains the number 200. Evaluate the effective address if the addressing mode of the instruction is (a) direct; (b) immediate (c) relative (d) register indirect; (e) index with R1 as the index register.
- Let the address stored in the program counter be designated by the symbol X1. The instruction stored in X1 has the address part (operand reference) X2. The operand needed to execute the instruction is stored in the memory word with address X3. An index register contains the value X4. What is the relationship between these various quantities if the addressing mode of the instruction is
- (a) direct (b) indirect (c) PC relative (d) indexed?

Module 3 – Phases of Instruction Cycle
ALU

Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

Phases of Instruction Cycle

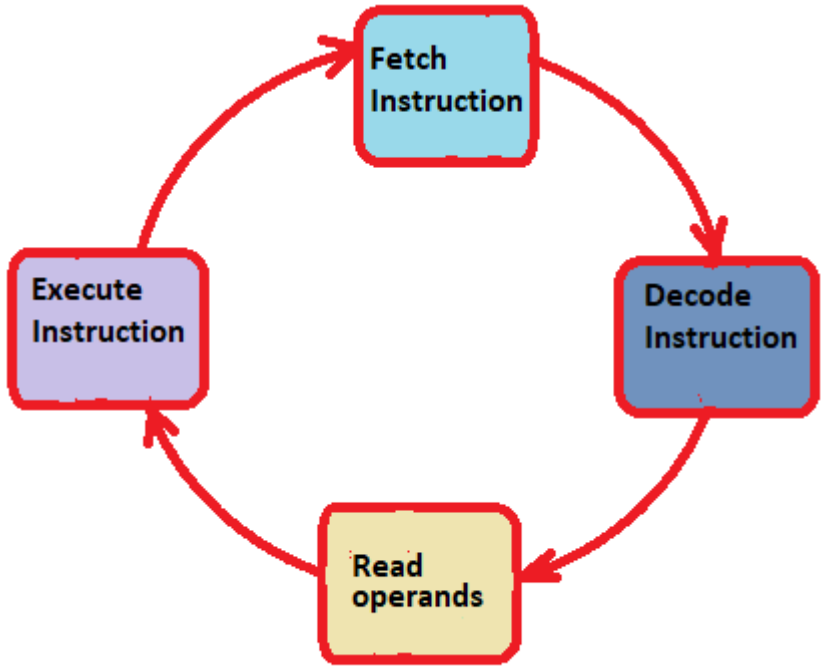
- The IAS operates repetitively performing an instruction cycle.
- Each instruction cycle consists of two sub cycles.
- **Two Phases:**
  - Fetch
  - Execute



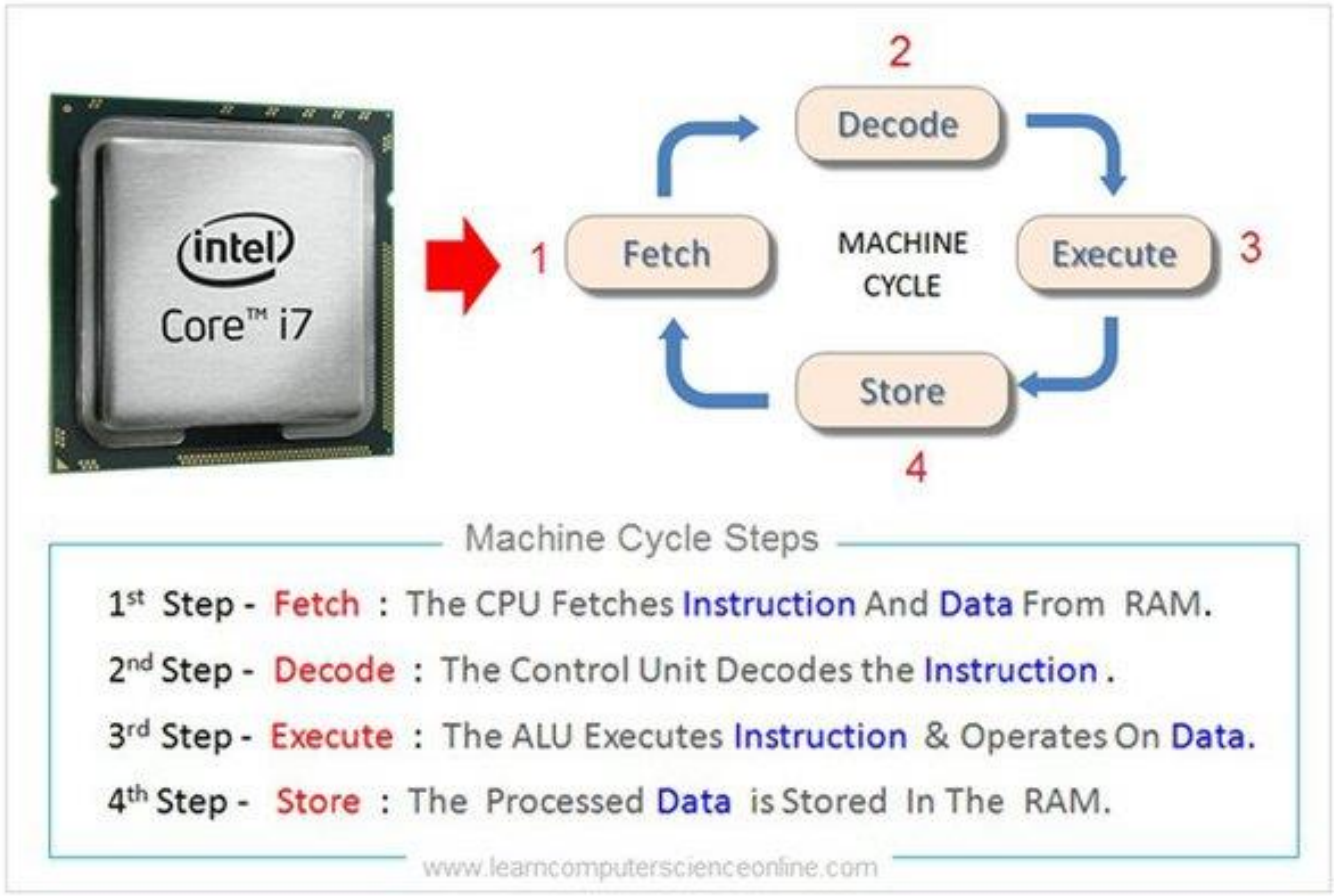
Basic Instruction Cycle

Phases of Instruction Cycle

- 4 phases of Instruction Cycle

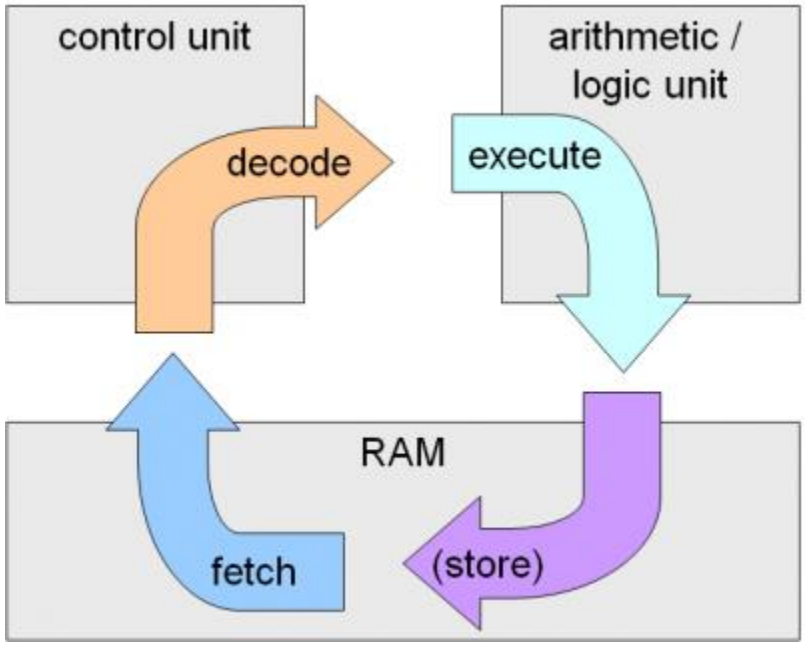


An Instruction Cycle



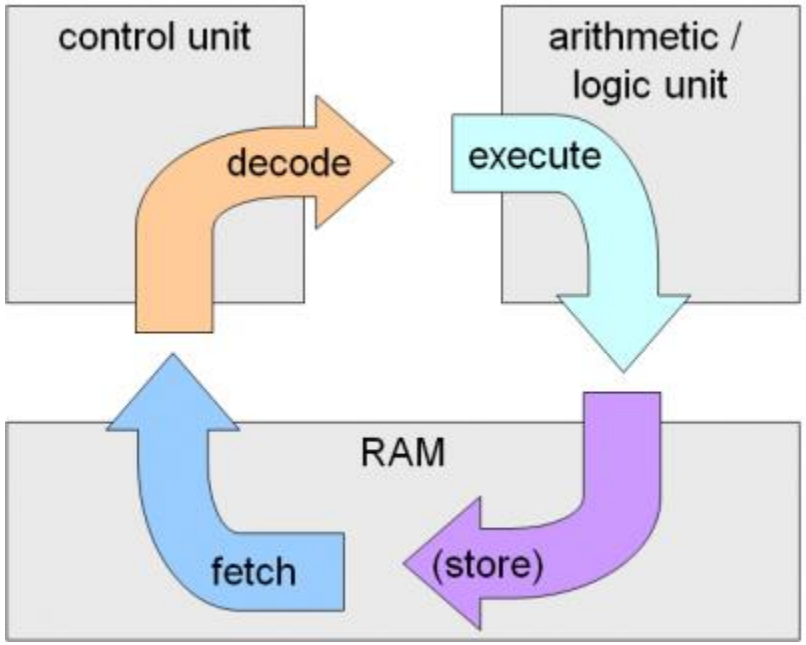
Phases of Instruction Cycle

- **Fetch Phase:**
  - PC – holds the address of Instruction
  - Processor – Fetches the instruction from memory and stores in IR
  - Increment PC
    - Unless told Otherwise
  - Processor interprets instruction and performs required actions
- **Execute Phase:**
  - Carry out the actions specified by the instruction in the IR (execution phase).
  - The instruction decoder and control logic unit is responsible for implementing the action specified by the instruction loaded in the IR



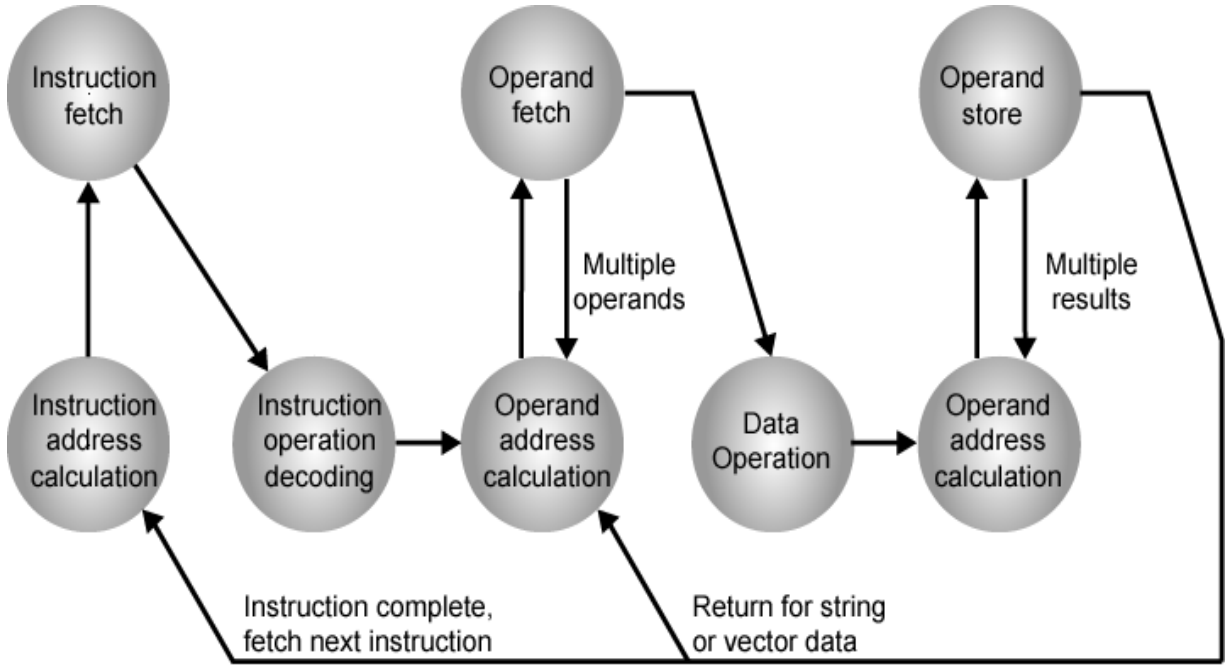
Phases of Instruction Cycle

- An instruction can be executed by performing one or more of the following operations in some specified sequence.
  - **Transfer a word** of data from one processor **register** to another or to the ALU.
  - Perform an **arithmetic or a logic operation** and store the result in a processor register.
  - **Fetch** the **contents** of a given **memory location** and load them into a processor register.
  - **Store** a word of **data from** a processor **register** into a given **memory location**.



Instruction Cycle - State Diagram

- **Instruction address calculation (IAC):**
  - Determine the address of the next instruction to be executed. Adding a fixed number to a next number.
- **Instruction fetch: (IF)**
  - Read the instruction from its memory location into the processor.
- **Instruction operation decoding (IOD)**
  - Analyze instruction to determine type of operation to be performed and operand(s) to be used.
- **Operand Address Calculation: (OAC)**
  - If the operation involves the reference to an operand in memory or available via I/O, then determine the address of the operand.
- **Operand Fetch (OF):**
  - Fetch the operand from memory or read it from I/O.



- **Data Operation (DO):**
  - Perform the operation indicated in the instruction.
- **Operand store (OS):**
  - Write the result into memory or out to I/O.

Interrupts

Table 3.1 Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware Failure	Generated by a failure such as power failure or memory parity error.

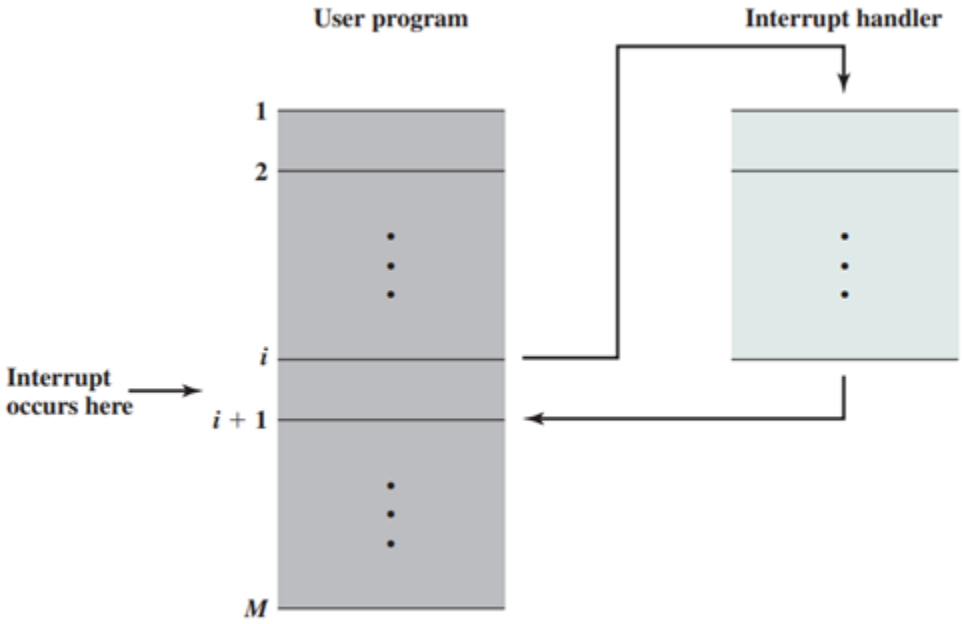


Figure 3.8 Transfer of Control via Interrupts

Instruction cycle with Interrupts

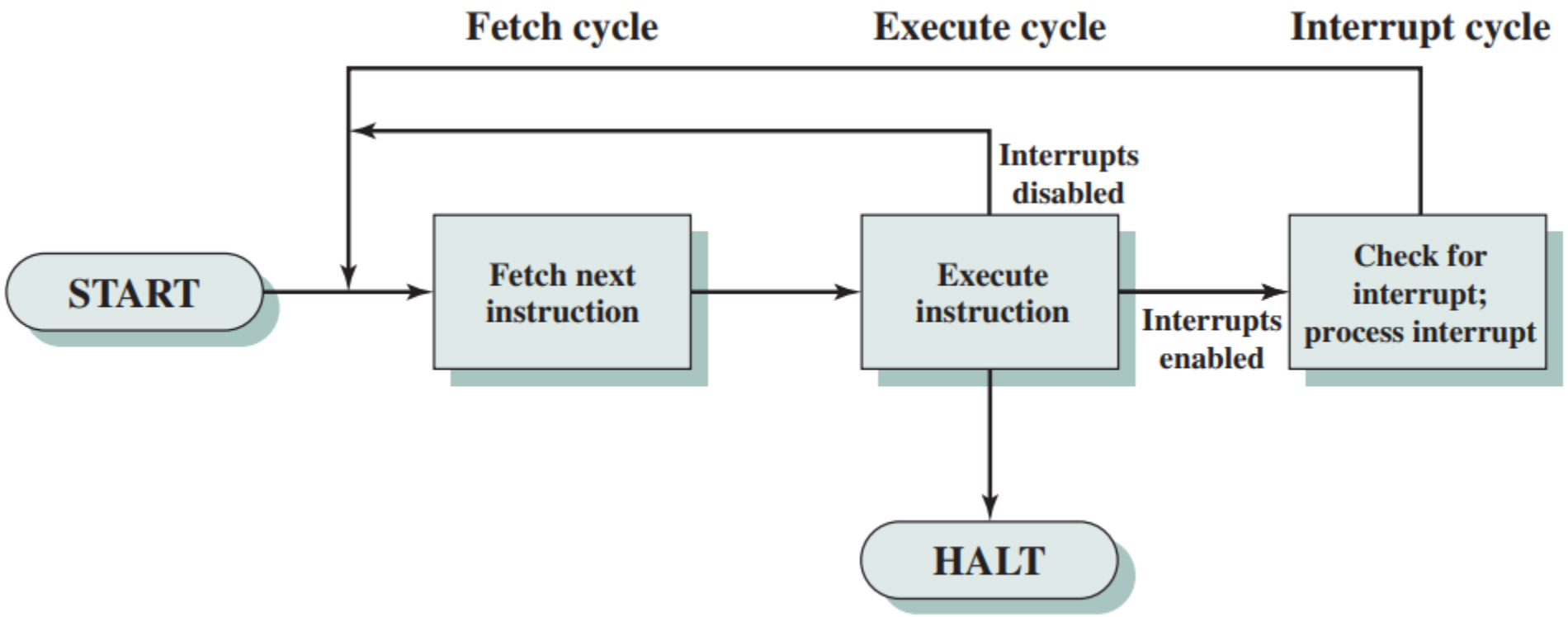


Figure 3.9 Instruction Cycle with Interrupts

Instruction cycle with Interrupts – State Diagram

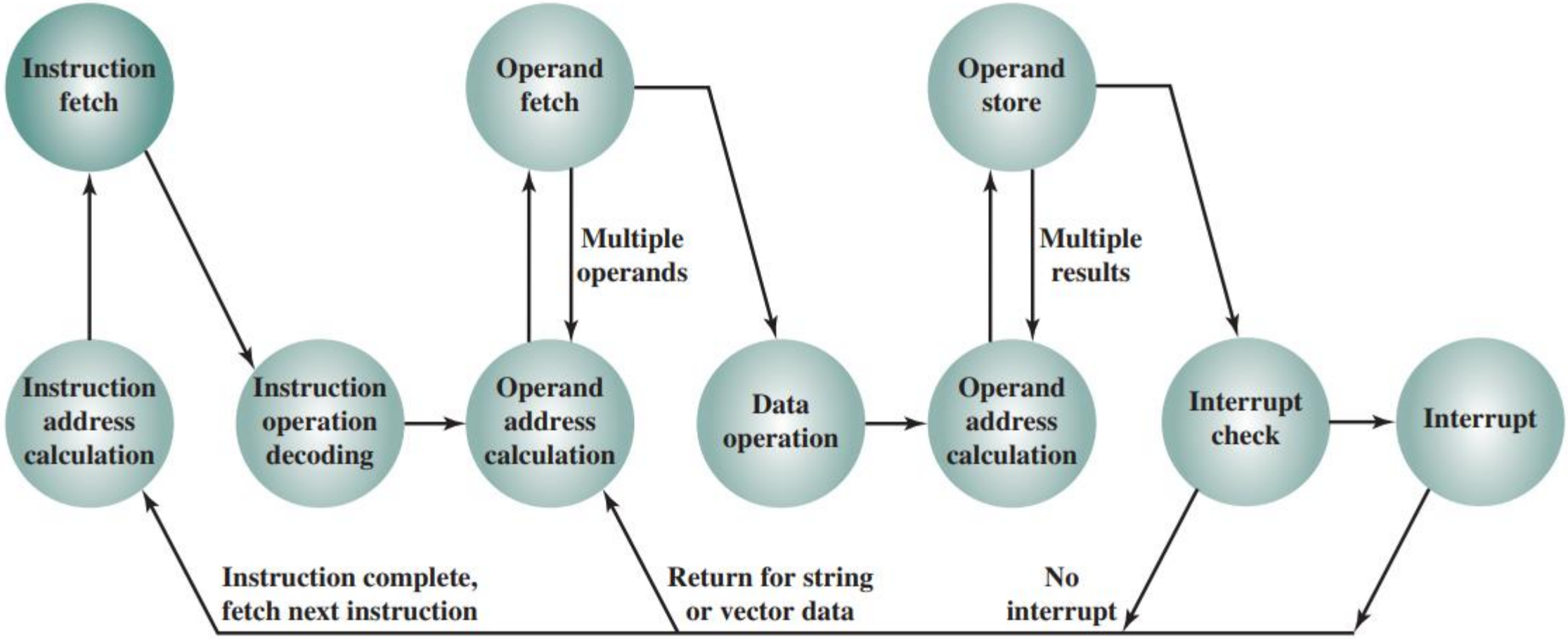
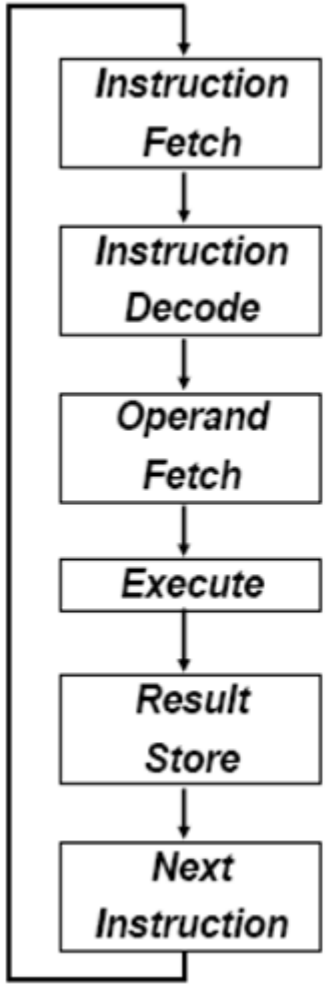


Figure 3.12 Instruction Cycle State Diagram, with Interrupts

Instruction Execution Cycle



Obtain instruction from program storage

Determine required actions and instruction size

Locate and obtain operand data

Compute result value or status

Deposit results in storage for later use

Determine successor instruction

ALU

- **Arithmetic-Logic Unit (ALU)** is the part of a CPU that carries out arithmetic and logic operations on the operands.
- ALU is divided into two units:
    - Arithmetic Unit (AU)
    - Logic Unit (LU).
- Some processors contain more than one AU
    - For example, one for fixed-point operations and another for floating-point operations.
- **Control Unit (CU) -** supplies the data required by the ALU from memory, or from input devices, and directs the ALU to perform a specific operation based on the instruction fetched from the memory

ALU

Operations on ALU

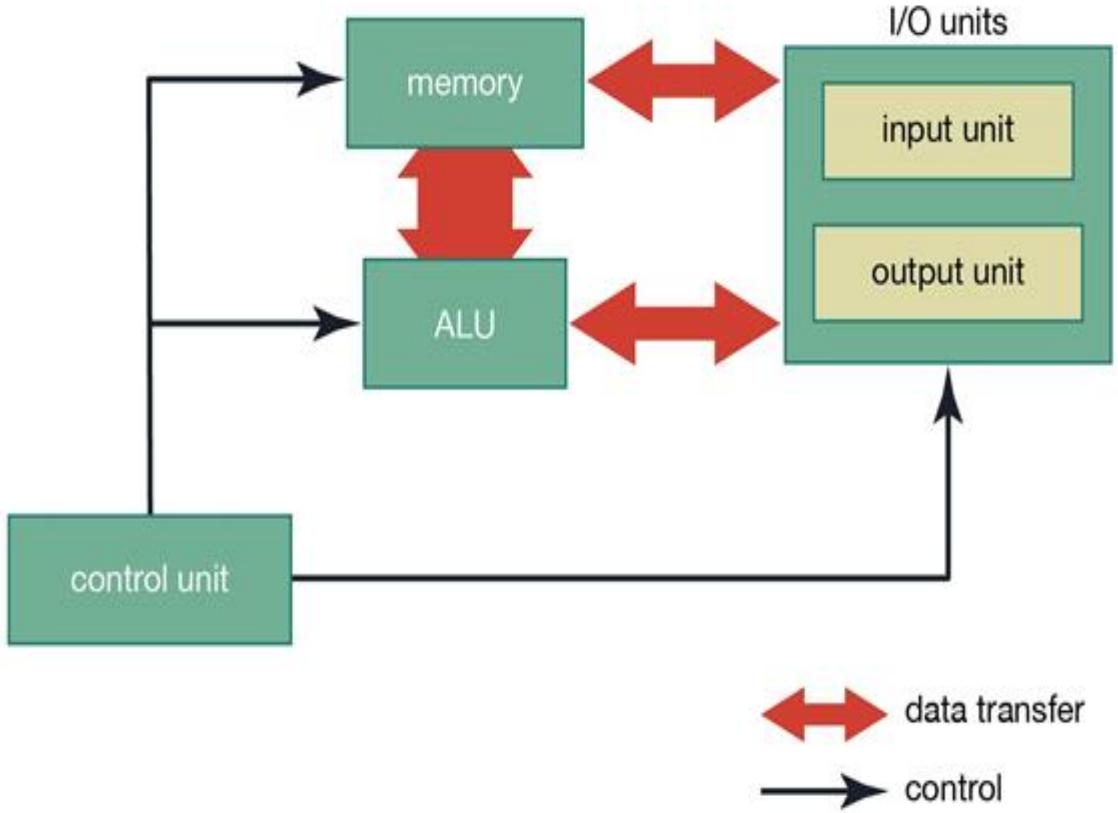
- **logical operations** – These include operations like AND, OR, NOT, XOR, NOR, NAND, etc.

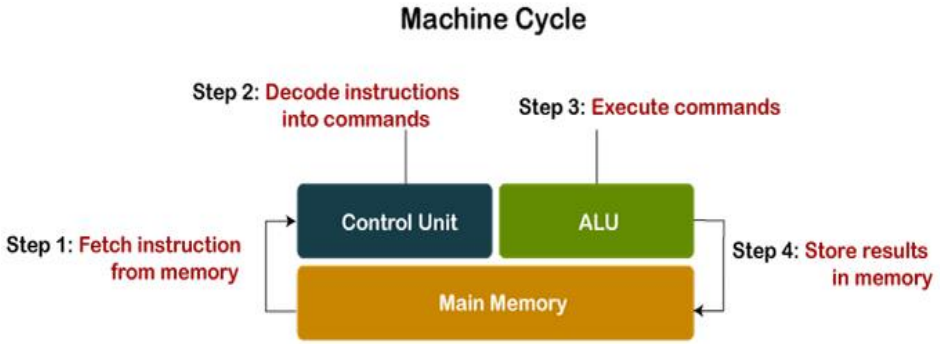
- **Bit-Shifting Operations** – This pertains to shifting the positions of the bits by a certain number of places either towards the right or left, which is considered a multiplication or division operations.

- **Arithmetic operations** – This refers to bit addition and subtraction

How ALU Works?

- ALU has direct input and output access to
  - processor controller,
  - main memory (random access memory or RAM in a personal computer)
  - input/output devices
- Inputs and outputs flow along an electronic path that is called a bus





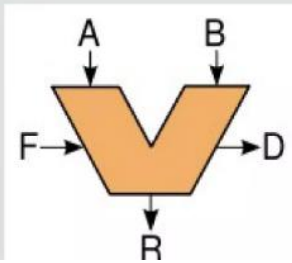
ALU

- The ALU is that part of the computer that actually performs arithmetic and logical operations on data
- All of the other elements of the computer system—control unit, registers, memory, I/O—are there mainly to bring data into the ALU for it to process and then to take the results back out
- We have, in a sense, reached the core or essence of a computer when we consider the ALU
- An ALU and indeed, all electronic components in the computer, are based on the use of simple digital logic devices that can store binary digits and perform simple Boolean logic operations
- Operands for arithmetic and logic operations are presented to the ALU in registers, and the results of an operation are stored in registers
- These registers are temporary storage locations within the processor that are connected by signal paths to the ALU
- The ALU may also set flags as the result of an operation
- For example, an overflow flag is set to 1 if the result of a computation exceeds the length of the register into which it is to be stored

ALU

Typical Schematic Symbol of an ALU

A and B: the inputs to the ALU
R: Output or Result
F: Code or Instruction from the Control Unit
D: Output status; it indicates cases



- The processor provides signals that control the operation of the ALU and the movement of the data into and out of the ALU.

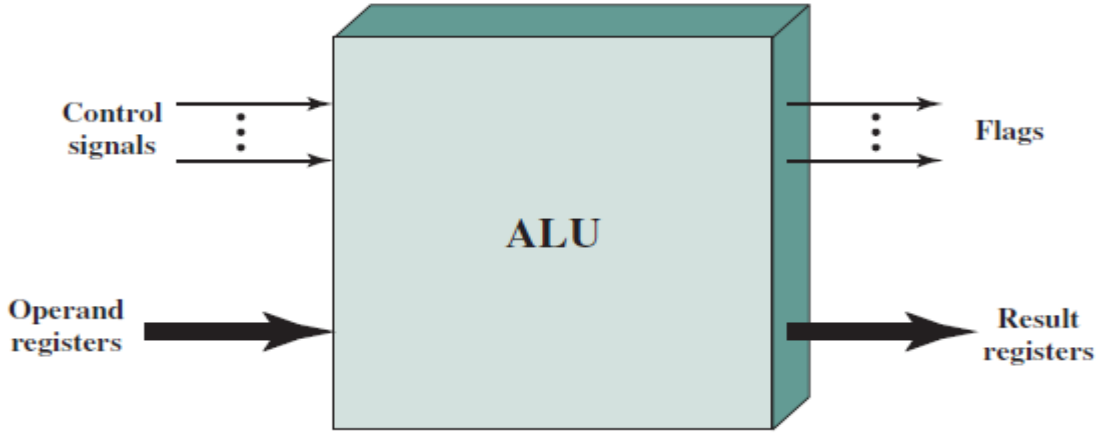


Figure 10.1 ALU Inputs and Outputs

Flags

- Z = 1 result is zero
- N = 1 result is negative
- V = 1 signed overflow
- C = 1 carry out

Module 3 – Data Path and Control Unit
-Hardwired Control
- Microprogrammed Control

Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

Datapath and Control

- CPU can be divided into Data section & Control Section

Control Section issues control signals to the datapath

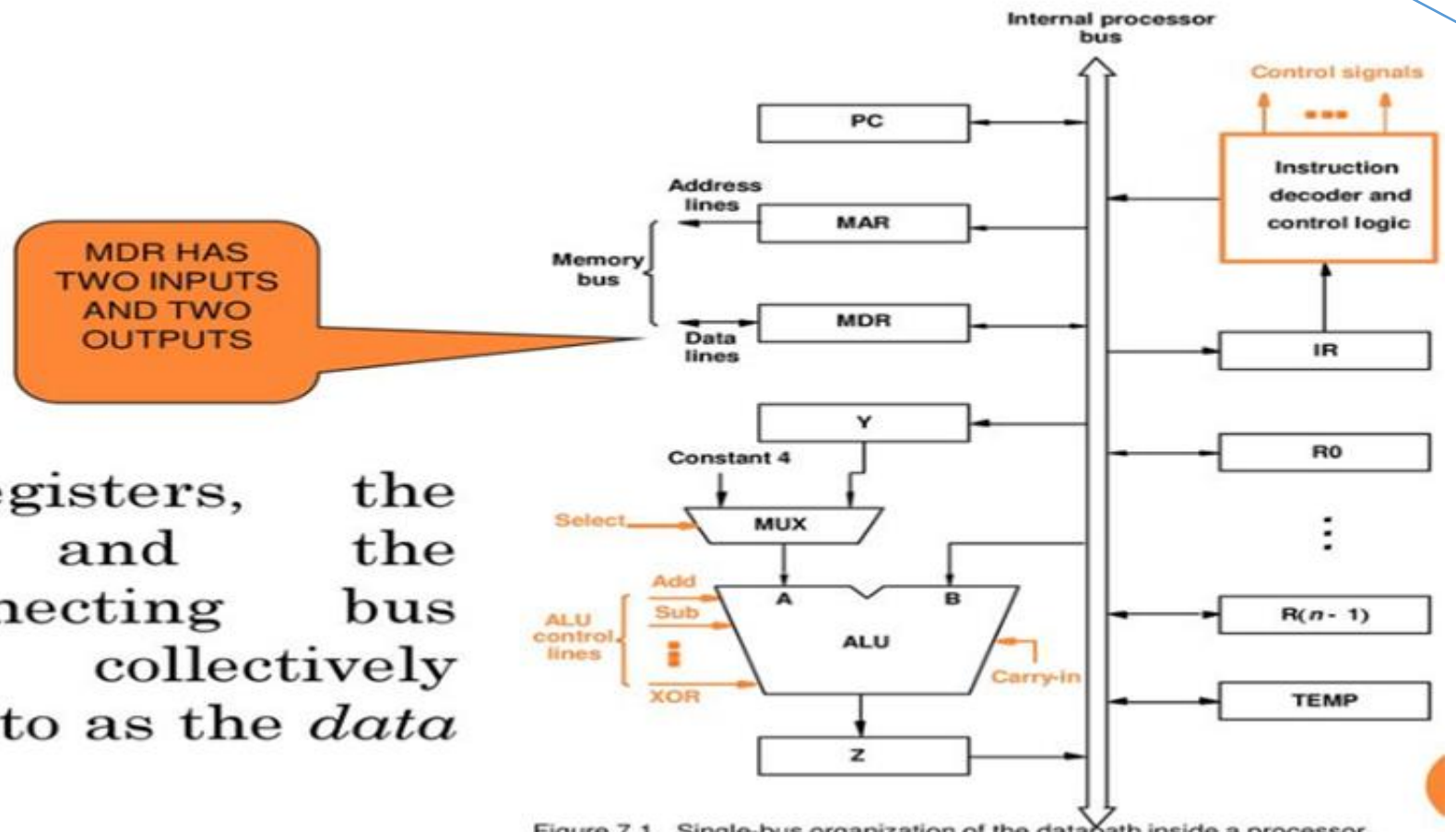
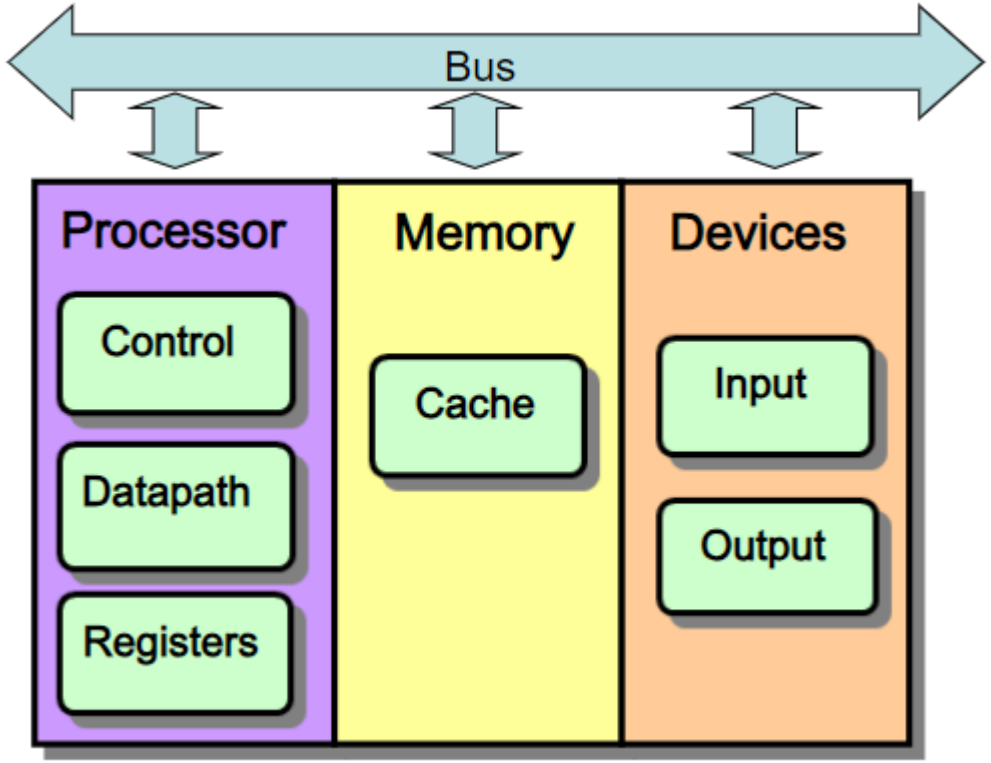


Figure 7.1. Single-bus organization of the datapath inside a processor.

- The registers, the ALU, and the interconnecting bus are collectively referred to as the *data path*.

Recap : Data Path and Control



Fundamental Concepts

- Processor (CPU): the active part of the computer, which does all the work (data manipulation and decision-making).
- Datapath: portion of the processor which contains hardware necessary to perform all operations required by the computer (the brawn).
- Control: portion of the processor (also in hardware) which tells the datapath what needs to be done (the brain).

Data Path and Control

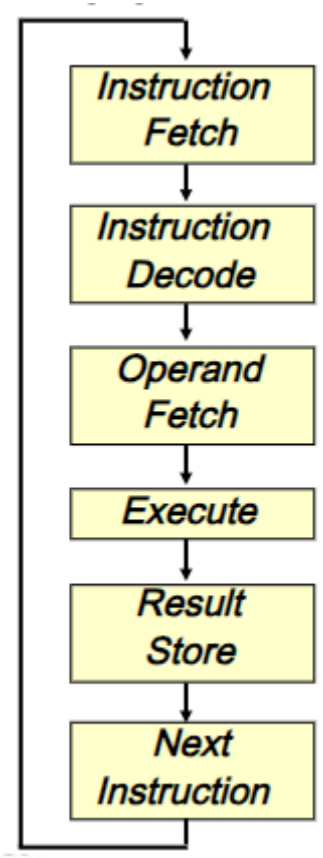
- To execute an instruction, a processor must perform the following 3 steps:
  - Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR **(Fetch phase).**

$$IR \leftarrow [[PC]]$$

  - Assuming that the memory is byte addressable, increment the contents of the PC by 4 (fetch phase).

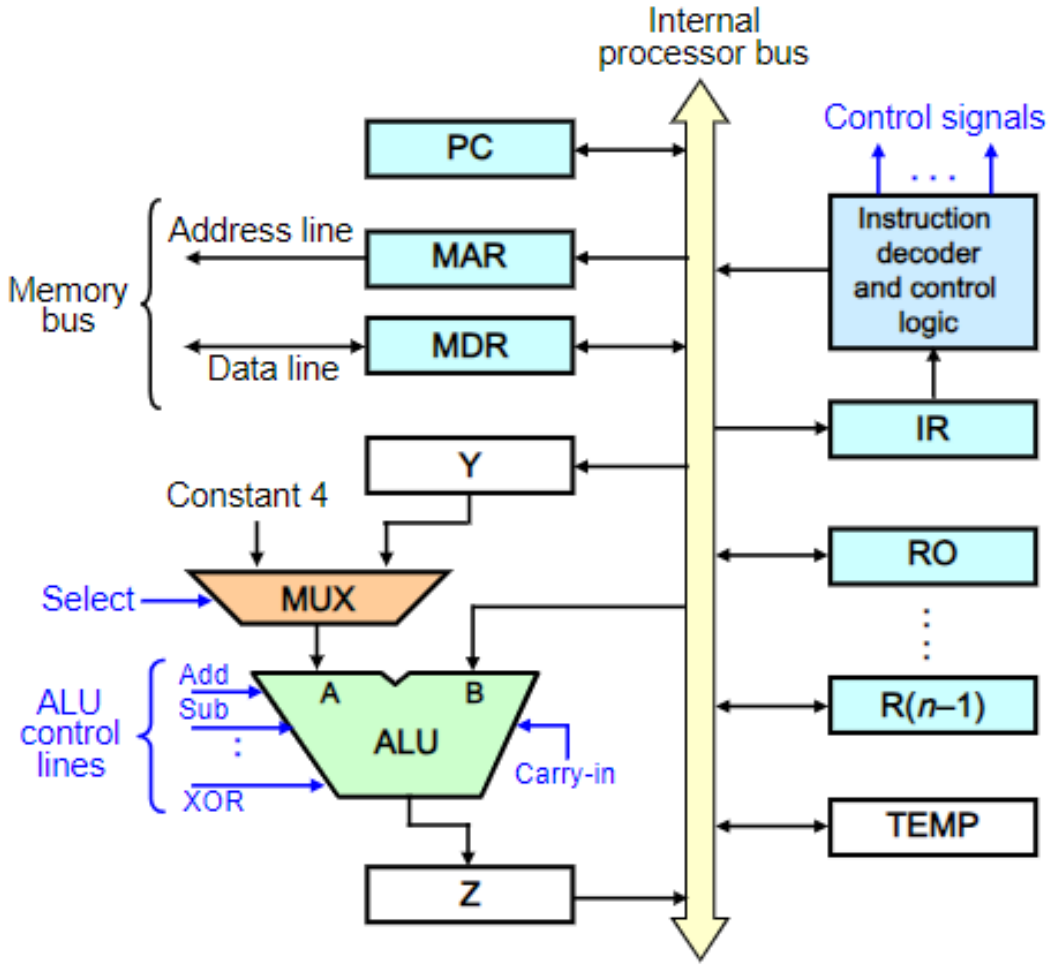
$$PC \leftarrow [PC] + 4$$

  - Carry out the actions specified by the instruction in the IR **(execution phase).**



Data Path and Control – Single Bus

Single-bus Organization



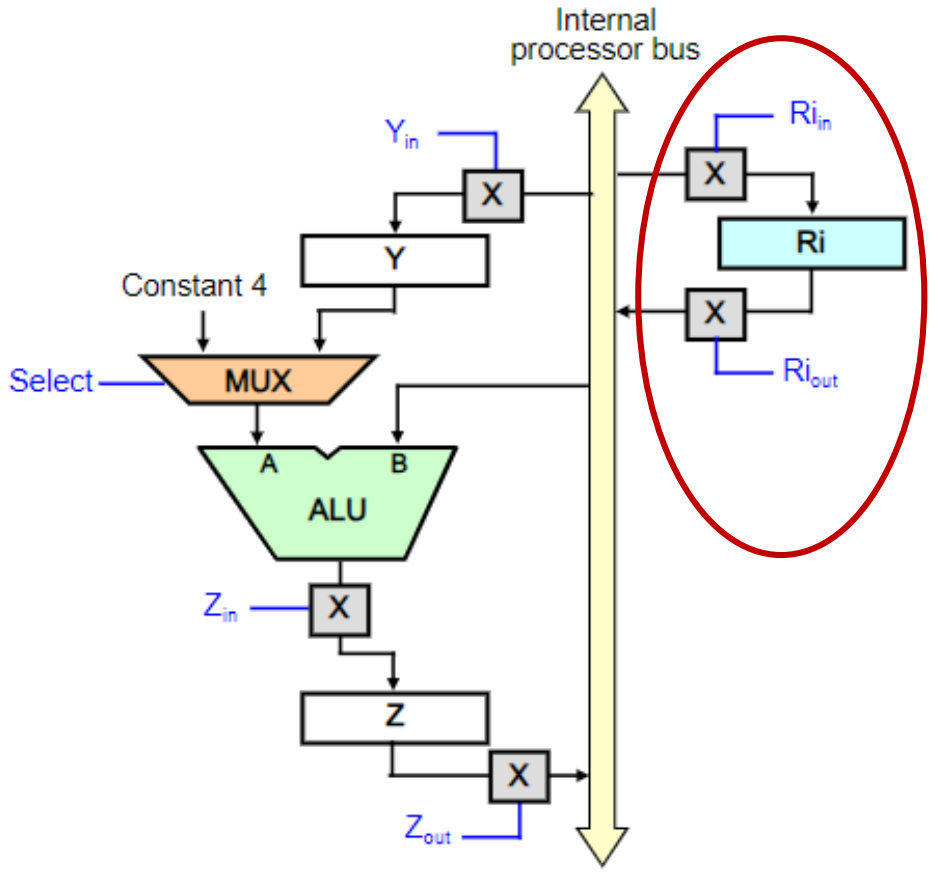
Instruction Execution

- An instruction can be executed by performing one or more of the following operations in some specified sequence:
  - Transfer a word of data from one register to another or to the ALU (Arithmetic Logic Unit). **Register Transfer**
  - Perform an arithmetic or a logic operation and store the result in a register. **ALU Operation**
  - Fetch the contents of a given memory location and load them into a register. **Reading a word from memory**
  - Store a word of data from a register into a given memory location. **Storing a word in memory**

Data Path and Control – Single Bus

Register Transfer

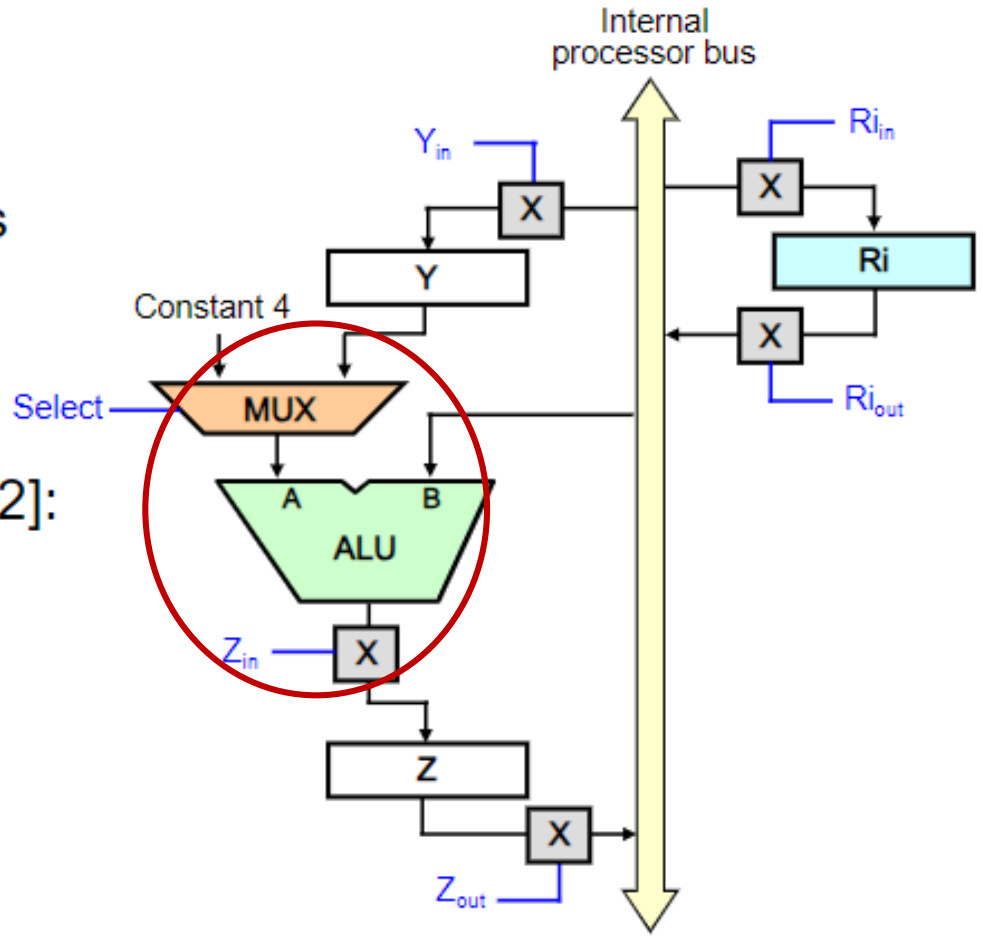
- Register to register transfer:
  - For each register Ri, two control signals:
    - Ri_{in} used to load the data on the bus into the register.
    - Ri_{out} to place the register's contents on the bus.
  - Example: To transfer contents of R1 to R4:
    - Set $R1_{out}$ to 1. This places contents of R1 on the bus.
    - Set $R4_{in}$ to 1. This loads data from the processor bus into R4.

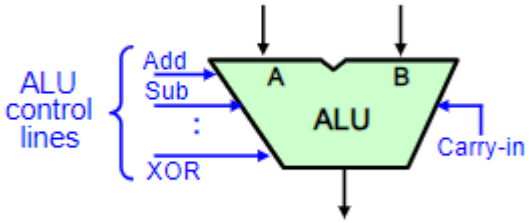


Data Path and Control – Single Bus

Arithmetic/Logic Operation

- ALU: Performs arithmetic and logic operations on its A and B inputs.
- To perform R3 ← [R1] + [R2]:
  1. $R1_{out}$, Y_{in}
  2. $R2_{out}$, SelectY, Add, Z_{in}
  3. Z_{out}, $R3_{in}$

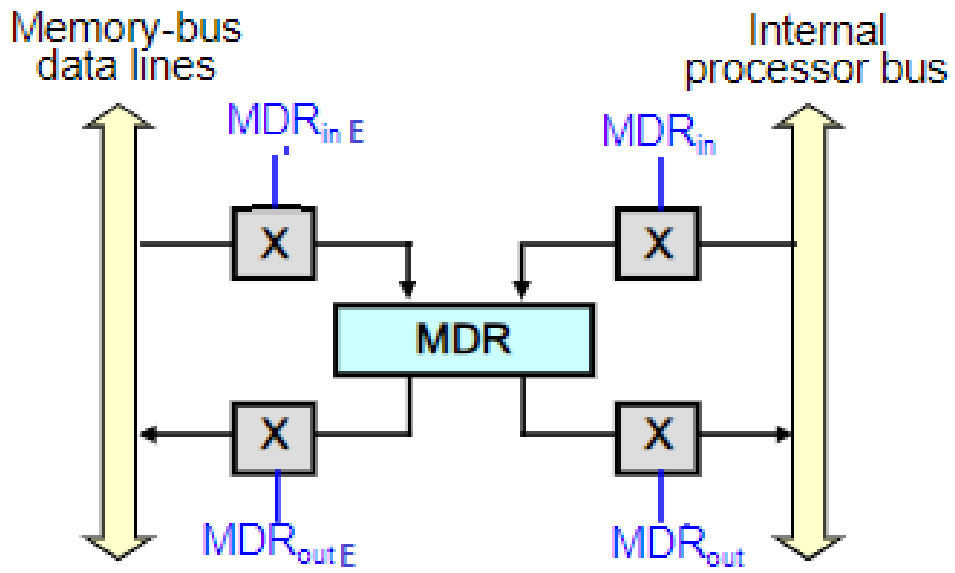




Data Path and Control – Single Bus

Reading a Word from Memory

- Move (R1), R2 /* R2 ← [[R1]]
  1. MAR ← [R1]
  2. Start a Read operation on the memory bus
  3. Wait for the MFC response from the memory
  4. Load MDR from the memory bus
  5. R2 ← [MDR]
- MDR has four control signals: MDR_{in}, MDR_{out}, MDR_{inE} and MDR_{outE}.



- Move (R1), R2 /* R2 ← [[R1]]
- Sequence of control steps:
  1. $R1_{out}$, MAR_{in}, Read
  2. MDR_{inE}, WMFC
  3. MDR_{out}, $R2_{in}$
- WMFC: Wait for arrival of MFC (Memory-Function-Completed) signal.
- MFC: To accommodate variability in response time, the processor waits until it receives an indication that the Read/Write operation has been completed. The addressed device sets MFC to 1 to indicate this.

Storing a Word in Memory

- Move R2, (R1) /* [R1] ← [R2]
- Sequence of control steps:
  1. $R1_{out}$, MAR_{in}
  2. $R2_{out}$, MDR_{in}, Write
  3. MDR_{outE}, WMFC

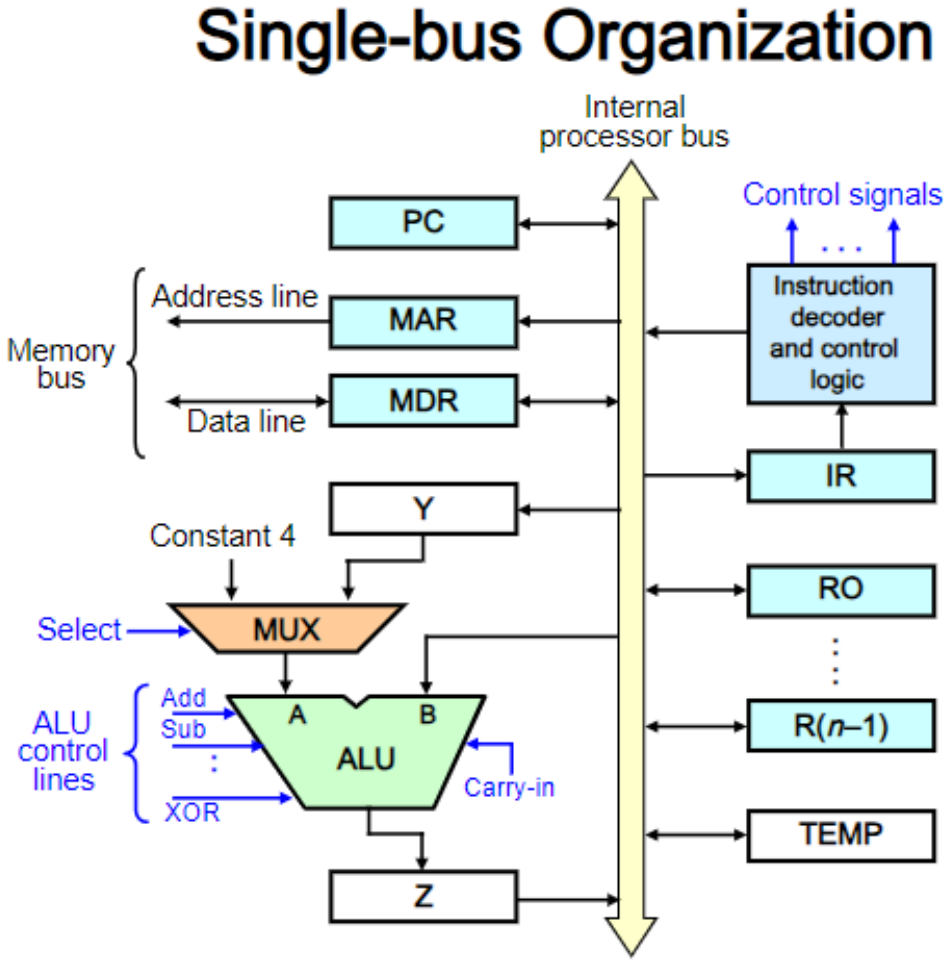
Data Path and Control – Single Bus

- Add (R3), R1 /* R1 ← [R1] + [[R3]]
- Adds the contents of a memory location pointed to by R3 to register R1.
- Sequence of control steps:

1. PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2. Z_{out}, PC_{in}, WMFC
3. MDR_{out}, IR_{in}
4. $R3_{out}$, MAR_{in}, Read
5. $R1_{out}$, Y_{in}, WMFC
6. MDR_{out}, SelectY, Add, Z_{in}
7. Z_{out}, $R1_{in}$, End

Steps 1 – 3: Instruction fetch

Instruction Execute

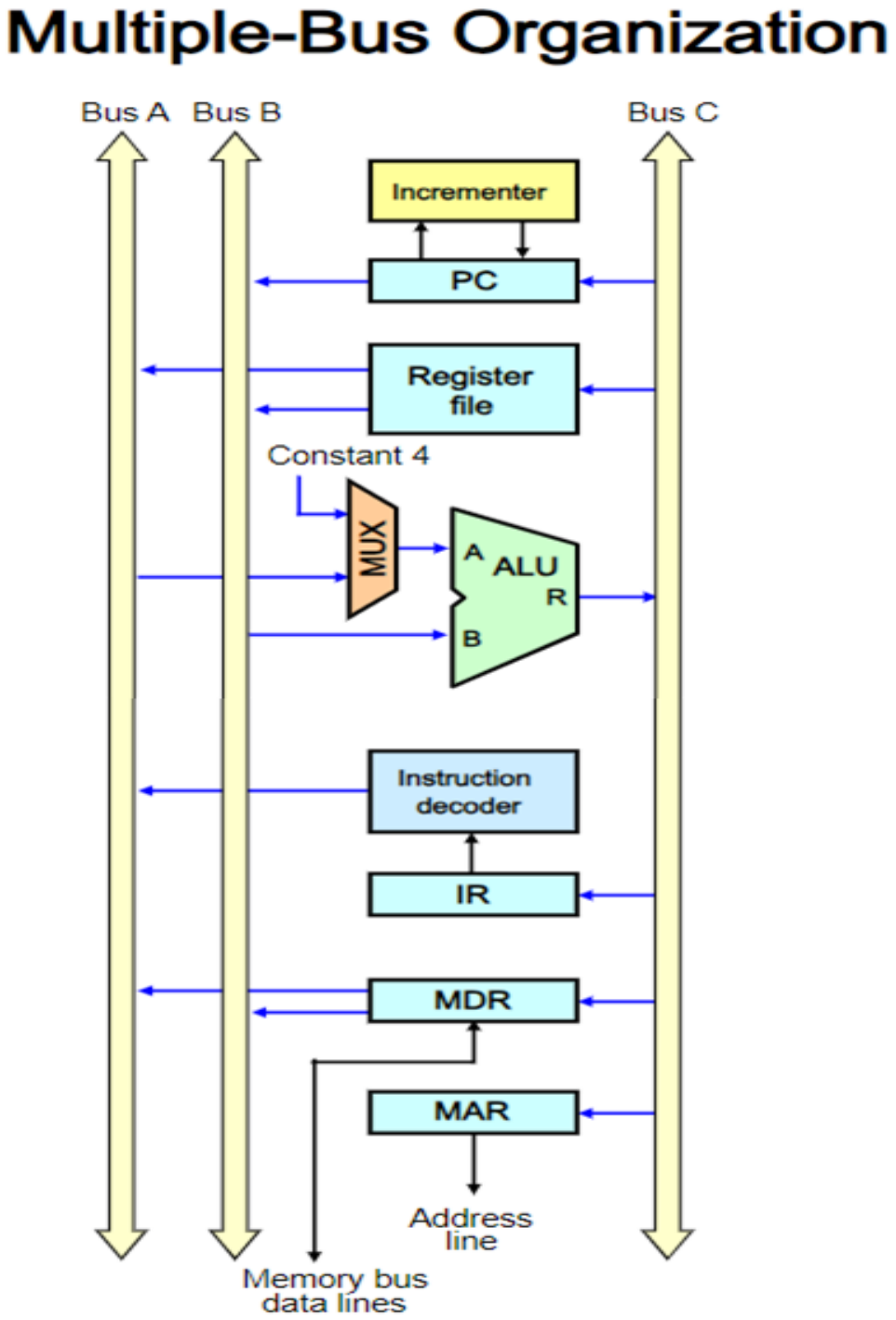


Data Path and Control – Multiple Bus

- Single-bus structure: Control sequences are long as only one data item can be transferred over the bus in a clock cycle.

Multiple Bus

- All registers are combined into a single block called register file with three ports: 2 outputs allowing 2 registers to be accessed simultaneously and have their contents put on buses A and B, and 1 input allowing data on bus C to be loaded into a third register.
- Buses A and B are used to transfer source operands to the A and B inputs of ALU, and result transferred to destination over bus C.

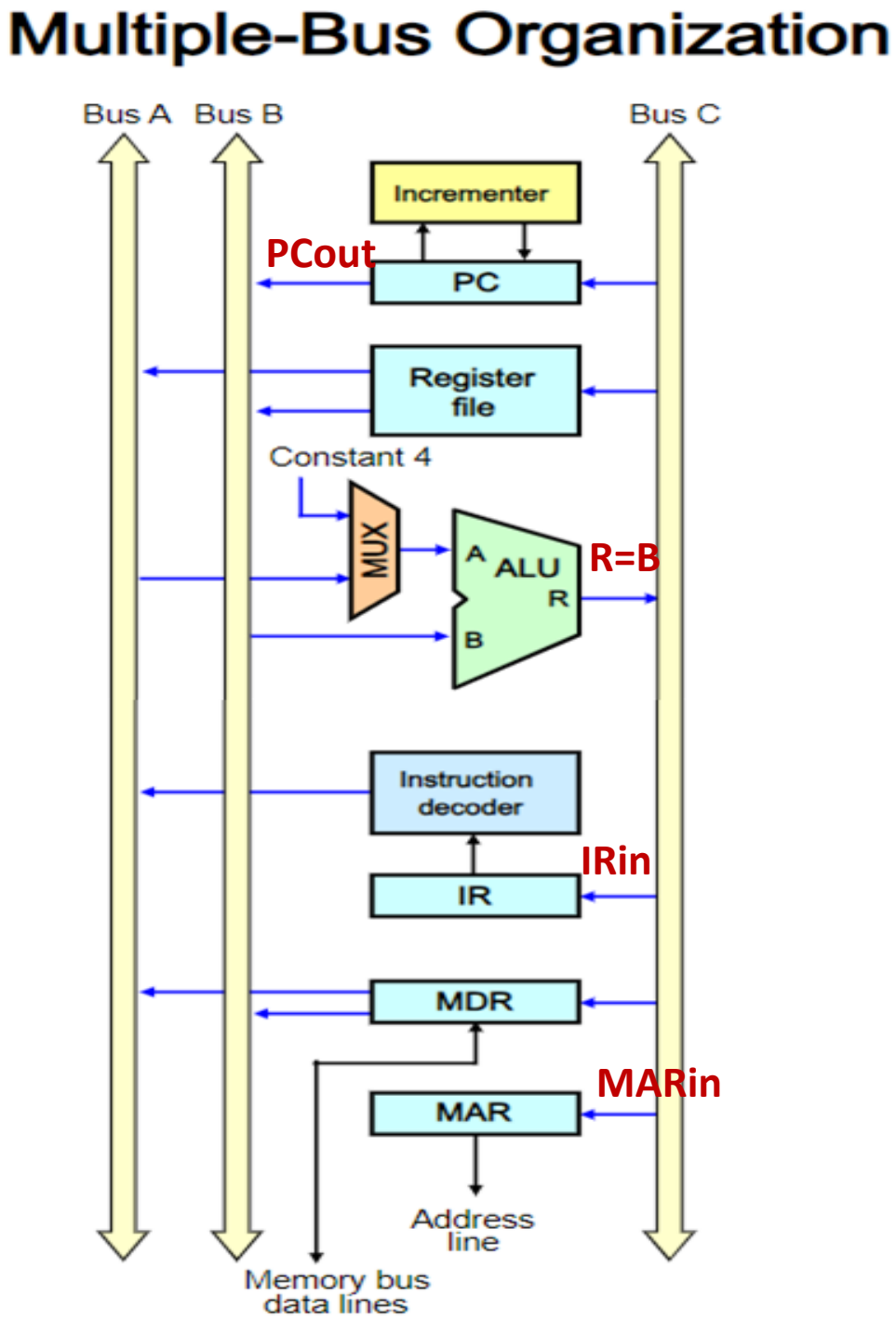


Data Path and Control – Multiple Bus

- For the ALU, R=A (or R=B) means that its A (or B) input is passed unmodified to bus C.
- Add R4, R5, R6 /* R6 ← [R4] + [R5]
  - Adds the contents of R4 and R5 to R6.
- Sequence of control steps:
  1. PC_{out}, R=B, MAR_{in}, Read, IncPC
  2. WMFC
  3. MDR_{outB}, R=B, IR_{in}
  4. $R4_{outA}$, $R5_{outB}$, SelectA, Add, $R6_{in}$, End

Instruction fetch

Instruction Execute

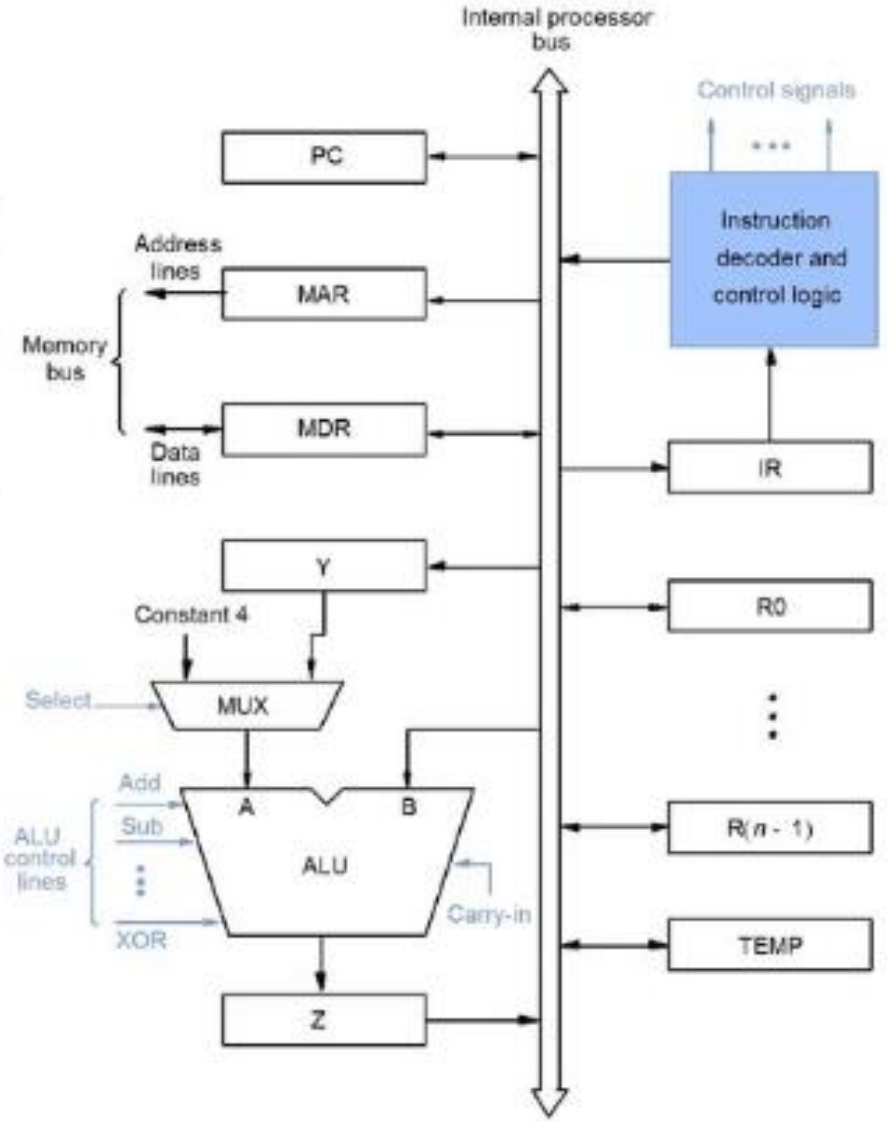


QUIZ

- What is the control sequence for execution of the instruction

  Add R1, R2

  including the instruction fetch phase? (Assume single bus architecture)



Revisit - Stages of Data Path

- Problem: a single, atomic block which "executes an instruction" (performs all necessary operations beginning with fetching the instruction) would be too bulky and inefficient.
- Solution: break up the process of "executing an instruction" into stages, and then connect the stages to create the whole datapath.
  - Smaller stages are easier to design.
  - Easy to optimize (change) one stage without touching the others.
- There is a *wide* variety of MIPS instructions: so what general steps do they have in common?
- Stages
  1. Instruction Fetch
  2. Instruction Decode
  3. ALU
  4. Memory Access
  5. Register Write

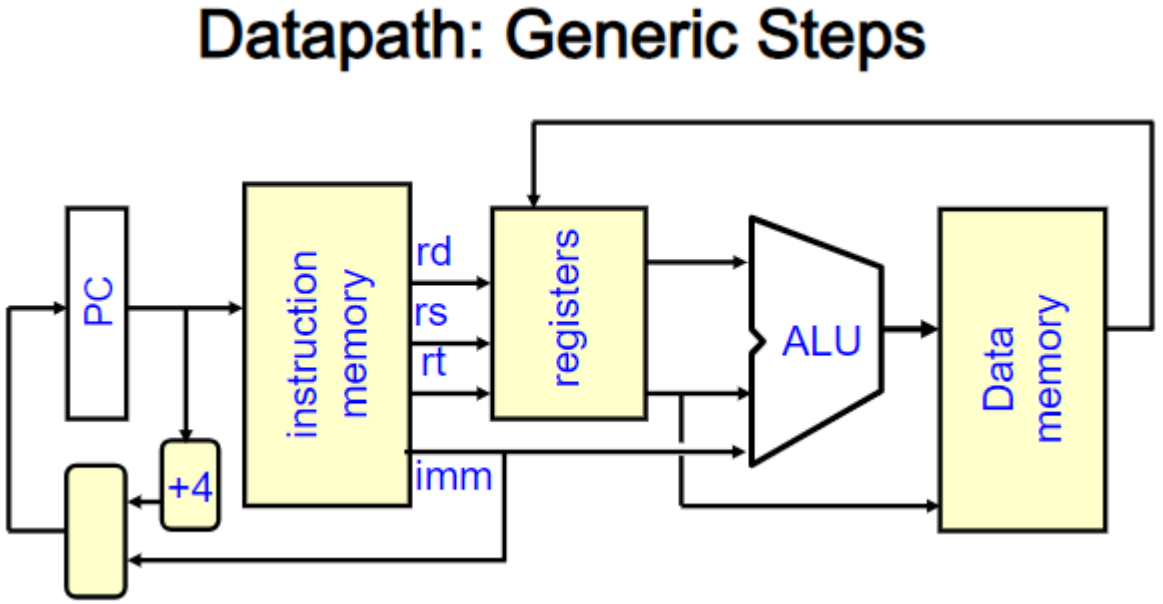
Stages of Data Path - Examples

- Stage 1: **Instruction Fetch** .
  - No matter what the instruction is, the 32-bit instruction word must first be fetched from memory (the cache-memory hierarchy).
  - Also, this is where we **increment PC** (that is, PC = PC + 4, to point to the next instruction; byte addressing so + 4).
- Stage 2: **Instruction Decode**
  - Upon fetching the instruction, we next gather data from the fields (*decode* all necessary instruction data).
  - First, read the **opcode** to determine instruction type and field lengths.
  - Second, read in data from all necessary registers.
    - For `add`, read two registers.
    - For `addi`, read one register.
    - For `jal`, no read necessary.

jump-and-link (JAL) instruction is a simple datapath that branches the PC by a specified offset

- Stage 3: **ALU** (Arithmetic-Logic Unit)
  - The real work of most instructions is done here: arithmetic (+, -, *, /), shifting, logic (&, |), comparisons (`slt`).
  - What about loads and stores?
    - `lw $t0, 40($t1)`
    - The address we are accessing in memory = the value in `$t1` plus the value 40.
    - We do this addition at this stage.
- Stage 4: **Memory Access**
  - Actually only the load and store instructions do anything during this stage; for the other instructions, they remain idle during this stage.
  - Since these instructions have a unique step, we need this extra stage to account for them.
  - As a result of the cache system, this stage is expected to be just as fast (on average) as the others.
- Stage 5: **Register Write**
  - Most instructions write the result of some computation into a register.
  - Examples: arithmetic, logical, shifts, loads, `slt`
  - What about stores, branches, jumps?
    - They do not write anything into a register at the end.
    - These remain idle during this fifth stage.

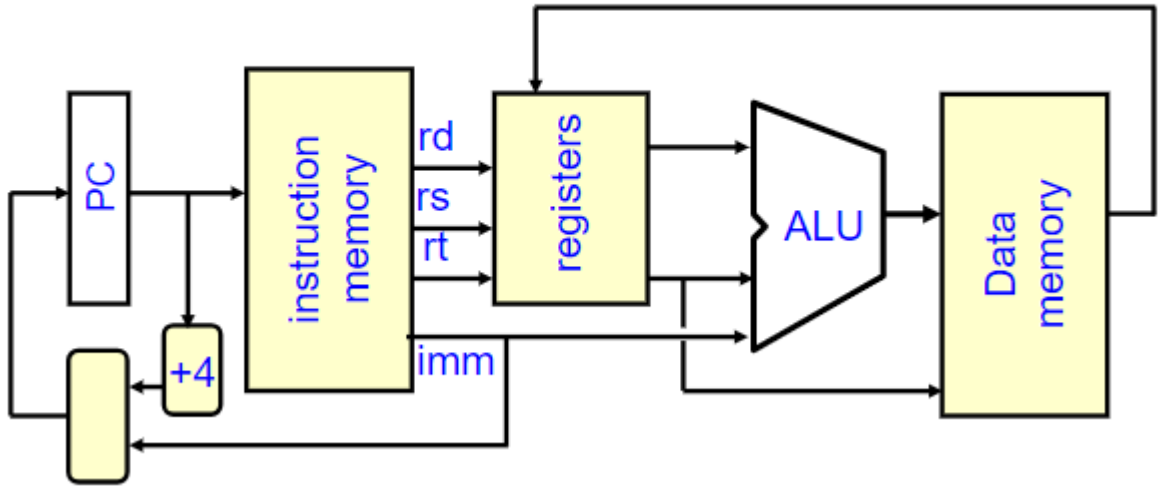
Stages of Data Path - Examples



- `add $r3,$r1,$r2 # r3 = r1+r2`
  - Stage 1: Fetch this instruction, increment PC.
  - Stage 2: Decode to find that it is an `add` instruction, then read registers `$r1` and `$r2`.
  - Stage 3: Add the two values retrieved in stage 2.
  - Stage 4: Idle (nothing to write to memory).
  - Stage 5: Write result of stage 3 into register `$r3`.

- `slti $r3,$r1,17`
  - Stage 1: Fetch this instruction, increment PC.
  - Stage 2: Decode to find it is an `slti`, then read register `$r1`.
  - Stage 3: Compare value retrieved in stage 2 with the integer 17.
  - Stage 4: Go idle.
  - Stage 5: Write the result of stage 3 in register `$r3`.

Stages of Data Path - Examples



- `sw $r3, 20($r1)`
  - Stage 1: Fetch this instruction, increment PC.
  - Stage 2: Decode to find it is an `sw`, then read registers `$r1` and `$r3`.
  - Stage 3: Add 20 to value in register `$r1` (retrieved in stage 2).
  - Stage 4: Write value in register `$r3` (retrieved in stage 2) into memory address computed in stage 3.
  - Stage 5: Go idle (nothing to write into a register).

Stages of Data Path - Examples

Why Five Stages?

- Could we have a different number of stages?
  - Yes, and other architectures do.
- So why does MIPS have five stages, if instructions tend to go idle for at least one stage?
  - There is one instruction that uses all five stages: the load.
- `lw $r3, 40($r1)`
  - Stage 1: Fetch this instruction, increment PC.
  - Stage 2: Decode to find it is a `lw`, then read register `$r1`.
  - Stage 3: Add 40 to value in register `$r1` (retrieved in stage 2).
  - Stage 4: Read value from memory address compute in stage 3.
  - Stage 5: Write value found in stage 4 into register `$r3`.

- Construct datapath based on register transfers required to perform instructions.
- Control part causes the right transfers to happen.

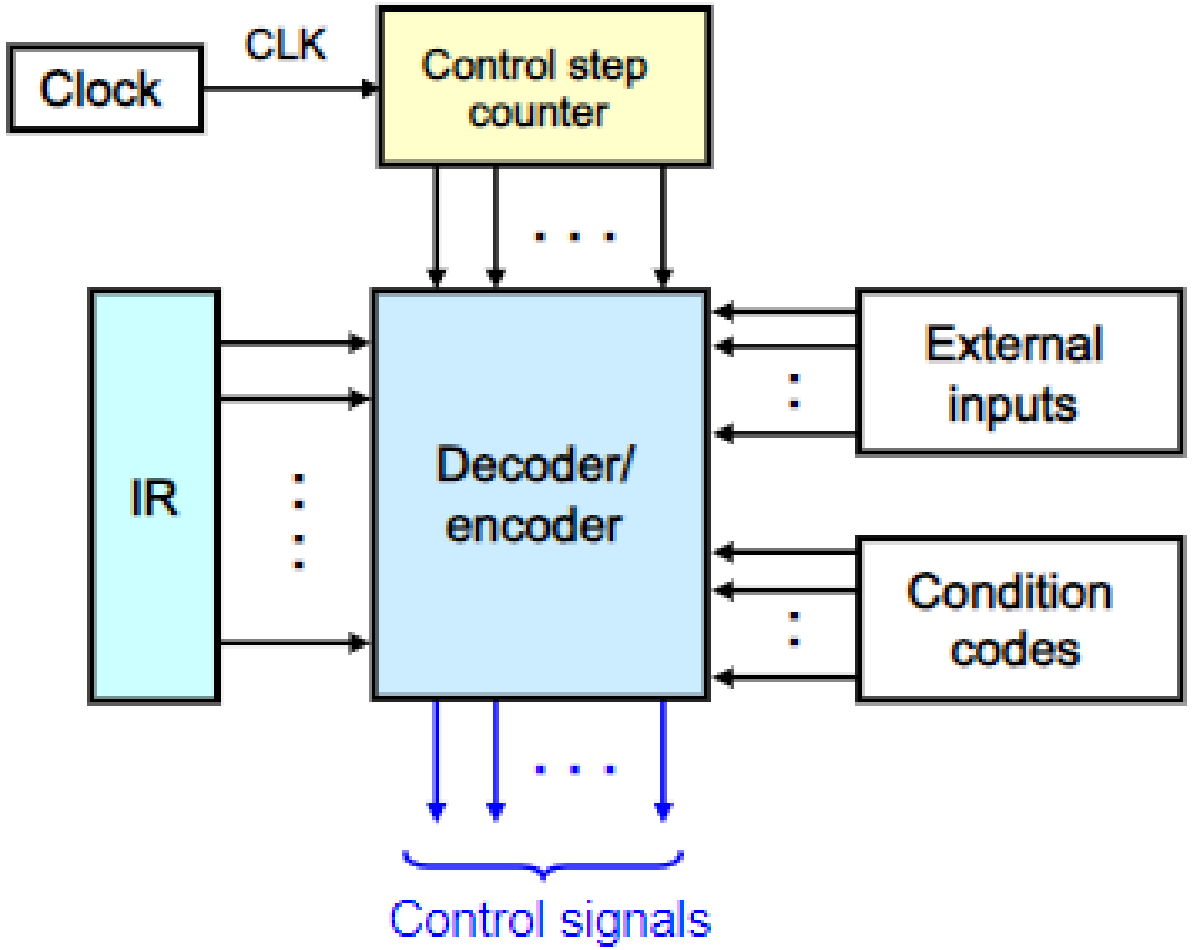
Control Unit

- **Control Unit (CU) -** supplies the data required by the ALU from memory, or from input devices, and directs the ALU to perform a specific operation based on the instruction fetched from the memory
- To execute instructions – processor – generates control signals in proper sequence
- Two ways – to generate control signals in sequence
  - Hardwired Control
  - Microprogrammed Control

Hardwired Control

- Operates at high speed
- Little flexibility

- The sequence of control signals step is complete in one clock period.
- A counter may be used to keep track of the control steps
- Each state, or count, of this counter corresponds to one control step.
- The required control signals are determined by the following information:
  1. Contents of the control step counter
  2. Contents of the instruction register
  3. Contents of the condition code flags
  4. External input signals, such as MFC and interrupt requests

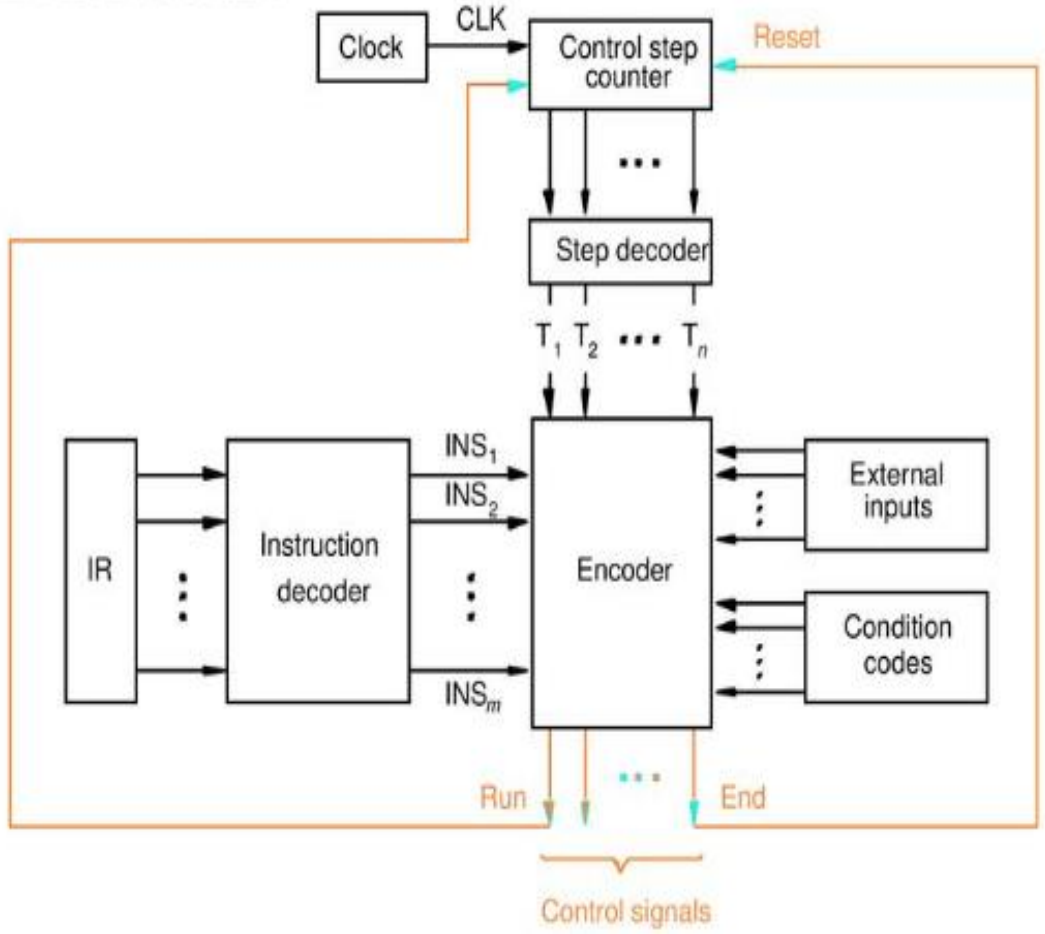


Hardwired Control

Separate Decoder and Encoder

- The step decoder provides a separate signal line for each step, or time slot, in the control sequence.
- Similarly, the output of the instruction decoder consists of a separate line for each machine instruction. For any instruction loaded in the IR, one of the output lines INS1 through INSm is set to 1, and all other lines are set to 0.
- The input signals to the encoder block in diagram combined to generate the individual control signals Yin, PCout, Add, End, and so on.

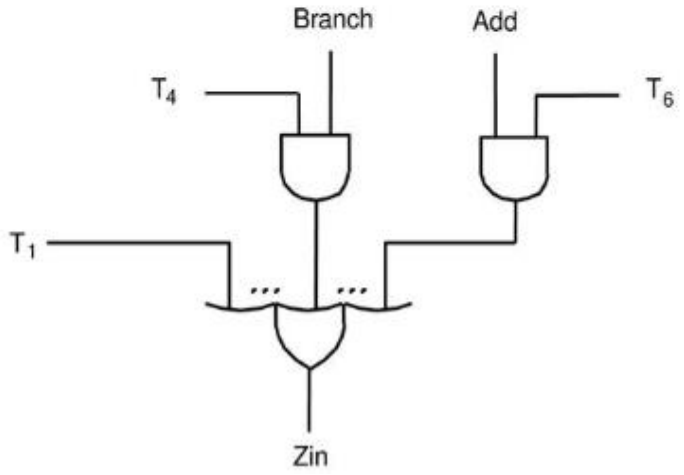
BY SEPARATING THE DECODING AND ENCODING FUNCTIONS, WILL OBTAIN THE MORE DETAILED BLOCK DIAGRAM



Hardwired Control

Generating Z_{IN}

- $Z_{in} = T_1 + T_6 \cdot ADD + T_4 \cdot BR + \ldots$
- This signal is asserted during time slot T1 for all instructions, during T6 for an Add instruction, during T4 for an unconditional branch instruction, and so on. The logic function for Zin is derived from the control sequences



Generating End

- $End = T_7 \cdot ADD + T_5 \cdot BR + (T_5 \cdot N + T_4 \cdot \overline{N}) \cdot BRN + \ldots$
- The End signal starts a new instruction fetch cycle by resetting the control step counter to its starting value.
- Set to 1, RUN causes the counter to be incremented by one at the end of every clock cycle. When RUN is equal to 0, the counter stops counting.
- This is needed whenever the WMFC signal is issued, to cause the processor to wait for the reply from the memory.

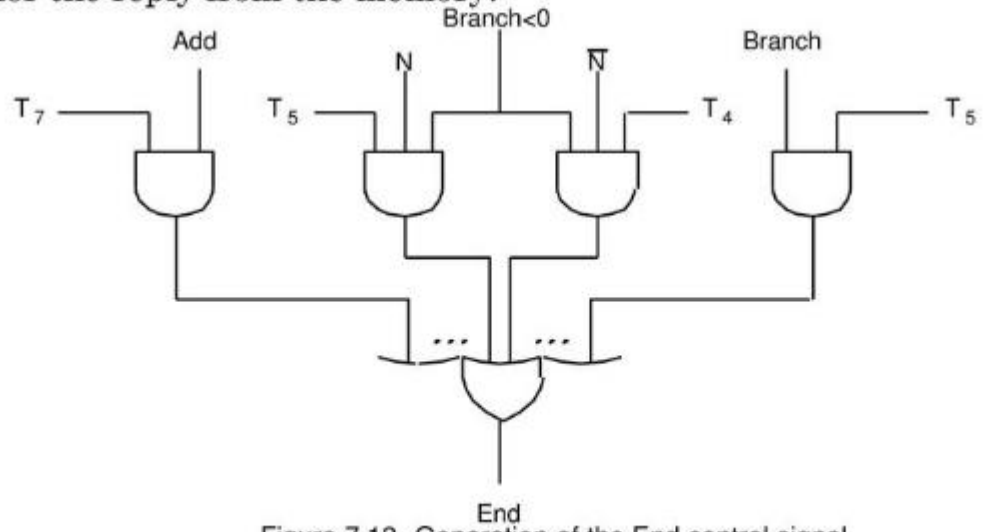


Figure 7.13. Generation of the End control signal.

Microprogrammed Control

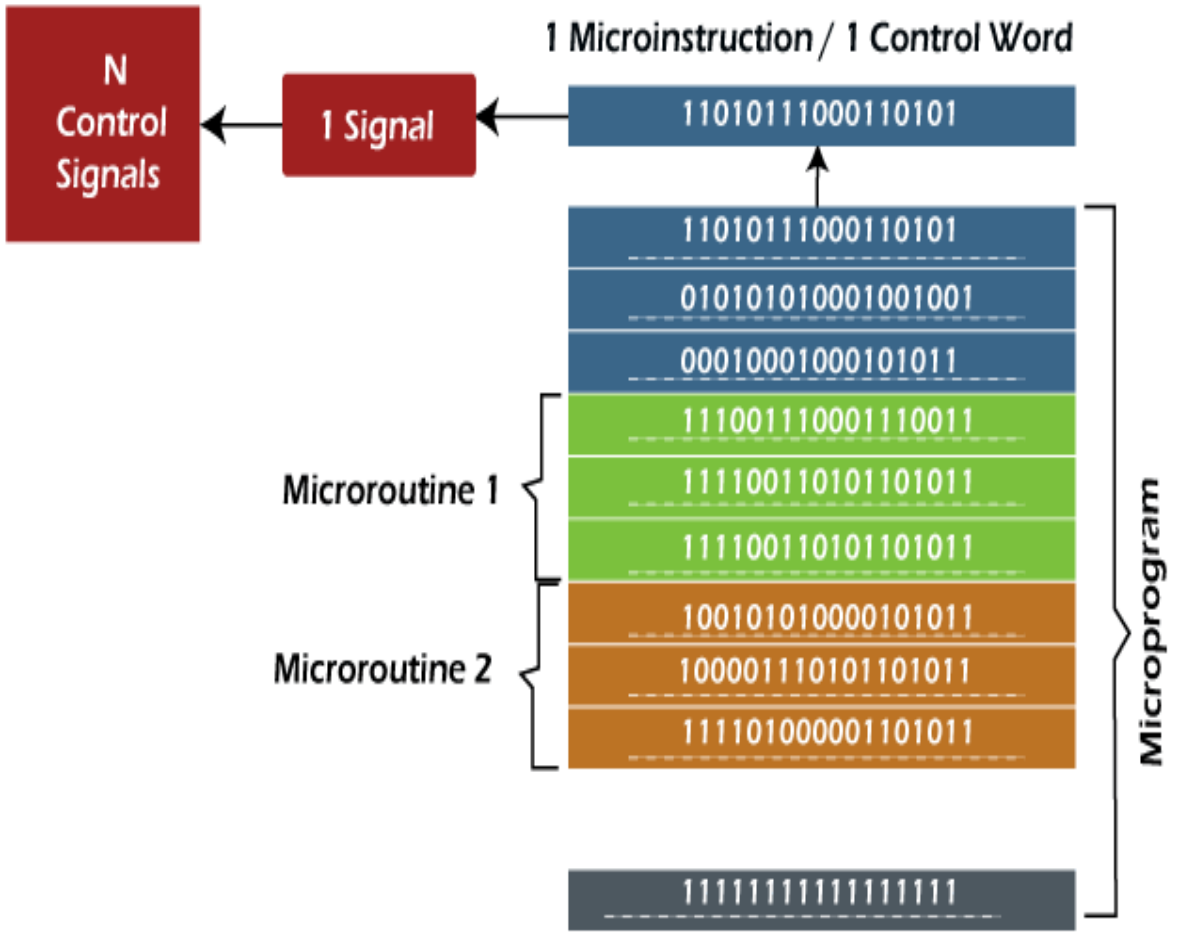
- Control signals are generated by a program similar to machine language programs.
- Control Word (CW); microroutine; microinstruction

Micro-instruction	..	PC_{in}	PC_{out}	MAR_{in}	Read	MDR_{out}	IR_{in}	Y_{in}	Select	Add	Z_{in}	Z_{out}	$R1_{out}$	$R1_{in}$	$R3_{out}$	WMFC	End	..
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	1. PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2		1	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	2. Z_{out}, PC_{in}, WMFC
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	3. MDR_{out}, IR_{in}
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	4. $R3_{out}$, MAR_{in}, Read
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	5. $R1_{out}$, Y_{in}, WMFC
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	6. MDR_{out}, SelectY, Add, Z_{in}
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	7. Z_{out}, $R1_{in}$, End

- Popular in CISC because complex instruction sets require complex controllers that can more easily be implemented as microprograms.

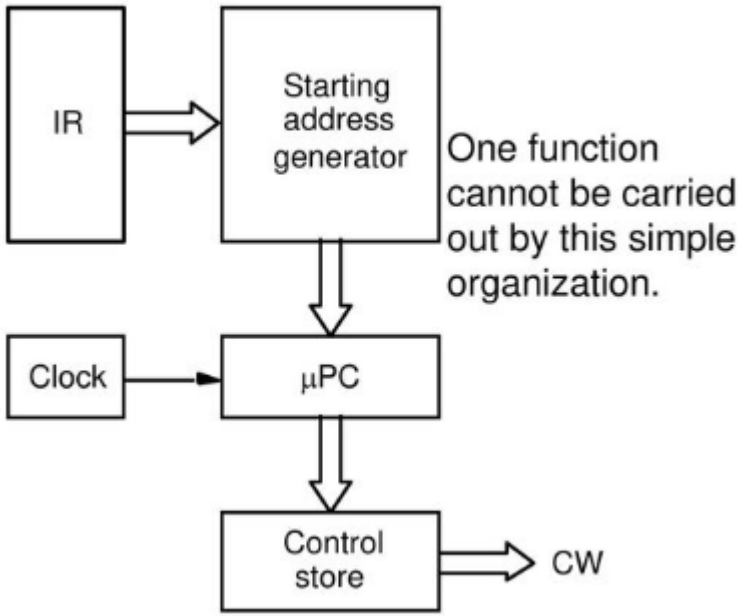
Microprogrammed Control

1. **Control Word:** A control word is a word whose individual bits represent various control signals.
2. **Micro-routine:** A sequence of control words corresponding to the control sequence of a machine instruction constitutes the micro-routine for that instruction.
3. **Micro-instruction:** Individual control words in this micro-routine are referred to as microinstructions.
4. **Micro-program:** A sequence of micro-instructions is called a micro-program, which is stored in a ROM or RAM called a Control Memory (CM).
5. **Control Store:** the micro-routines for all instructions in the instruction set of a computer are stored in a special memory called the Control Store.



Microprogrammed Control

Basic Organization of Microprogrammed Control Unit

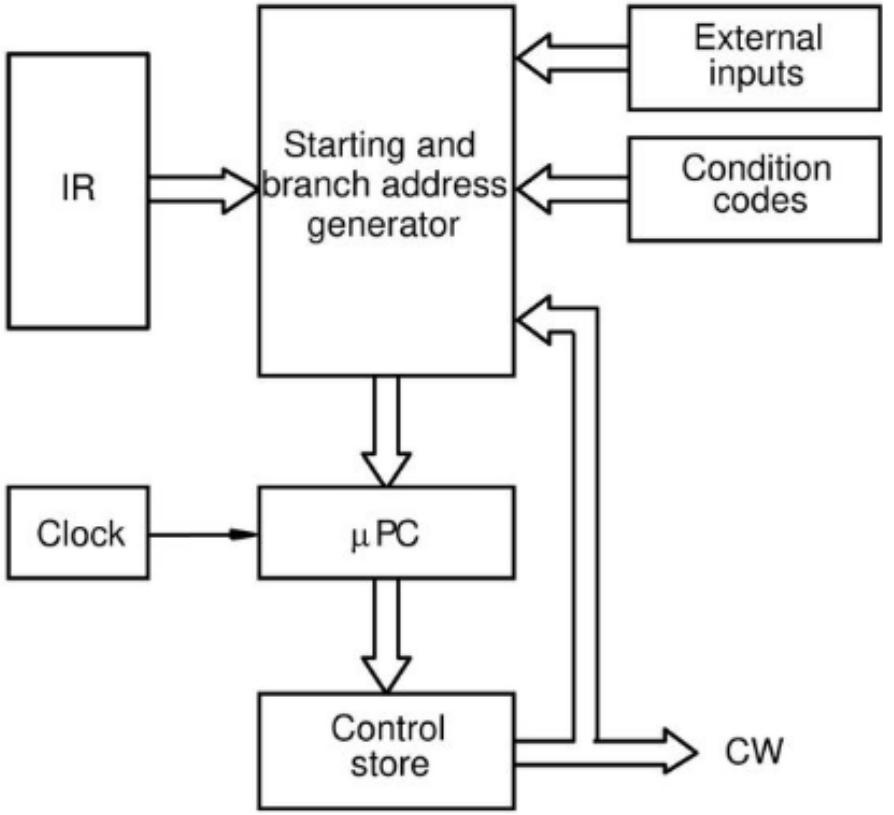


- The control unit can generate the control signals for any instruction by sequentially reading the CWs of the corresponding micro routine from the control store.
- To read the control words sequentially from the control store, a micro program counter (µPC) is used. Every time a new instruction is loaded into the IR, the output of the block labeled "starting address generator" is loaded into the µPC.
- The µPC is then automatically incremented by the clock, causing successive microinstructions to be read from the control store.
- Hence, the control signals are delivered to various parts of the processor in the correct sequence.

Microprogrammed Control

- The previous organization cannot handle the situation when the control unit is required to check the status of the condition codes or external inputs to choose between alternative courses of action.
- Use conditional branch microinstruction.

Organization of the control unit to allow conditional branching in the microprogram



Microprogrammed Control – Branch Inst.

Address	Microinstruction
0	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
1	Z_{out} , PC_{in} , Y_{in} , WMFC
2	MDR_{out} , IR_{in}
3	Branch to starting address of appropriate microroutine
.....	
25	If N=0, then branch to microinstruction 0
26	Offset-field-of-IR_{out} , SelectY, Add, Z_{in}
27	Z_{out} , PC_{in} , End

Microroutine for the instruction Branch<0.

Micro - instruction	..	PC_{in}	PC_{out}	MAR_{in}	Read	MDR_{out}	IR_{in}	Y_{in}	Select	Add	Z_{in}	Z_{out}	$R1_{out}$	$R1_{in}$	$R3_{out}$	WMFC	End	..
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	

- The instruction Branch <0 loading this instruction into IR, a branch microinstruction transfers control to the corresponding micro routine, which is assumed to start at location 25 in the control store. This address is the output of staring address generator block codes.
- If this bit is equal to 0, a branch takes place to location 0 to fetch a new machine instruction. Otherwise, the microinstruction at location 0 to fetch a new machine instruction.
- Otherwise the microinstruction at location 27 loads this address into the PC.

- A straightforward way to structure microinstructions is to assign one bit position to each control signal.
- However, this is very inefficient.
- The length can be reduced: most signals are not needed simultaneously, and many signals are mutually exclusive.
- All mutually exclusive signals are placed in the same group in binary coding.
- Assigning individual bits to each control signal results in long microinstructions because the number of required signals is usually large.
- Moreover, only a few bits are set to 1 (to be used for active gating) in any given microinstruction, which means the available bit space is poorly used.

Microprogrammed Control

Partial Format for the Microinstructions-1

Microinstruction

F1	F2	F3	F4	F5

F1 (4 bits)	F2 (3 bits)	F3 (3 bits)	F4 (4 bits)	F5 (2 bits)
0000: No transfer	000: No transfer	000: No transfer	0000: Add	00: No action
0001: PC $_{out}$	001: PC $_{in}$	001: MAR $_{in}$	0001: Sub	01: Read
0010: MDR $_{out}$	010: IR $_{in}$	010: MDR $_{in}$	⋮	10: Write
0011: Z $_{out}$	011: Z $_{in}$	011: TEMP $_{in}$	1111: XOR	
0100: R0 $_{out}$	100: R0 $_{in}$	100: Y $_{in}$	(16 ALU functions)	
0101: R1 $_{out}$	101: R1 $_{in}$			
0110: R2 $_{out}$	110: R2 $_{in}$			
0111: R3 $_{out}$	111: R3 $_{in}$			
1010: TEMP $_{out}$				
1011: Offset $_{out}$				

F6	F7	F8	...

F6 (1 bit)	F7 (1 bit)	F8 (1 bit)
0: SelectY	0: No action	0: Continue
1: Select4	1: WMFC	1: End

- The length of the microinstructions can be reduced easily.
- Most signals are not needed simultaneously, and many signals are mutually exclusive. For e.g., only one function of the ALU can be activated at a time.
- The source for a data transfer must be unique because it is not possible to gate the contents of two different registers onto the bus at the same time.
- Read and Write signals to the memory cannot be active simultaneously.
- This suggests that signals can be grouped so that all mutually exclusive signals are placed in the same group. Thus, at most one *micro operation* per group is specified in any microinstruction.

Microprogrammed Control

Microinstructions with Next-Address Field

- The microprogram we discussed requires several branch microinstructions, which perform no useful operation in the datapath.
- A powerful alternative approach is to include an address field as a part of every microinstruction to indicate the location of the next microinstruction to be fetched.
- Pros: separate branch microinstructions are virtually eliminated; few limitations in assigning addresses to microinstructions.
- Cons: additional bits for the address field (around 1/6)

Microinstruction

F0	F1	F2	

F0 (8 bits)	F1 (3 bits)	F2 (3 bits)	F3 (3 bits)
Address of next microinstruction	000: No transfer 001: PC $_{out}$ 010: MDR $_{out}$ 011: Z $_{out}$ 100: Rsrc $_{out}$ 101: Rdst $_{out}$ 110: TEMP $_{out}$	000: No transfer 001: PC $_{in}$ 010: IR $_{in}$ 011: Z $_{in}$ 100: Rsrc $_{in}$ 101: Rdst $_{in}$	000: No transfer 001: MAR $_{in}$ 010: MDR $_{in}$ 011: TEMP $_{in}$ 100: Y $_{in}$

	F5	F6	F7

F4 (4 bits)	F5 (2 bits)	F6 (1 bit)	F7 (1 bit)
0000: Add 0001: Sub ⋮ 1111: XOR	00: No action 01: Read 10: Write	0: SelectY 1: Select4	0: No action 1: WMFC

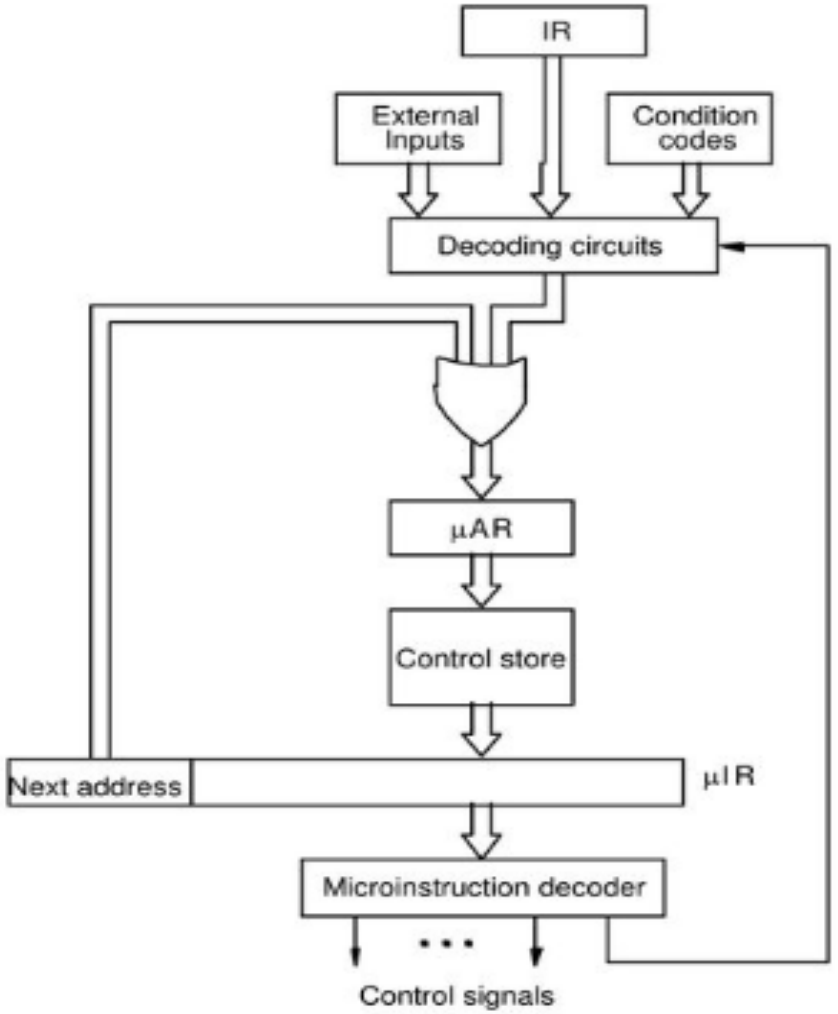
	F9	F10

F8 (1 bit)	F9 (1 bit)	F10 (1 bit)
0: NextAdrs 1: InstDec	0: No action 1: OR $_{mode}$	0: No action 1: OR $_{indsrc}$

Figure 7.23. Format for microinstructions in the example of Section 7.5.3.

Microprogrammed Control

Microinstructions with Next-Address Field



Microinstruction

F0	F1	F2	

F0 (8 bits)	F1 (3 bits)	F2 (3 bits)	F3 (3 bits)
Address of next microinstruction	000: No transfer 001: PC $_{out}$ 010: MDR $_{out}$ 011: Z $_{out}$ 100: Rsrc $_{out}$ 101: Rdst $_{out}$ 110: TEMP $_{out}$	000: No transfer 001: PC $_{in}$ 010: IR $_{in}$ 011: Z $_{in}$ 100: Rsrc $_{in}$ 101: Rdst $_{in}$	000: No transfer 001: MAR $_{in}$ 010: MDR $_{in}$ 011: TEMP $_{in}$ 100: Y $_{in}$

	F5	F6	F7

F4 (4 bits)	F5 (2 bits)	F6 (1 bit)	F7 (1 bit)
0000: Add 0001: Sub ⋮ 1111: XOR	00: No action 01: Read 10: Write	0: SelectY 1: Select4	0: No action 1: WMFC

	F9	F10

F8 (1 bit)	F9 (1 bit)	F10 (1 bit)
0: NextAdrs 1: InstDec	0: No action 1: OR $_{mode}$	0: No action 1: OR $_{indsrc}$

Figure 7.23. Format for microinstructions in the example of Section 7.5.3.

Microprogrammed Control

Further Improvement-1

- Enumerate the patterns of required signals in all possible microinstructions. Each meaningful combination of active control signals can then be assigned a distinct code.
- Vertical organization
- Horizontal organization

- Example of a horizontal organization scheme:

1. PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2. Z_{out}, PC_{in}, Y_{in}, WMFC
3. MDR_{out}, IR_{in}
4. $R3_{out}$, MAR_{in}, Read
5. $R1_{out}$, Y_{in}, WMFC
6. MDR_{out}, SelectY, Add, Z_{in}
7. Z_{out}, $R1_{in}$, End

Micro-instruction	::	PC_{in}	PC_{out}	MAR_{in}	Read	MDR_{out}	IR_{in}	Y_{in}	Select	Add	Z_{in}	Z_{out}	$R1_{out}$	$R1_{in}$	$R3_{out}$	WMFC	End	::
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	

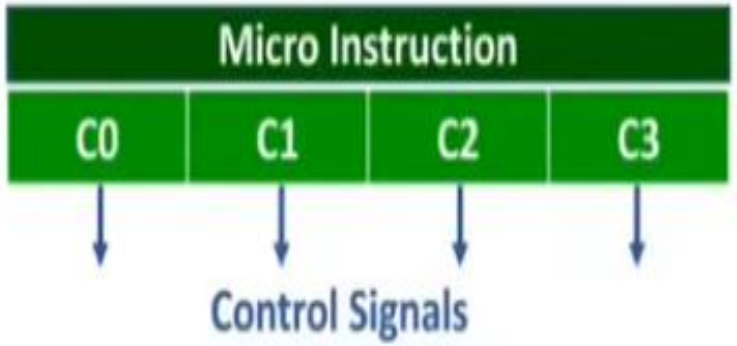
Select=0: SelectY
Select=1: Select4

- Highly encoded schemes that use compact codes to specify only a small number of control functions in each microinstruction are referred to as a ***vertical organization***.
- On the other hand, the minimally encoded scheme, in which many resources can be controlled with a single microinstruction, is called a ***horizontal organization***.
- The horizontal approach is useful when a higher operating speed is desired and when the machine structure allows parallel use of resources.
- The vertical approach results in considerably slower operating speeds because more microinstructions are needed to perform the desired control functions. Although fewer bits are required for each microinstruction, this does not imply that the total number of bits in the control store is smaller. The significant factor is that less hardware is needed to handle the execution of microinstructions.
- Horizontal and vertical organizations represent the two organizational extremes in micro programmed control.

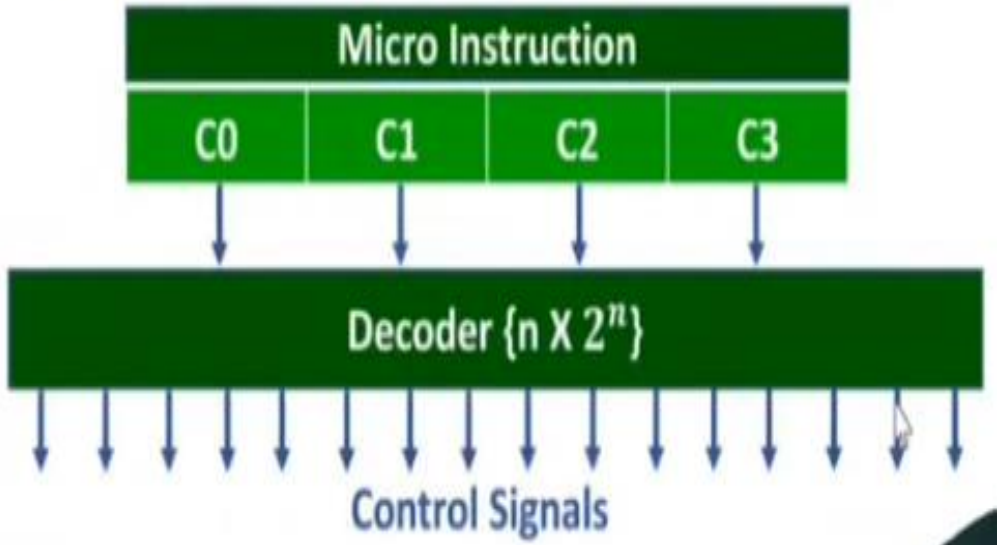
Microprogrammed Control

Horizontal Vs Vertical μInstruction

- Horizontal Microinstruction
  - Here every bit of microinstruction corresponds to control signal.
  - Based on value of instruction bits directly control signals are generated.



- Vertical Microinstruction
  - Here every bit of microinstruction have to be decoded for control signal generation.



Microprogrammed Control

Horizontal Microprogramming	Vertical Microprogramming
Horizontal microprogramming involves horizontal microinstructions that use no encoding at all.	The control bits are encoded in vertical microprogramming.
Horizontal microprogramming generally follows a sequential approach to specify the next microinstruction in a microprogram.	Vertical microprogramming may use a relatively addressing scheme.
Horizontal microprograms generally represent multiple micro-operations that are executed at the same time.	Each vertical microinstruction represents a single micro-operation, while operands may specify the data sink and source.
Horizontal microprograms offer improved flexibility because each control bit is independent of each other.	Vertical microinstructions are more compact but less flexible than horizontal microinstructions are easier to write.

Hardwired Vs Microprogrammed Control

Hardwired Control Unit	Microprogrammed Control Unit
Hardwired control unit generates the control signals needed for the processor using logic circuits	Microprogrammed control unit generates the control signals with the help of micro instructions stored in control memory
Hardwired control unit is faster when compared to microprogrammed control unit as the required control signals are generated with the help of hardware.	This is slower than the other as micro instructions are used for generating signals here
Difficult to modify as the control signals that need to be generated are hard wired	Easy to modify as the modification need to be done only at the instruction level
More costlier as everything has to be realized in terms of logic gates	Less costlier than hardwired control as only micro instructions are used for generating control signals
It cannot handle complex instructions as the circuit design for it becomes complex	It can handle complex instructions
Only limited number of instructions are used due to the hardware implementation	Control signals for many instructions can be generated
Used in computer that makes use of Reduced Instruction Set Computers(RISC)	Used in computer that makes use of Complex Instruction Set Computers(CISC)

Module 3 – Performance Metrics : Execution Time Calculation, MIPS, MFLOPS

Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

Performance

- Measure Performance : How fast the computer works?
- Time – important metric to measure performance
- A computer exhibits higher performance if it executes programs faster.

Performance Metrics

- **To maximize performance**, we want to minimize response time or execution time for task. Thus, we can relate performance and execution time for a computer X:

$$Performance_X = \frac{1}{Execution\ time_X}$$

- This means that for two computers X and Y, if the performance of X is greater than the performance of Y, we have

$$Performance_X > Performance_Y$$

$$\frac{1}{Execution\ time_X} > \frac{1}{Execution\ time_Y}$$

- That is, the execution time on y is longer than X, so X is faster than Y.

Performance Metrics - Example

EXAMPLE

If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

ANSWER

$$\text{Performance}_A = n \times \text{Performance}_B$$

We know that A is *n* times as fast as B if,

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = n \text{ B}$$

Thus the performance ratio is,

$$\frac{15}{10} = 1.5$$

and A is therefore **1.5 times** as fast as B.

Performance Metrics - Example

EXAMPLE

If a computer X runs a program in 20 seconds and computer Y runs the same program in 10 seconds, how much slower is A than B?

ANSWER

Hint : Performance (X) < Performance (Y)

We know that X is n times as slow as Y if,

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n \text{ X}$$

$$\text{Performance}_X = n \times \text{Performance}_Y$$

Thus the performance ratio is,

$$\frac{10}{20} = 0.5$$

and X is therefore 0.5 times as slow as Y.

Performance Metrics

- For some program running on machine X,

$$\text{Performance} = \frac{1}{\text{Execution time(X)}}$$

- "X is n times faster than Y" – represented as

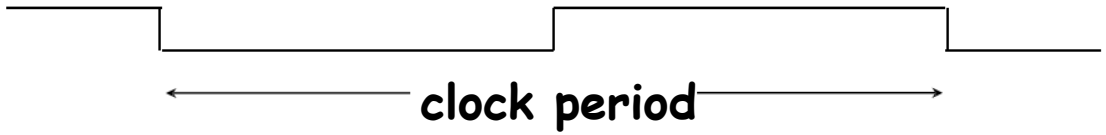
$$\frac{\text{Performance(X)}}{\text{Performance(Y)}} = n$$

- Problem: Machine A runs a program in 20 seconds. Machine B runs the same program in 25 seconds. How many times faster is machine A?

$$\frac{25}{20} = 1.25$$

Computer Clock

- Almost all computers are constructed using a clock that determines when events take place in the hardware.
- These discrete time intervals are called **clock cycles (ticks, clock ticks, clock periods, clocks, cycles).**

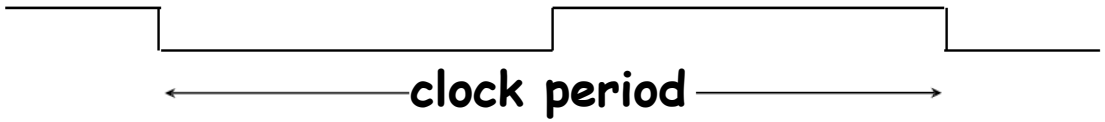


- Designers refer to the length of a **clock period** in time for a complete **clock cycle** (e.g., 250 picoseconds) and as the **clock rate/frequency** (e.g. 4 GHz, 5 MHz), which is the inverse of the clock period.

Computer Clock

- Clock cycle time - the amount of time for one **clock period** to elapse (e.g. 5 ns, 250 picoseconds….).
- Clock rate/frequency- inverse of the clock cycle time.
- For example, if a computer has a clock cycle time of 5 ns, the clock rate is:

$$\frac{1}{5 \times 10^{-9} \text{ sec}} = 200 \text{ MHz}$$



Processor Performance Equation

- Performance Equation: $$CPU\ execution\ time = CPU\ clock\ cycles \times Clock\ cycle\ time$$
- Alternatively

$$CPU\ execution\ time = \frac{CPU\ clock\ cycles}{Clock\ rate}$$

$$CPU\ time = \frac{Instruction\ count \times CPI}{Clock\ rate}$$

- Also, execution time - depend on the number of instructions in a program
- Execution time equals the number of instructions executed multiplied by the average time per instruction. Therefore, the number of clock cycles required for a program can be written as

$$CPU\ clock\ cycles = Instruction\ Count \times CPI$$

$$Instruction\ Count = \frac{CPU\ clock\ cycles}{CPI}$$

- **Clock cycles per instruction (CPI)**, which is the average number of clock cycles for each instruction takes to execute, is often abbreviated as **CPI**.

Processor Performance Equation

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

- To Summarize,

$$\boxed{\textit{CPU clock cycles} = \textit{Instruction Count} \times \textit{CPI}}$$

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

$$\textit{Clock Rate} = \frac{1}{\textit{clock cycle Time}} \qquad \textit{Instruction Count} = \frac{\textit{CPU clock cycles}}{\textit{CPI}}$$

- **CPI** (cycles per instruction)

  A floating point intensive application might have a higher CPI

MIPS & MFLOPS

- MIPS - millions of instructions per second

$$MIPS = \frac{Inst\_Count}{Execution\_Time * 10^6}$$

- MFLOPS - millions of floating-point operations per second

$$MFLOPS = \frac{FP\_Operations}{Execution\_Time * 10^6}$$

Practice Problems

- Example 1:
  - CPU clock rate is 1 MHz
  - Program takes 45 million cycles to execute
  - What's the CPU time?

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

45,000,000 * (1 / 1,000,000) = 45 seconds

Practice Problems

- Example 2:
  - CPU clock rate is 500 MHz
  - Program takes 45 million cycles to execute
  - What's the CPU time?

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

45,000,000 * (1 / 500,000,000) = 0.09 seconds

Practice Problems

- Example 3:

Suppose we have two implementations of the same instruction set architecture (ISA).

  - For some program,
    - Machine A has a clock cycle time of 10 ns. and a CPI of 2.0
    - Machine B has a clock cycle time of 20 ns. and a CPI of 1.2
- Which machine is faster for this program, and by how much?
- Assume that # of instructions in the program is 1,000,000,000.

Practice Problems

- Example 3: Solution

Machine A has a clock cycle time of 10 ns. and a CPI of 2.0
Machine B has a clock cycle time of 20 ns. and a CPI of 1.2
# of instructions - 1,000,000,000.

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

CPU / Execution Time$_A$ = 10^9 * 2.0 * 10 * 10^{-9} = 20 seconds

CPU/ Execution Time$_B$ = 10^9 * 1.2 * 20 * 10^{-9} = 24 seconds

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = n\ B$$

= 24/20 = 1.2 times

Machine A is faster

Practice Problems

- Example 4:

Our favorite program runs in 10 seconds on computer A, which has a 2 GHz clock. We are trying to help a computer designer build a computer, B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate should we tell the designer to target?

Practice Problems

- Example 4: Solution

A – Exec time = 10 sec, Clock rate – 2GHz
B- Exec.time = 6 sec.
Clock cycle of B = 1.2 (clock cycle of A)
Find Clock rate of B?

Let's first find the number of clock cycles required for the program on A:

$$\text{CPU time}_A = \frac{\text{CPU clock cycles}_A}{\text{Clock rate}_A}$$

$$10 \text{ seconds} = \frac{\text{CPU clock cycles}_A}{2 \times 10^9 \frac{\text{cycles}}{\text{second}}}$$

$$\text{CPU clock cycles}_A = 10 \text{ seconds} \times 2 \times 10^9 \frac{\text{cycles}}{\text{second}} = 20 \times 10^9 \text{ cycles}$$

Practice Problems

- Example 4: Solution

A – Exec time = 10 sec, Clock rate – 2GHz
B- Exec.time = 6 sec.
Clock cycle of B = 1.2 (clock cycle of A)
Find Clock rate of B?

CPU time for B can be found using this equation:

$$\text{CPU time}_B = \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B}$$

$$6 \text{ seconds} = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{\text{Clock rate}_B}$$

$$\text{Clock rate}_B = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{6 \text{ seconds}} = \frac{0.2 \times 20 \times 10^9 \text{ cycles}}{\text{second}} = \frac{4 \times 10^9 \text{ cycles}}{\text{second}} = 4 \text{ GHz}$$

To run the program in 6 seconds, B must have twice the clock rate of A.

Practice Problems

- Example 5:

Suppose we have two implementations of the same instruction set architecture. Computer A has a clock cycle time of 250 ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?

Practice Problems

- Example 5: Solution

We know that each computer executes the same number of instructions for the program; let's call this number I. First, find the number of processor clock cycles for each computer:

$$\text{CPU clock cycles}_A = I \times 2.0$$
$$\text{CPU clock cycles}_B = I \times 1.2$$

Now we can compute the CPU time for each computer:

$$\begin{aligned}\text{CPU time}_A &= \text{CPU clock cycles}_A \times \text{Clock cycle time} \\ &= I \times 2.0 \times 250 \text{ ps} = 500 \times I \text{ ps}\end{aligned}$$

Likewise, for B:

$$\text{CPU time}_B = I \times 1.2 \times 500 \text{ ps} = 600 \times I \text{ ps}$$

Clearly, computer A is faster. The amount faster is given by the ratio of the execution times:

$$\frac{\text{CPU performance}_A}{\text{CPU performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{600 \times I \text{ ps}}{500 \times I \text{ ps}} = 1.2$$

We can conclude that computer A is 1.2 times as fast as computer B for this program.

Average Cycles per Instruction

$$CPU\ clock\ cycles = \sum_{i=1}^{n} (CPI_i \times C_i)$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

- Total CPI= $\frac{\sum_{i=1}^{n} IC_i * CPIi}{\text{Instruction count(Ic)}}$

Practice Problems

- Example 6:

A compiler designer is trying to decide between two code sequences for a particular computer. The hardware designers have supplied the following facts:

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

For a particular high-level language statement, the compiler writer is considering two code sequences that require the following instruction counts:

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

Which code sequence executes the most instructions? Which will be faster? What is the CPI for each sequence?

Practice Problems

- Example 6: Solution

Sequence 1 executes 2 + 1 + 2 = 5 instructions. Sequence 2 executes 4 + 1 + 1 = 6 instructions. Therefore, sequence 1 executes fewer instructions.

We can use the equation for CPU clock cycles based on instruction count and CPI to find the total number of clock cycles for each sequence:

$$\text{CPU clock cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{C}_i)$$

B is faster

Since Clock cycle time is not given, we can estimate using CPU clock cycles itself

This yields

$$\text{CPU clock cycles}_1 = (2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10 \text{ cycles}$$

$$\text{CPU clock cycles}_2 = (4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9 \text{ cycles}$$

Practice Problems

- Example 6: Solution

So code sequence 2 is faster, even though it executes one extra instruction. Since code sequence 2 takes fewer overall clock cycles but has more instructions, it must have a lower CPI. The CPI values can be computed by

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}}$$

$$\text{CPI}_1 = \frac{\text{CPU clock cycles}_1}{\text{Instruction count}_1} = \frac{10}{5} = 2.0$$

$$\text{CPI}_2 = \frac{\text{CPU clock cycles}_2}{\text{Instruction count}_2} = \frac{9}{6} = 1.5$$

Practice Problems

- Example 7:

EXAMPLE

1. **Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.**
   a. **Which processor has the highest performance expressed in instructions per second?**
   b. **If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.**

ANSWER

a. **Which processor has the highest performance expressed in instructions per second?**

CPU execution time can be calculated using the following formula:

$$CPU\ time = \frac{Instruction\ count \times CPI}{Clock\ rate}$$

Let the instruction count be 'n' and assume it is same for all the three processors. Now calculate the CPU time for each processor.

$$CPU\ time_{P1} = \frac{n \times 1.5}{3 \times 10^9} = n \times 0.5 \times 10^{-9}\ sec$$

$$CPU\ time_{P2} = \frac{n \times 1.0}{2.5 \times 10^9} = n \times 0.4 \times 10^{-9}\ sec$$

$$CPU\ time_{P3} = \frac{n \times 2.2}{4 \times 10^9} = n \times 0.55 \times 10^{-9}\ sec$$

CPU time for processor P2 is smallest than other processor. Hence, P2 produces highest performance than other processors.

Practice Problems

- Example 7: Solution

(ii) Instruction Count

The formula for finding CPU clock cycles can be rewritten to find the instruction count.

$$CPU\ clock\ cycles = Instruction\ Count \times CPI$$

$$Instruction\ Count = \frac{CPU\ clock\ cycles}{CPI}$$

$$Instruction\ count_{P1} = \frac{30 \times 10^9}{1.5} = \mathbf{20 \times 10^9\ instructions}$$

$$Instruction\ count_{P2} = \frac{25 \times 10^9}{1.0} = \mathbf{25 \times 10^9\ instructions}$$

$$Instruction\ count_{P1} = \frac{40 \times 10^9}{2.2} = \mathbf{18.18 \times 10^9\ instructions}$$

b. If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.

(i) Number of clock cycles

CPU time for P1, P2 and P3 = 10 sec

The formula for finding CPU time can be rewritten to find the number of clock cycles.

$$CPU\ time = \frac{CPU\ Clock\ cycles}{Clock\ rate}$$

$$CPU\ Clock\ cycles = CPU\ time \times Clock\ rate$$

$$CPU\ Clock\ cycles_{P1} = 10\ sec \times 3 \times 10^9\ Hz = \mathbf{30 \times 10^9\ cycles}$$

$$CPU\ Clock\ cycles_{P2} = 10\ sec \times 2.5 \times 10^9\ Hz = \mathbf{25 \times 10^9\ cycles}$$

$$CPU\ Clock\ cycles_{P3} = 10\ sec \times 4 \times 10^9\ Hz = \mathbf{40 \times 10^9\ cycles}$$

With Frequency

$$CPU\ clock\ cycles = \sum_{i=1}^{n} (CPI_i \times C_i)$$

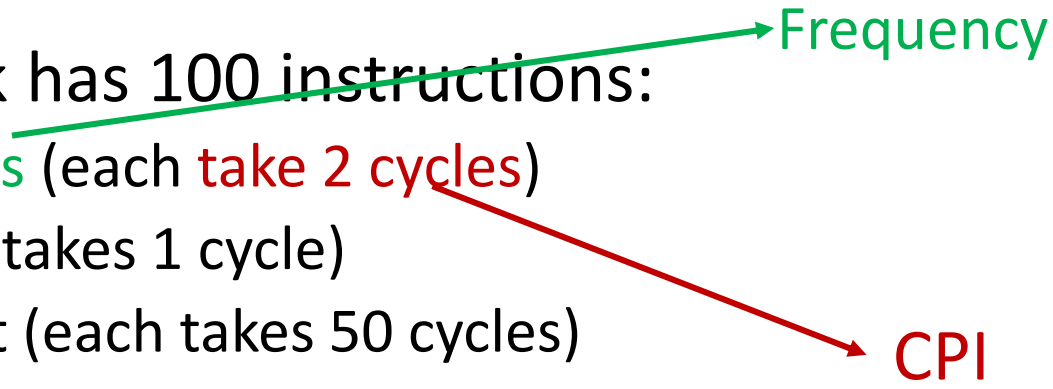
- CPI= $\frac{\text{CPU clock cycles(c)}}{\text{Instruction count(Ic)}}$
- Total CPI= $\frac{\sum_{i=1}^{n} IC_i * CPIi}{\text{Instruction count(Ic)}}$

Op	F	CPI	CPI x F	% Time
ALU	50%	1	.5	23%
Load	20%	5	1.0	45%
Store	10%	3	.3	14%
Branch	20%	2	.4	18%
Total	100%		2.2	100%

- The CPI is the average number of cycles per instruction.
- If for each instruction type, we know its frequency and number of cycles need to execute it, we can compute the overall CPI as follows:
- CPI= $\sum_{i=1}^{n} \text{Freq}_i * CPI_i$

Practice Problems

- Example 8: Solution
- Let assume that a benchmark has 100 instructions:
  - 25 instructions are loads/stores (each take 2 cycles)
  - 50 instructions are adds (each takes 1 cycle)
  - 25 instructions are square root (each takes 50 cycles)
- What is the CPI for this benchmark?



$$CPI = ((0.25 * 2) + (0.50 * 1) + (0.25 * 50))$$
$$= 13.5$$

Practice Problems

- Example 9:

Two different compilers are being tested for a 500 MHz. machine with three different classes of instructions: Class A, Class B, and Class C, which require one, two, and three cycles (respectively).

Both compilers are used to produce code for a large piece of software.

The first compiler's code uses 5 billions Class A instructions, 1 billion Class B instructions, and 1 billion Class C instructions.

The second compiler's code uses 10 billions Class A instructions, 1 billion Class B instructions, and 1 billion Class C instructions.

- Which sequence will be faster according to MIPS?
- Which sequence will be faster according to execution time?

Practice Problems

- Example 9: Solution

	Instruction counts (in billions) for each instruction class		
Code from	A	B	C
Compiler 1	5	1	1
Compiler 2	10	1	1

Clock rate – 500 MHz
Cycles – class1-1 cycle,
class2-2 cycles,
class3-3 cycles.

$$CPU\ clock\ cycles = Instruction\ Count \times CPI$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$MIPS = \frac{Inst\_Count}{Execution\_Time * 10^6}$$

$\text{CPU Clock cycles}_1 = (5 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 10 \times 10^9$

$\text{CPU Clock cycles}_2 = (10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 15 \times 10^9$

$\text{CPU time}_1 = 10 \times 10^9 / 500 \times 10^6 = 20 \text{ seconds}$

$\text{CPU time}_2 = 15 \times 10^9 / 500 \times 10^6 = 30 \text{ seconds}$

$\text{MIPS}_1 = (5 + 1 + 1) \times 10^9 / 20 \times 10^6 = 350$

$\text{MIPS}_2 = (10 + 1 + 1) \times 10^9 / 30 \times 10^6 = 400$

Problems with Arithmetic Mean

- For example, two machines timed on two benchmarks:

	Machine A	Machine B
Program 1	2 seconds (20%)	6 seconds (20%)
Program 2	12 seconds (80%)	10 seconds (80%)

Average execution time$_A$ = (2 + 12) / 2 = 7 seconds

Average execution time$_B$ = (6 + 10) / 2 = 8 seconds

Weighted average execution time$_A$ = 2*0.2 + 12*0.8 = 10 seconds

Weighted average execution time$_B$ = 6*0.2 + 10*0.8 = 9.2 seconds

MIPS rate

- A common measure of performance for a processor is the rate at which instructions are executed, expressed as millions of instructions per second (MIPS), referred to as the **MIPS rate**.

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

- Ic → Instruction count
- T→ CPU time
- f→Clock rate
- CPI→ Cycles Per Instruction

$$\frac{Ic * f}{\text{CPU clock cycles} * 10^6}$$

Practice Problems

- Example 10:

A 40 MHz processor was used to execute a benchmark program with the following instruction mix and clock cycle count:

Instruction TYPE	Instruction count	Clock cycle count (CPI)
Integer Arithmetic	45000	1
Data transfer	32000	2
Floating point	15000	2
Control transfer	8000	2

Determine the

1. Effective CPI,
2. MIPS (Millions of instructions per second) rate, and
3. Execution time for this program

1. Effective CPI

$$\text{Effective CPI} = \frac{\text{Total clock cycle needed to execute all instructions in the program}}{\text{Instruction count of the entire program}}$$

$$\textbf{Effective CPI} = \frac{\sum_{i=1}^{n}(CPI_i * I_{ci})}{I_c}$$

Given, Total no. of instructions i.e **Instruction Count (Ic) = 45000+32000+15000+8000**

= 100000

$$CPI = \frac{(45000 \times 1)+(32000 \times 2)+(15000 \times 2)+(8000 \times 2)}{100000}$$

$$= \frac{45000+64000+30000+16000}{100000}$$

$$= \frac{155000}{100000}$$

$$= 1.55$$

2. MIPS

Given, Clock rate f = 40 MHz = 40 * 10^6 Hz

$$MIPS = \frac{f}{CPI * 10^6} = \frac{Clock\ frequency}{CPI * 10^6}$$

$$MIPS = \frac{40 \times 10^6}{1.55 \times 10^6}$$

$$= \frac{40}{1.55}$$

$$= 25.8$$

3. Execution Time (T)

$$T = \frac{Ic * CPI}{f}$$

$$T = \frac{100000 \times 1.55}{40 \times 10^6}$$

$$= \frac{155000}{40000000}$$

$$= 0.003875$$

$$= 3.875\ ms$$

Practice Problems

- Example 11:

Consider the execution of an object code with 200,000 instructions on a 40-MHz processor. The program consists of four major types of instructions. The instruction mix and the number of cycles (CPI - Cycles Per Instruction) needed for each instruction type are given below based on the result of a program trace experiment

Instruction TYPE	CPI	Instruction Mix
Arithmetic and Logic Operations	1	60%
Load/Store with cache hit	2	18%
Branch	4	12%
Memory reference with cache miss	8	10%

Calculate the average CPI when the program is executed on a uniprocessor with the above trace results.

Calculate the corresponding MIPS rate based on the CPI obtained in part (a)

Calculate the Execution Time

Given, **Instruction count (Ic) = 200000= 2* 10^5**, and **Clock rate (f) = 40 MHz = 40 * 10^6 Hz**

- **No. of Arithmetic and Logic Operations** = 60% of 2 * 10^5 = 60/100 * 2* 10^5 = **120 * 10^3**
- **No. of Load/Store with cache hit** = 18% of 2 * 10^5 = 18/100 * 2* 10^5 = **36 * 10^3**
- **No. of Branch instructions** = 12% of 2* 10^5 = 12/100 * 2* 10^5 = **24 * 10^3**
- **No. Memory reference with cache miss** = 10% of 2* 10^5 = 10/100 * 2 * 10^5 = **20 * 10^3**

a) Average CPI $= \dfrac{(120*1 + 36*2 + 24*4 + 20*8) * 10^3}{(120 + 36 + 24 + 20) * 10^3}$

$= \dfrac{448}{220}$

$= 2.24$

$= \dfrac{\sum_{i=1}^{n} (CPI_i * I_{ci})}{I_c}$

b) MIPS Rate

$$MIPS = \frac{f}{CPI * 10^6} = \frac{Clock\ frequency}{CPI * 10^6}$$

$= \dfrac{40 * 10^6}{2.24 * 10^6}$

$= 17.86$

c) Execution Time (T)

Given, Clock rate f = 40 MHz = 40 * 10^6 Hz

Instruction Count Ic = 200000= 2* 10^5

$$T = \frac{Ic * CPI}{f}$$

$= \dfrac{2 * 10^5 * 2.24}{40 * 10^6}$

$= 0.0112$

$= 11.2$ ms

Practice Problems

- Example 12:

Assume that a benchmark has 100 instructions with the clock rate of 300Mhz. 20% instructions are loads/stores (each take 3 cycles), 40% instructions are adds (each takes 2 cycles), and 40% instructions are square root (each takes 60 cycles), what is the CPI and MIPS rate for this benchmark?

Ans: CPI=25.4 , MIPS=11.8

MFLOPS

- Floating-point performance is expressed as millions of floating-point operations per second (MFLOPS), defined as follows:

$$\text{MFLOPS rate} = \frac{\text{Number of executed floating} - \text{point operations in a program}}{\text{Execution time} \times 10^6}$$